

Java Fundamentals — Подробные ответы

1. JVM, JDK, JRE — что это такое, кто кого включает и как взаимодействуют

Начнём с самого фундаментального. В экосистеме Java существуют три ключевых компонента, которые часто путают, но каждый из них выполняет свою чётко определённую роль.

JVM (Java Virtual Machine) — это виртуальная машина Java, абстрактная вычислительная машина, которая обеспечивает среду выполнения для байт-кода Java. JVM не понимает исходный код Java напрямую — она работает исключительно с байт-кодом, который представляет собой промежуточное представление программы в виде файлов с расширением `.class`. Главная задача JVM — взять байт-код и выполнить его на конкретной платформе (Windows, Linux, macOS и так далее). Именно благодаря JVM достигается знаменитый принцип "Write Once, Run Anywhere" (напиши один раз — запускай где угодно), потому что один и тот же байт-код может исполняться на любой платформе, для которой существует реализация JVM. Внутри JVM работают такие компоненты, как загрузчик классов (ClassLoader), область памяти (Runtime Data Areas), движок исполнения (Execution Engine), включающий интерпретатор и JIT-компилятор, а также сборщик мусора (Garbage Collector).

JRE (Java Runtime Environment) — это среда выполнения Java, которая включает в себя JVM плюс набор стандартных библиотек классов (Java Class Library), необходимых для запуска Java-программ. Если вы просто хотите запустить уже скомпилированную Java-программу, вам достаточно JRE. JRE содержит реализацию JVM, стандартные библиотеки (такие как `java.lang`, `java.util`, `java.io` и другие), а также вспомогательные файлы и ресурсы. Однако JRE не содержит инструментов разработки — в ней нет компилятора, отладчика и других утилит для создания программ. Начиная с Java 11 компания Oracle перестала распространять JRE как отдельный продукт, объединив всё в JDK.

JDK (Java Development Kit) — это полный набор инструментов для разработки на Java, который включает в себя JRE (а значит, и JVM) плюс инструменты разработчика. Среди этих инструментов: компилятор `javac`, который преобразует исходный код `.java` в байт-код `.class`; утилита `java` для запуска программ; отладчик `jdb`; инструмент создания документации `javadoc`; архиватор `jar` для упаковки классов в JAR-файлы; и множество других утилит. Если вы разрабатываете программы на Java, вам нужен именно JDK.

Таким образом, вложенность следующая: JDK включает JRE, а JRE включает JVM. То есть JDK — это самый большой набор, JRE — подмножество JDK, а JVM — подмножество JRE. Взаимодействие между ними выглядит следующим образом: разработчик пишет исходный код, используя инструменты JDK, компилятор `javac` из JDK превращает этот код в байт-код, затем JVM из состава JRE загружает этот байт-код, верифицирует его, интерпретирует или компилирует в машинный код через JIT-компилятор и выполняет, используя при этом стандартные библиотеки из JRE.

2. Как скомпилировать и запустить класс, используя консоль

Предположим, у нас есть файл `HelloWorld.java` со следующим содержимым:

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Для компиляции необходимо открыть терминал (командную строку), перейти в директорию, где находится файл, и выполнить команду `javac HelloWorld.java`. Утилита `javac` — это компилятор Java, который входит в состав JDK. После выполнения этой команды в той же директории появится файл `HelloWorld.class`, содержащий байт-код. Если в исходном коде есть синтаксические ошибки, компилятор выведет сообщения об ошибках и файл

.class не будет создан.

Для запуска скомпилированного класса используется команда `java HelloWorld` — обратите внимание, что расширение `.class` указывать не нужно, указывается только имя класса. Утилита `java` запускает JVM, которая загружает указанный класс, находит в нём метод `main` и начинает выполнение программы.

Если класс находится в пакете, например `package com.example;`, то файл должен находиться в соответствующей структуре директорий `com/example/HelloWorld.java`. Компиляция в этом случае выполняется из корневой директории командой `javac com/example/HelloWorld.java`, а запуск — `java com.example.HelloWorld`, где указывается полное квалифицированное имя класса с точками вместо слэшей.

Также можно использовать дополнительные флаги: например, `javac -d out HelloWorld.java` скомпилирует класс и поместит результат в директорию `out`, а `java -cp out HelloWorld` запустит класс с указанием `classpath`. Начиная с Java 11 появилась возможность запуска однофайловых программ без явной компиляции: достаточно выполнить `java HelloWorld.java`, и JVM сама скомпилирует и запустит файл в одном шаге — это удобно для быстрого тестирования и обучения.

3. JIT-компилятор

JIT-компилятор (Just-In-Time compiler) — это компонент движка исполнения JVM, который значительно повышает производительность Java-программ. Чтобы понять его роль, нужно сначала разобраться в общей схеме исполнения кода в JVM.

Когда JVM загружает байт-код, она может исполнять его двумя способами: интерпретировать или компилировать в машинный (нативный) код. Интерпретатор читает байт-код инструкция за инструкцией и выполняет каждую из них, транслируя в соответствующие машинные операции. Это

относительно медленный процесс, потому что каждая инструкция транслируется заново при каждом выполнении, даже если один и тот же участок кода вызывается тысячи раз.

Именно здесь вступает в дело JIT-компилятор. JVM отслеживает, какие участки кода выполняются часто — их называют "горячими точками" (hot spots, отсюда, кстати, и название одной из реализаций JVM — HotSpot). Когда количество вызовов определённого метода или итераций цикла превышает определённый порог, JIT-компилятор берёт байт-код этого участка и компилирует его в нативный машинный код целевой платформы. После этого при последующих вызовах используется уже скомпилированный машинный код, который выполняется значительно быстрее, чем интерпретация.

JIT-компилятор также выполняет различные оптимизации: инлайнинг методов (встраивание тела маленьких методов прямо в место вызова, чтобы избежать накладных расходов на вызов), удаление мёртвого кода, развёртывание циклов, оптимизация ветвлений на основе профилирования и многие другие. Причём JIT-компилятор может делать оптимизации, которые недоступны обычным статическим компиляторам (как в C++), потому что он обладает информацией о реальном поведении программы в рантайме — например, он знает, какие ветви условий выполняются чаще, какие типы объектов реально передаются в полиморфные вызовы и так далее.

В HotSpot JVM существует два уровня JIT-компиляции: C1 (клиентский компилятор), который быстро компилирует код с базовыми оптимизациями, и C2 (серверный компилятор), который тратит больше времени на компиляцию, но применяет более агрессивные оптимизации. Современные JVM используют tiered compilation — сначала код интерпретируется, затем компилируется C1 с профилированием, а когда накопится достаточно информации о поведении кода, он перекомпилируется C2 с максимальными оптимизациями. Также существует относительно новый JIT-компилятор Graal, написанный на Java, который в некоторых случаях может давать ещё лучшую производительность.

4. Области памяти JVM: где размещаются простые типы, объекты, ссылки, константы, методы, пул строк

Внутренняя структура памяти JVM — это одна из ключевых тем для понимания работы Java. Память JVM делится на несколько основных областей, каждая из которых служит своей цели.

Heap (куча) — это самая большая область памяти, которая выделяется при запуске JVM и является общей для всех потоков. Именно здесь размещаются все объекты, создаваемые с помощью оператора `new`, а также массивы. Куча управляется сборщиком мусора (Garbage Collector), который автоматически освобождает память от объектов, на которые больше нет ссылок. Куча обычно делится на поколения: Young Generation (молодое поколение, куда попадают вновь созданные объекты, подразделяющееся на Eden Space и два Survivor Space) и Old Generation (старое поколение, куда перемещаются объекты, пережившие несколько циклов сборки мусора). Начиная с Java 8, вместо Permanent Generation появилось Metaspace, но об этом ниже.

Stack (стек) — каждый поток в Java имеет свой собственный стек, который создаётся одновременно с потоком. Стэк хранит фреймы (stack frames), каждый из которых соответствует одному вызову метода. Внутри фрейма хранятся: локальные переменные метода (включая примитивные типы и ссылки на объекты), параметры метода, промежуточные результаты вычислений и адрес возврата. Важно понимать: сами примитивные типы, объявленные как локальные переменные, хранятся непосредственно в стеке (целочисленные значения, числа с плавающей точкой, булевы значения и так далее). Ссылки на объекты тоже хранятся в стеке, но сами объекты, на которые эти ссылки указывают, находятся в куче. Когда метод завершается, его фрейм удаляется из стека. Если стек переполняется (например, при бесконечной рекурсии), выбрасывается `StackOverflowError`.

Metaspace (метапространство) — область памяти, появившаяся в Java 8 взамен Permanent Generation. В метапространстве хранятся метаданные

классов: информация о загруженных классах, структура классов, информация о методах, полях, аннотациях, а также байт-код методов. Важное отличие от Permanent Generation заключается в том, что Metaspase использует нативную память операционной системы, а не память кучи JVM, и по умолчанию может расти динамически (хотя можно ограничить его размер флагом `-XX:MaxMetaspaceSize`).

Program Counter Register (регистр счётчика программы) — каждый поток имеет свой собственный PC-регистр, который указывает на адрес текущей выполняемой инструкции байт-кода. Если поток выполняет нативный метод, значение PC-регистра не определено.

Native Method Stack (стек нативных методов) — стек для выполнения нативных методов, написанных на C или C++ и вызываемых через JNI (Java Native Interface).

Пул строк (String Pool) — это специальная область, которая до Java 7 находилась в Permanent Generation, а начиная с Java 7 была перемещена в кучу (heap). Строковый пул хранит интернированные строки — то есть уникальные экземпляры строковых литералов. Когда вы пишете `String s = "hello";`, JVM сначала проверяет, есть ли строка "hello" в пуле, и если есть — возвращает ссылку на существующий объект, а если нет — создаёт новый и помещает в пул. Это позволяет экономить память, когда одинаковые строки используются многократно. Метод `intern()` позволяет принудительно поместить строку в пул.

Constant Pool (пул констант) — это часть метаданных каждого класса, хранящаяся в Metaspase. Пул констант класса содержит литералы (числовые значения, строковые литералы) и символические ссылки (имена классов, методов, полей), которые были определены на этапе компиляции. При загрузке класса JVM разрешает символические ссылки, связывая их с реальными адресами в памяти.

Что касается полей объектов: если объект имеет поля примитивных типов, эти значения хранятся прямо внутри объекта в куче (как часть его внутренней структуры). Если поле является ссылочным типом, то в объекте хранится ссылка, а объект, на который она указывает, находится где-то ещё в куче. Статические переменные класса также хранятся в куче (ассоциированы с объектом `java.lang.Class` соответствующего класса).

5. Пакеты в Java: зачем применяются и принцип именования

Пакеты (`packages`) в Java — это механизм группировки связанных классов, интерфейсов, перечислений и аннотаций в логические единицы. Пакет объявляется в первой строке исходного файла с помощью ключевого слова `package`, например `package com.example.myapp.model;`.

Зачем применяются пакеты? Во-первых, они обеспечивают логическую организацию кода. В крупном проекте могут быть сотни и тысячи классов, и без пакетов было бы невозможно поддерживать структуру проекта. Классы группируются по функциональности: классы модели данных в один пакет, классы сервисов в другой, утилиты в третий и так далее. Во-вторых, пакеты предотвращают конфликты имён. Два разных разработчика могут создать класс с именем `User`, но если один находится в пакете `com.company1.model`, а другой в `com.company2.model`, они не конфликтуют, потому что их полные квалифицированные имена различаются. В-третьих, пакеты обеспечивают контроль доступа. В Java существует уровень доступа по умолчанию (`package-private`), при котором класс или его член доступен только из того же пакета — это мощный инструмент инкапсуляции. В-четвёртых, пакеты упрощают поиск и повторное использование кода, поскольку физическая структура директорий на диске соответствует структуре пакетов.

Принцип именования пакетов основан на конвенции обратного доменного имени. Идея в том, чтобы гарантировать уникальность имён пакетов в глобальном масштабе. Если ваша компания владеет доменом `example.com`, то пакеты именуются как `com.example`, после чего добавляются подпакеты по проекту и функциональности: `com.example.projectname.module`. Имена

пакетов пишутся строчными буквами, без подчёркиваний и прочих символов. Стандартные библиотеки Java используют пакеты, начинающиеся с `java` (например, `java.util`, `java.io`) и `javax` (например, `javax.swing`). Важно также понимать, что структура пакетов напрямую отображается на структуру директорий файловой системы: класс в пакете `com.example.model` должен лежать в директории `com/example/model/`.

6. Отличия между ссылочными и примитивными типами

В Java существует фундаментальное разделение всех типов данных на две категории: примитивные и ссылочные, и понимание различий между ними критически важно.

Примитивных типов в Java ровно восемь: `byte` (8 бит), `short` (16 бит), `int` (32 бита), `long` (64 бита), `float` (32 бита), `double` (64 бита), `char` (16 бит, символ Unicode), `boolean` (размер зависит от реализации JVM). Примитивные типы хранят непосредственно значение. Когда вы пишете `int x = 42;`, переменная `x` содержит само число 42. Примитивные типы не являются объектами, у них нет методов, они не могут быть `null`, и они всегда имеют значение по умолчанию (0 для числовых типов, `false` для `boolean`, `'\u0000'` для `char`). Примитивные типы размещаются в стеке, если это локальные переменные, или внутри объекта в куче, если это поля экземпляра.

Ссылочные типы — это все остальные типы: классы, интерфейсы, массивы и перечисления. Переменная ссылочного типа хранит не сам объект, а ссылку (по сути, указатель) на объект, расположенный в куче. Когда вы пишете `String s = "hello";`, переменная `s` содержит ссылку на объект строки в куче, а не саму строку. Ссылочные типы могут принимать значение `null`, что означает, что переменная не указывает ни на какой объект. У ссылочных типов есть методы, которые можно вызывать, все они наследуют от класса `Object`.

При передаче параметров в метод поведение также различается. В Java всё передаётся по значению. Для примитивных типов это означает, что метод получает копию значения, и изменение параметра внутри метода не влияет на

оригинал. Для ссылочных типов передаётся копия ссылки — метод получает ту же ссылку на тот же объект, поэтому может изменить состояние объекта, но не может изменить саму оригинальную ссылку (сделать так, чтобы она указывала на другой объект).

Сравнение тоже работает по-разному: для примитивов оператор `==` сравнивает значения, а для ссылочных типов `==` сравнивает ссылки (указывают ли они на один и тот же объект в памяти), поэтому для сравнения содержимого объектов используется метод `equals()`.

Примитивные типы не могут быть использованы в коллекциях (например, `ArrayList<int>` невозможен), для этого существуют классы-обёртки. Примитивные типы, как правило, более эффективны по производительности и потреблению памяти, потому что не имеют накладных расходов на заголовок объекта и ссылку.

7. Модификаторы доступа. Что такое `package level access`. Пример использования

В Java существует четыре уровня доступа, определяемых тремя ключевыми словами (и отсутствием ключевого слова для четвёртого уровня).

`public` — самый открытый модификатор. Класс, метод или поле, объявленные как `public`, доступны из любого места программы, из любого пакета. Если класс объявлен `public`, он доступен отовсюду. Если метод `public`, его можно вызвать из любого другого класса.

`protected` — элемент с этим модификатором доступен внутри того же пакета (как `package-private`), а также из подклассов, даже если подкласс находится в другом пакете. Этот модификатор используется, когда вы хотите предоставить доступ наследникам, но скрыть элемент от остального мира. Модификатор `protected` применяется только к членам класса (методам и полям), но не к классам верхнего уровня.

Package-private (default access, package level access) — это уровень доступа, который действует, когда не указан никакой модификатор доступа. Элемент с таким уровнем доступен только из классов того же пакета. Это очень полезный механизм инкапсуляции, который часто недооценивается. Он позволяет создать группу тесно связанных классов внутри одного пакета, которые могут свободно общаться друг с другом, но при этом их внутренняя реализация скрыта от внешнего мира.

Пример использования package level access: допустим, у вас есть пакет com.example.engine, в котором находится публичный класс GameEngine и вспомогательный класс PhysicsCalculator. Класс PhysicsCalculator используется только внутри пакета engine, поэтому его можно объявить без модификатора доступа: `class PhysicsCalculator { ... }`. Это гарантирует, что классы из других пакетов не смогут создать его экземпляр или использовать его напрямую, а GameEngine внутри того же пакета имеет к нему полный доступ. Аналогично, если у публичного класса есть метод, который является частью внутренней логики и нужен другим классам того же пакета, но не должен быть виден клиентам, его можно объявить без модификатора: `void calculateInternal() { ... }`.

private — самый закрытый модификатор. Элемент доступен только внутри того же класса, где он объявлен. Даже подклассы не имеют к нему доступа. Этот модификатор используется для полной инкапсуляции внутренних деталей реализации. Обычно поля класса объявляются private, а доступ к ним предоставляется через публичные геттеры и сеттеры.

Для классов верхнего уровня (не вложенных) допускается только public или package-private. Вложенные классы могут иметь любой модификатор доступа.

8. Почему метод main() объявлен как public static void? Как main выглядит в Java 25?

Каждое ключевое слово в объявлении `public static void main(String[] args)` имеет свою причину.

`public` — метод `main` должен быть публичным, потому что он вызывается JVM извне класса. JVM не является частью вашего кода и не находится в том же пакете, что и ваш класс, поэтому ей нужен публичный доступ к точке входа. Если бы метод был `private` или `package-private`, JVM просто не смогла бы его вызвать.

`static` — метод должен быть статическим, потому что JVM вызывает его без создания экземпляра класса. Когда программа только запускается, никаких объектов ещё не существует. JVM должна иметь возможность вызвать метод `main` напрямую через класс, без необходимости сначала создавать объект этого класса с помощью конструктора. Если бы `main` не был статическим, JVM пришлось бы сначала создать экземпляр класса, а для этого нужно было бы знать, какой конструктор вызвать, с какими аргументами — это создало бы курицу-и-яйцо проблему.

`void` — метод не возвращает значения, потому что после завершения метода `main` программа завершается, и возвращать значение некому. В языках вроде C функция `main` возвращает `int` как код завершения для операционной системы, но в Java для этой цели используется метод `System.exit(int code)`, поэтому возвращаемое значение не нужно.

`String[] args` — параметр представляет собой массив строк, содержащий аргументы командной строки, переданные при запуске программы. Например, если вы запускаете `java MyApp hello world`, то `args[0]` будет `"hello"`, а `args[1]` будет `"world"`.

Как `main` выглядит в Java 25 (и начиная с Java 21)? В рамках проекта по упрощению входа в язык (JEP 477 — `Implicitly Declared Classes and Instance Main Methods`, ставший финальной фичей) Java значительно упростила написание простых программ. Теперь метод `main` может быть нестатическим, может не принимать параметров, а класс может быть неявно объявленным. То есть файл программы может выглядеть так:

Java

```
void main() {  
    System.out.println("Hello, World!");  
}
```

Здесь нет явного объявления класса, метод не статический, не публичный и не принимает аргументов. JVM будет искать метод `main` в следующем приоритете: `static void main(String[] args)`, затем `static void main()`, затем нестатический `void main(String[] args)`, затем нестатический `void main()`. Для нестатического варианта JVM сама создаёт экземпляр класса и вызывает метод. Также в неявно объявленных классах автоматически импортируется модуль `java.base`, что даёт доступ ко всему из `java.io`, `java.util` и так далее без явного импорта. Это сделано для того, чтобы снизить порог входа для начинающих программистов и убрать "церемониальный" код.

9. Массив — класс или примитивный тип?

Массив в Java — это не примитивный тип и не обычный класс в привычном понимании. Массив является объектом, но особого рода. Формально массив — это объект, который наследует от класса `java.lang.Object`, поэтому у массивов есть все методы `Object` (такие как `toString()`, `hashCode()`, `equals()`, `clone()`, `getClass()`), и переменная массива является ссылочным типом. Массив размещается в куче, как и любой объект, и на него хранится ссылка.

Однако массивы не являются экземплярами обычного класса, определённого программистом или в стандартной библиотеке. Классы массивов создаются динамически самой JVM во время выполнения. Например, класс `int[]` создаётся JVM автоматически, и вы нигде не найдёте его исходный код. Имя такого класса можно увидеть, вызвав `int[].class.getName()` — результатом будет `"[I"`, а для `String[]` — `"[Ljava.lang.String;"`.

Массив имеет специальное поле `length` (это не метод, а именно поле), которое хранит длину массива и является `public final`. Для создания массива

используется оператор `new` (например, `int[] arr = new int[10];`) или литерал массива (`int[] arr = {1, 2, 3};`), а для доступа к элементам — оператор индексации `[]`, который является частью синтаксиса языка, а не обычным вызовом метода. Если попытаться обратиться к индексу за пределами массива, будет выброшено исключение `ArrayIndexOutOfBoundsException`.

Также массив реализует интерфейс `Cloneable` и интерфейс `java.io.Serializable`. Массивы объектов являются ковариантными: если `Dog` наследует от `Animal`, то `Dog[]` считается подтипом `Animal[]`, что может приводить к `ArrayStoreException` при попытке положить объект неподходящего типа.

Итого: массив — это объект (не примитив), но специальный объект, класс которого создаётся JVM динамически и не является обычным классом.

10. Классы-обёртки. Integer cache (пул чисел). Отличия от примитивных типов

Классы-обёртки (wrapper classes) — это классы из пакета `java.lang`, которые предоставляют объектное представление для каждого из восьми примитивных типов: `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character`, `Boolean`. Они нужны потому, что в ряде ситуаций Java требует именно объект, а не примитив. Например, коллекции (`ArrayList`, `HashMap` и другие) работают только с объектами и не могут хранить примитивы — нельзя написать `ArrayList<int>`, нужно писать `ArrayList<Integer>`. Также обёртки используются, когда нужно передать `null` (примитив не может быть `null`), при работе с рефлексией, с generics и в других ситуациях.

Классы-обёртки являются неизменяемыми (`immutable`) и финальными (`final`). Это значит, что однажды созданный объект-обёртка не может быть изменён — любая операция создаёт новый объект. И от них нельзя наследоваться.

`Integer cache` (пул чисел) — это важная оптимизация, реализованная в JVM. Для типа `Integer` JVM кеширует объекты для значений в диапазоне от -128 до 127 (включительно). Это означает, что когда вы используете автобоксинг или

метод `Integer.valueOf(int)` для числа в этом диапазоне, JVM не создаёт новый объект, а возвращает ссылку на уже существующий кешированный объект. Поэтому `Integer a = 127; Integer b = 127; a == b` вернёт `true` (обе ссылки указывают на один и тот же кешированный объект), а `Integer a = 128; Integer b = 128; a == b` вернёт `false` (каждый раз создаётся новый объект, потому что 128 выходит за границы кеша). Верхнюю границу кеша можно изменить с помощью JVM-флага `-XX:AutoBoxCacheMax`, но нижняя граница всегда -128. Аналогичный кеш существует для `Byte`, `Short`, `Long` (все в диапазоне -128..127), `Character` (0..127) и `Boolean` (оба значения `TRUE` и `FALSE`).

Отличия классов-обёрток от примитивных типов: обёртки являются объектами, хранятся в куче и имеют дополнительные накладные расходы по памяти (заголовок объекта занимает 12-16 байт, плюс само значение, плюс выравнивание — итого объект `Integer` занимает 16 байт вместо 4 байт для `int`). Обёртки могут быть `null`, примитивы — нет. Обёртки сравниваются по ссылке оператором `==` и по значению методом `equals()`, примитивы сравниваются только по значению. Обёртки работают медленнее из-за создания объектов и дополнительного уровня косвенности. Но обёртки необходимы для работы с коллекциями, `generics` и `API`, которые требуют объекты.

11. Autoboxing и unboxing — принцип действия на примерах

Автоупаковка (`autoboxing`) и автораспаковка (`unboxing`) — это механизм автоматического преобразования между примитивными типами и соответствующими классами-обёртками, введённый в Java 5 для удобства программистов.

`Autoboxing` — это автоматическое преобразование примитивного типа в соответствующий объект-обёртку. Когда компилятор видит, что примитив используется в контексте, где ожидается объект, он автоматически вставляет вызов метода `valueOf()`. Например:

Java

```
Integer x = 42; // автобоксинг: компилятор превращает это в Integer x = Integer.valueOf(42);
```

```
List<Integer> list = new ArrayList<>();
```

```
list.add(10); // автобоксинг: 10 превращается в Integer.valueOf(10)
```

Компилятор фактически подставляет `Integer.valueOf(42)`, а не `new Integer(42)`.

Именно поэтому работает кеширование — метод `valueOf` проверяет, попадает ли значение в кешированный диапазон, и если да, возвращает объект из кеша.

Unboxing — это обратный процесс: автоматическое преобразование объекта-обёртки в примитив. Когда компилятор видит, что объект-обёртка используется в контексте, где ожидается примитив, он вставляет вызов соответствующего метода `xxxValue()`. Например:

Java

```
Integer obj = Integer.valueOf(100);
```

```
int value = obj; // анбоксинг: компилятор превращает это в int value = obj.intValue();
```

```
Integer a = 5;
```

```
Integer b = 10;
```

```
int sum = a + b; // анбоксинг обоих операндов: int sum = a.intValue() + b.intValue();
```

Есть важные подводные камни, которые нужно знать. Первый — это `NullPointerException` при анбоксинге `null`:

Java

```
Integer obj = null;
```

```
int value = obj; // выбросит NullPointerException, потому что JVM пытается вызвать null.intValue()
```

Второй — это проблема сравнения с помощью `==`:

Java

```
Integer a = 127;  
Integer b = 127;  
System.out.println(a == b); // true, потому что оба берутся из кеша и указывают  
на один объект
```

```
Integer c = 128;  
Integer d = 128;  
System.out.println(c == d); // false, потому что 128 вне кеша и создаются два  
разных объекта
```

```
Integer e = 127;  
int f = 127;  
System.out.println(e == f); // true, потому что здесь происходит анбоксинг e, и  
сравниваются два int
```

Третий — это производительность в циклах. Если написать:

Java

```
Long sum = 0L;  
for (long i = 0; i < 1_000_000; i++) {  
    sum += i; // каждая итерация: анбоксинг sum, сложение примитивов,  
    автобоксинг результата обратно  
}
```

Здесь на каждой итерации создаётся новый объект Long, что крайне неэффективно. Правильно использовать примитив `long sum = 0L;`.

Автобоксинг также работает при передаче параметров в методы — если метод принимает Integer, а вы передаёте int, произойдёт автобоксинг, и наоборот.

12. Нововведения Java 8

Java 8, выпущенная в марте 2014 года, стала одним из самых революционных релизов в истории языка, кардинально изменившим подход к написанию кода на Java. Вот ключевые нововведения.

Лямбда-выражения — пожалуй, самое значительное изменение. Лямбды позволяют передавать поведение как параметр, писать более компактный и выразительный код. Лямбда — это по сути анонимная функция, краткая запись реализации функционального интерфейса. Например, вместо создания анонимного класса `new Comparator<String>() { @Override public int compare(String a, String b) { return a.compareTo(b); } }` можно просто написать `(a, b) -> a.compareTo(b)`. Лямбды сделали возможным функциональный стиль программирования в Java. Синтаксис лямбды: (параметры) -> выражение или (параметры) -> { блок кода }.

Функциональные интерфейсы — это интерфейсы, содержащие ровно один абстрактный метод. Они являются целевыми типами для лямбда-выражений. Аннотация `@FunctionalInterface` служит для маркировки таких интерфейсов и вызывает ошибку компиляции, если интерфейс содержит более одного абстрактного метода. В пакете `java.util.function` появилось множество стандартных функциональных интерфейсов: `Predicate<T>` (принимает объект, возвращает `boolean`), `Function<T,R>` (принимает объект одного типа, возвращает другого), `Consumer<T>` (принимает объект, ничего не возвращает), `Supplier<T>` (не принимает ничего, возвращает объект), `BiFunction`, `UnaryOperator`, `BinaryOperator` и многие другие.

Stream API — новый мощный API для обработки коллекций данных в функциональном стиле. Стримы позволяют выполнять цепочки операций над данными: фильтрацию, маппинг, сортировку, агрегацию и многое другое, причём код получается декларативным и читаемым. Стрим не хранит данные и не изменяет источник — это конвейер операций. Операции делятся на промежуточные (intermediate), такие как `filter()`, `map()`, `sorted()`, `distinct()`, `limit()`, и терминальные (terminal), такие как `collect()`, `forEach()`, `reduce()`,

`count()`, `findFirst()`. Промежуточные операции ленивы — они не выполняются, пока не будет вызвана терминальная операция. Стримы также поддерживают параллельную обработку через `parallelStream()`, что позволяет легко задействовать многоядерные процессоры.

Методы по умолчанию (default methods) в интерфейсах — теперь интерфейсы могут содержать методы с реализацией, помеченные ключевым словом `default`. Это было сделано прежде всего для того, чтобы можно было добавлять новые методы в существующие интерфейсы стандартной библиотеки (например, метод `forEach()` в `Iterable`, метод `stream()` в `Collection`), не ломая обратную совместимость с уже написанным кодом, реализующим эти интерфейсы. Также интерфейсы получили возможность содержать статические методы.

`Optional` — новый класс `java.util.Optional<T>`, представляющий контейнер, который может содержать значение или быть пустым. Он был введён для борьбы с `NullPointerException` и для более явного выражения намерения, что метод может не вернуть результат. Вместо возврата `null` метод возвращает `Optional.empty()`, а вместо проверки на `null` используются методы `isPresent()`, `ifPresent()`, `orElse()`, `orElseGet()`, `map()`, `flatMap()` и другие.

Новый API для работы с датой и временем (`java.time`) — пакет `java.time` с классами `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant`, `Duration`, `Period` и другими. Этот API заменил устаревшие и неудобные классы `java.util.Date` и `java.util.Calendar`. Новые классы являются неизменяемыми и потокобезопасными, имеют интуитивно понятный API и корректно обрабатывают временные зоны.

Ссылки на методы (method references) — это сокращённый синтаксис для лямбда-выражений, когда лямбда просто вызывает уже существующий метод. Существуют четыре вида: ссылка на статический метод (`ClassName::staticMethod`), ссылка на метод экземпляра конкретного объекта (`instance::method`), ссылка на метод экземпляра произвольного объекта

определённого типа (`ClassName::instanceMethod`) и ссылка на конструктор (`ClassName::new`).

Nashorn JavaScript Engine — новый движок JavaScript, заменивший устаревший Rhino, позволяющий выполнять JavaScript-код из Java (впоследствии был удалён в Java 15).

Усовершенствования в работе с коллекциями — метод `forEach()` в `Iterable`, метод `removeIf()` в `Collection`, метод `replaceAll()` и `sort()` в `List`, методы `getOrDefault()`, `putIfAbsent()`, `compute()`, `merge()` в `Map`, и многое другое.

Все эти нововведения в совокупности кардинально изменили стиль программирования на Java, сделав код более лаконичным, выразительным и функциональным.

Java Strings & Classes — Подробные Ответы

13. Разница между `new String()` и строковым литералом. Метод `intern()`

Строки в Java можно создать двумя принципиально разными способами, и важно понимать, что происходит в памяти в каждом случае.

Когда вы создаёте строку с помощью литерала в двойных кавычках, например `String s = "hello";`, JVM сначала проверяет, существует ли уже строка с таким содержимым в специальной области памяти, называемой пулом строк (`String Pool`). Если такая строка уже есть в пуле — переменная `s` получает ссылку на уже существующий объект, и новый объект не создаётся. Если строки в пуле ещё нет — JVM создаёт новый объект, помещает его в пул и возвращает ссылку на него. Таким образом, два литерала с одинаковым содержимым всегда ссылаются на один и тот же объект в пуле. Это подтверждается следующим примером: `String a = "hello"; String b = "hello"; System.out.println(a == b);` — результат будет `true`, потому что обе переменные ссылаются на один объект из пула.

Когда вы создаёте строку с помощью `new String("hello")`, JVM всегда создаёт новый объект в обычной куче (`heap`), вне зависимости от того, есть ли строка

с таким содержимым в пуле. То есть `new String()` принудительно создаёт новый объект, игнорируя пул. Поэтому `String a = new String("hello"); String b = new String("hello"); System.out.println(a == b);` даст `false` — это два разных объекта в куче. При этом интересный момент: когда вы пишете `new String("hello")`, сначала JVM проверяет пул на наличие литерала "hello" (потому что "hello" — это тоже литерал), и если его там нет, помещает его туда, а затем дополнительно создаёт второй объект в куче. То есть может создаться два объекта: один в пуле, один в куче.

Начиная с Java 7 пул строк был перемещён из Permanent Generation в кучу (Heap), что позволило сборщику мусора удалять из него неиспользуемые строки. До Java 7 строки в пуле никогда не собирались GC, что могло приводить к утечкам памяти при интенсивном использовании метода `intern()`.

Метод `intern()` — это метод класса `String`, который возвращает каноническое представление строки из пула. Когда вы вызываете `intern()` на строке, JVM проверяет пул: если строка с таким содержимым уже есть в пуле, возвращает ссылку на неё; если нет — добавляет текущую строку в пул и возвращает ссылку на неё. Это позволяет "привязать" к пулу строку, созданную через `new` или полученную динамически (например, из файла или базы данных):

Java

```
String a = new String("hello");
String b = "hello";
System.out.println(a == b);      // false — разные объекты
System.out.println(a.intern() == b); // true — intern() вернул объект из пула,
                                   // который совпадает с литералом b
```

На практике `intern()` используется редко, потому что он имеет накладные расходы и может быть источником проблем с производительностью при массовом применении. Основная его область применения — оптимизация памяти, когда в программе накапливается огромное количество повторяющихся строк, полученных из внешних источников.

14. Как реализован класс String, какие поля там есть

Класс String в Java является неизменяемым (immutable), финальным и находится в пакете java.lang. Рассмотрим его внутреннюю реализацию.

До Java 9 класс String внутри хранил символы в виде поля `private final char[] value` — массива символов типа `char`. Каждый `char` в Java занимает 2 байта (UTF-16 кодировка). Это означало, что строка "hello" занимала массив из 5 элементов, каждый по 2 байта. Кроме того, было поле `private int hash`, хранящее закешированный хэш-код строки (изначально равный 0, вычисляется лениво при первом вызове `hashCode()` и сохраняется для повторного использования). В некоторых версиях также было поле `offset` и `count`, позволявшие нескольким строкам разделять один массив символов — это называлось `view`, однако от этого подхода отказались из-за проблем с памятью.

Начиная с Java 9 в рамках оптимизации JEP 254 (Compact Strings) реализация была изменена. Теперь String хранит данные в поле `private final byte[] value` — массиве байтов, а не символов. Дополнительно появилось поле `private final byte coder`, которое указывает, в какой кодировке хранятся данные: LATIN1 (значение 0) или UTF16 (значение 1). Если строка содержит только символы, укладываемые в Latin-1 (то есть коды от 0 до 255 — ASCII и стандартные европейские символы), она хранится в формате Latin-1, где каждый символ занимает всего 1 байт. Если строка содержит символы вне диапазона Latin-1 (например, кириллица, китайские иероглифы и прочие), она хранится в формате UTF-16, где каждый символ занимает 2 байта. Такая оптимизация позволяет существенно экономить память, потому что в типичных приложениях большинство строк содержат ASCII-символы, и для них память уменьшается вдвое. Статические константы `LATIN1 = 0` и `UTF16 = 1` определены непосредственно в классе String.

Также в классе есть поле `private int hash` для кеширования хэш-кода и `private boolean hashIsZero`, добавленное позже для корректной работы с хэш-кодом,

равным нулю (ведь 0 — это также дефолтное значение поля hash, и без флага нельзя отличить "хэш ещё не вычислен" от "хэш реально равен нулю").

Класс String реализует интерфейсы Serializable, Comparable<String> и CharSequence. Объявлен как public final class String, что делает невозможным создание подклассов.

15. Понятие Unicode. ASCII. UTF-8 и отличие от UTF-16

ASCII (American Standard Code for Information Interchange) — это одна из старейших кодировок символов, разработанная в 1963 году. Стандартный ASCII включает 128 символов (7-битная кодировка): латинские буквы в верхнем и нижнем регистре (A-Z, a-z), цифры (0-9), знаки пунктуации и управляющие символы (перевод строки, возврат каретки и так далее). Каждый символ представлен числом от 0 до 127. Расширенный ASCII использует 8 бит и добавляет ещё 128 символов (символы с кодами 128-255), однако их значение варьируется в зависимости от кодовой страницы, что породило хаос с совместимостью.

Unicode — это универсальный международный стандарт кодирования символов, разработанный для поддержки всех письменных систем мира. Unicode не является кодировкой сам по себе — это таблица символов (кодировое пространство), где каждому символу (букве, цифре, знаку, эмодзи и так далее) присвоен уникальный номер, называемый кодовой точкой (code point). Кодовые точки записываются как U+XXXX (например, 'A' — это U+0041, буква 'А' кириллическая — U+0410, символ '€' — U+20AC). Unicode охватывает более 1,1 миллиона кодовых точек (от U+0000 до U+10FFFF), хотя реально задействовано из них порядка 150 000. Первые 128 кодовых точек Unicode полностью совпадают с ASCII, что обеспечивает обратную совместимость.

UTF-8 (Unicode Transformation Format, 8-bit) — это переменная кодировка на основе байтов. В UTF-8 каждый символ Unicode кодируется последовательностью от 1 до 4 байт. ASCII-символы (кодировые точки 0-127)

кодируются одним байтом и полностью совпадают с ASCII — это делает UTF-8 обратно совместимой с ASCII. Символы с кодовыми точками от 128 до 2047 кодируются двумя байтами, от 2048 до 65535 — тремя байтами (это охватывает большинство живых языков, включая кириллицу, арабский, китайский), от 65536 до 1114111 — четырьмя байтами (редкие символы, исторические письменности, эмодзи). Схема кодирования устроена так, что по первому байту всегда можно определить длину последовательности, а начало символа всегда можно найти в потоке байт (самосинхронизирующаяся кодировка). UTF-8 является наиболее распространённой кодировкой в интернете (используется на более чем 97% веб-страниц) и файловых системах.

UTF-16 (Unicode Transformation Format, 16-bit) — это тоже переменная кодировка, где каждый символ кодируется либо двумя байтами (одна 16-битная единица кода, для символов из Basic Multilingual Plane, кодовые точки U+0000 — U+FFFF), либо четырьмя байтами (две 16-битные единицы, называемые суррогатной парой, для символов с кодовыми точками выше U+FFFF). Java внутренне использует UTF-16 для представления строк (поле `char` и до Java 9 массив `char[]` в классе `String`). Тип `char` в Java — это 16-битная единица кода UTF-16, что означает, что символы за пределами Basic Multilingual Plane (например, некоторые эмодзи) представляются двумя `char`-ами (суррогатной парой), и `"emoji".length()` может вернуть 2, а не 1 для такого символа.

Ключевые отличия UTF-8 от UTF-16: UTF-8 более компактна для текстов с преобладанием ASCII (английский текст занимает в два раза меньше места, чем в UTF-16), UTF-16 более компактна для текстов с преобладанием символов из Basic Multilingual Plane (например, китайский текст займёт меньше места в UTF-16, чем в UTF-8, где каждый иероглиф кодируется тремя байтами). UTF-8 совместима с ASCII, UTF-16 — нет. UTF-16 имеет проблему с порядком байт (`big-endian` / `little-endian`, для её решения используется метка BOM — Byte Order Mark). В большинстве веб-приложений и файловых систем используется UTF-8 как стандарт, а Java внутри JVM использует

UTF-16.

16. String, StringBuilder, StringBuffer. Отличия. Почему объявлены как final
В Java для работы со строками существуют три основных класса, каждый из которых предназначен для своих сценариев.

String — неизменяемый (immutable) класс. Любая операция над строкой (конкатенация, замена, приведение к верхнему регистру и так далее) не изменяет оригинальный объект, а создаёт новый объект String. Это имеет важные следствия: объект String является потокобезопасным по своей природе (immutability гарантирует отсутствие состояния гонки), строки можно безопасно использовать в пуле строк (пул возможен только потому, что строки не меняются), строки безопасно использовать как ключи в HashMap (хэш-код не изменится). Однако если вы выполняете множество операций конкатенации в цикле через +, это крайне неэффективно, потому что каждая конкатенация создаёт новый объект.

StringBuilder — изменяемая (mutable) последовательность символов, введённая в Java 5. StringBuilder внутри хранит символы в массиве char[] с некоторым запасом ёмкости (capacity) и позволяет изменять его содержимое без создания новых объектов. Он предоставляет методы append(), insert(), delete(), replace(), reverse() и другие, которые модифицируют внутренний буфер. StringBuilder не является потокобезопасным — его методы не синхронизированы. Поэтому использовать один экземпляр StringBuilder из нескольких потоков без внешней синхронизации опасно. StringBuilder следует использовать в однопоточных контекстах, особенно для построения строк в цикле — это значительно эффективнее, чем конкатенация через +. Кстати, компилятор Java автоматически заменяет конкатенацию строк через + на использование StringBuilder (начиная с Java 9 — через StringConcatFactory), но только в случае одного выражения конкатенации, а не в цикле.

StringBuffer — это по сути тот же StringBuilder, но все его методы

синхронизированы (`synchronized`), что делает его потокобезопасным. Был введён ещё в Java 1.0. Из-за синхронизации `StringBuffer` работает медленнее, чем `StringBuilder` в однопоточных сценариях. На практике необходимость в `StringBuffer` возникает крайне редко, потому что если вам нужна потокобезопасная операция построения строки — обычно проще и лучше использовать локальный `StringBuilder` внутри каждого потока. `StringBuffer` следует считать устаревшим классом, который сохраняется из соображений обратной совместимости.

Почему все три класса объявлены как `final`? Для `String` это критично из-за принципа неизменяемости: если бы `String` не был `final`, кто-нибудь мог бы создать подкласс `EvilString extends String` и переопределить методы, например `hashCode()` или `equals()`, нарушив тем самым инварианты пула строк, работу `HashMap` и общие контракты. Поскольку ссылка типа `String` может указывать на объект подкласса (полиморфизм), нельзя было бы гарантировать неизменяемость. Для `StringBuilder` и `StringBuffer` — `final` предотвращает создание подклассов, которые могли бы нарушить поведение буфера, и является хорошей практикой проектирования для утилитных классов. Кроме того, JVM может применять определённые оптимизации к `final`-классам, зная, что их методы не будут переопределены.

17. Базовый класс в Java. Методы класса `Object`

В Java все классы неявно наследуют от класса `java.lang.Object` — это единственный класс, у которого нет родителя. Если при объявлении класса явно не указан родитель через `extends`, компилятор автоматически добавляет `extends Object`. Таким образом, `Object` является корнем всей иерархии классов в Java.

Класс `Object` предоставляет ряд методов, которые автоматически доступны в каждом объекте.

`public boolean equals(Object obj)` — метод сравнения на равенство. Базовая реализация в `Object` сравнивает ссылки (то есть ведёт себя как `==`), что

означает "два объекта равны, только если это один и тот же объект". Классы обычно переопределяют этот метод для сравнения по содержимому.

`public int hashCode()` — возвращает хэш-код объекта. Базовая реализация обычно основана на адресе объекта в памяти (или каком-то другом идентификаторе, зависящем от реализации JVM). Используется в хэш-таблицах (`HashMap`, `HashSet`). Контракт: если `a.equals(b)` истинно, то `a.hashCode()` должен равняться `b.hashCode()`.

`public String toString()` — возвращает строковое представление объекта. Базовая реализация возвращает строку вида `getClass().getName() + "@" + Integer.toHexString(hashCode())`, например `java.lang.Object@1b6d3586`. Обычно переопределяется для получения читаемого представления.

`public final Class<?> getClass()` — возвращает объект `Class`, представляющий класс данного объекта во время выполнения. Финальный — нельзя переопределить.

`protected Object clone()` — создаёт и возвращает копию объекта. Чтобы использовать этот метод, класс должен реализовать интерфейс `Cloneable`, иначе выбросится `CloneNotSupportedException`.

`protected void finalize()` — вызывается сборщиком мусора перед тем, как объект будет уничтожен. Устарел, начиная с Java 9, и помечен для удаления. Его использование непредсказуемо и не рекомендуется.

`public final void wait()`, `public final void wait(long timeout)`, `public final void wait(long timeout, int nanos)` — заставляют текущий поток ждать, пока другой поток не вызовет `notify()` или `notifyAll()` на этом же объекте. Могут вызываться только в синхронизированном блоке.

`public final void notify()` — пробуждает один случайный поток, ожидающий на этом объекте.

`public final void notifyAll()` — пробуждает все потоки, ожидающие на этом объекте.

18. Принципы ООП. Абстракция

Объектно-ориентированное программирование (ООП) базируется на четырёх основных принципах: инкапсуляция, наследование, полиморфизм и абстракция. Рассмотрим каждый кратко, а затем подробнее остановимся на абстракции.

Инкапсуляция — это принцип объединения данных (полей) и методов, работающих с этими данными, внутри одного класса, а также сокрытия внутренних деталей реализации от внешнего мира. Поля обычно делаются `private`, доступ к ним предоставляется через публичные геттеры и сеттеры. Это позволяет изменять внутреннюю реализацию, не затрагивая код, использующий класс.

Наследование — это возможность создавать новые классы на основе уже существующих, наследуя их поля и методы, и при необходимости переопределяя или расширяя их. В Java поддерживается только одиночное наследование (`extends` одного класса), но множественное наследование через интерфейсы.

Полиморфизм — это способность объекта принимать много форм. В Java проявляется прежде всего как возможность использовать объект подкласса там, где ожидается объект суперкласса (полиморфизм подтипов), и как способность метода работать с объектами разных классов через единый интерфейс. Переопределённые методы вызываются на основе реального типа объекта в рантайме (`dynamic dispatch` / позднее связывание).

Абстракция — это принцип выделения существенных характеристик объекта и сокрытия несущественных деталей. Абстракция позволяет работать с концепциями на более высоком уровне, не думая о деталях реализации. В

Java абстракция реализуется через абстрактные классы и интерфейсы. Абстрактный класс — это класс, который объявлен с ключевым словом `abstract` и может содержать как абстрактные методы (без реализации), так и конкретные методы (с реализацией). Объект абстрактного класса нельзя создать напрямую. Интерфейс — ещё более высокий уровень абстракции, который задаёт только контракт (что нужно делать), но не говорит, как это делать.

Практический смысл абстракции: когда вы работаете с коллекцией через интерфейс `List`, вас не интересует, `ArrayList` это или `LinkedList` внутри — вы используете абстракцию. Когда вы описываете животных через абстрактный класс `Animal` с абстрактным методом `makeSound()` — это тоже абстракция: вы фиксируете, что любое животное издаёт звук, но конкретная реализация определяется в подклассах `Dog`, `Cat` и так далее.

19. Правила переопределения метода `boolean equals(Object o)`

Метод `equals()` подчиняется строгому контракту, определённом в документации Java, и при переопределении необходимо соблюдать все его условия, иначе поведение структур данных (например, `HashMap`, `HashSet`) будет некорректным.

Рефлексивность: объект должен быть равен самому себе. `x.equals(x)` должно возвращать `true` для любого ненулевого `x`. Это требование выполняется естественно для большинства реализаций, но нужно помнить о нём.

Симметричность: если `x.equals(y)` возвращает `true`, то и `y.equals(x)` должно возвращать `true`. Нарушение симметрии — распространённая ошибка при реализации `equals` между классом и его подклассом.

Транзитивность: если `x.equals(y)` и `y.equals(z)`, то `x.equals(z)` должно быть `true`. Это также может нарушаться при наследовании, поэтому рекомендуется предпочитать композицию наследованию, когда нужно добавить компонент в значимое поле.

Согласованность: многократное вызов `x.equals(y)` должен возвращать одно и то же значение, если состояние объектов не изменялось.

Ненулевость: `x.equals(null)` должно возвращать `false` для любого ненулевого `x`, а не выбрасывать `NullPointerException`.

Типичная правильная реализация метода выглядит следующим образом. Сначала проверяем ссылку на самого себя (`if (this == o) return true;`) — это оптимизация. Затем проверяем, не является ли параметр `null` и совместим ли он по типу (`if (!(o instanceof MyClass)) return false;`) — `instanceof` сам обрабатывает `null` (возвращает `false`). Приводим параметр к нужному типу и сравниваем все значимые поля. Для примитивных полей используем `==`, для объектов — `Objects.equals()` (который корректно обрабатывает `null`), для массивов — `Arrays.equals()`. Начиная с Java 16 вместо `instanceof` с последующим кастом можно использовать `instanceof MyClass other` (паттерн-матчинг для `instanceof`):

Java

`@Override`

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (!(o instanceof Person other)) return false;  
    return age == other.age && Objects.equals(name, other.name);  
}
```

Важно помнить: при переопределении `equals` всегда нужно переопределять и `hashCode()`, иначе нарушится контракт между этими двумя методами.

20. Зачем переопределять `hashCode` и `equals` одновременно

Контракт между методами `equals()` и `hashCode()` гласит: если два объекта равны по `equals()`, у них должен быть одинаковый `hashCode()`. Обратное не обязательно — разные объекты могут иметь одинаковый хэш-код (это

называется коллизией), но это нежелательно с точки зрения производительности.

Почему это критично? Рассмотрим, как работает `HashMap`. При добавлении элемента методом `put(key, value)` `HashMap` сначала вычисляет хэш-код ключа, на основе хэша определяет, в какую "корзину" (`bucket`) поместить элемент, и затем, если в корзине уже есть элементы, использует `equals()` для проверки совпадения ключей.

Если вы переопределили `equals()`, но не переопределили `hashCode()`, может произойти следующее. Два объекта `Person("Alice", 30)` считаются равными по `equals()`, но их `hashCode()` (унаследованный от `Object`, основанный на адресе в памяти) различается. Тогда при добавлении первого объекта в `HashMap` он попадёт в одну корзину, при поиске второго (равного первому) объекта он попадёт в другую корзину и не будет найден — хотя по логике должен был бы. В результате `HashMap.containsKey()` вернёт `false` для объекта, который по `equals()` равен имеющемуся ключу, и вы не сможете найти или удалить элемент.

Если же переопределить `hashCode()` без переопределения `equals()`, оба объекта с "одинаковым" содержимым попадут в одну корзину, но `equals()` (поведение по умолчанию — сравнение ссылок) скажет, что они разные, и они оба останутся в структуре данных как разные ключи — опять логическая ошибка.

Поэтому правило простое и непреложное: всегда переопределяйте оба метода вместе. Современные IDE умеют автоматически генерировать корректные реализации обоих методов, также для этого удобно использовать `Objects.hash(field1, field2, ...)` и `Objects.equals()`.

21. Правила переопределения `int hashCode()`. Можно ли возвращать константу?

При переопределении `hashCode()` нужно соблюдать несколько правил.

Согласованность с equals(): если `a.equals(b)` возвращает `true`, то `a.hashCode()` должен быть равен `b.hashCode()`. Это обязательное требование.

Согласованность во времени: если объект не изменился (не изменились поля, участвующие в расчёте хэша), то многократные вызовы `hashCode()` должны возвращать одно и то же значение в пределах одного запуска JVM.

Хорошее распределение: хотя формально не обязательно, хэш-коды разных объектов должны по возможности различаться, чтобы избежать коллизий в хэш-таблицах. Чем больше коллизий — тем хуже производительность `HashMap` (в худшем случае поиск деградирует от $O(1)$ до $O(n)$ или $O(\log n)$ начиная с Java 8, где в корзинах длиной более 8 элементов используется красно-чёрное дерево вместо связного списка).

Хороший подход к реализации: включать в расчёт хэша те же поля, которые используются в `equals()`, и использовать утилитный метод `Objects.hash(field1, field2, ...)` или стандартную формулу: начать с числа 31 (простое число, используемое в стандартных реализациях: `result = 31 * result + field.hashCode()`). Число 31 выбрано потому, что умножение на него хорошо распределяет биты и эффективно вычисляется через побитовые операции.

Можно ли возвращать константу? Технически — да, это допустимо с точки зрения контракта. Если `hashCode()` всегда возвращает, например, 42, контракт не нарушается: у двух равных объектов будет одинаковый хэш-код (оба вернут 42). Однако это катастрофически плохо с точки зрения производительности: все объекты попадут в одну и ту же корзину `HashMap`, и поиск в такой хэш-таблице будет иметь сложность $O(n)$ вместо $O(1)$. Фактически `HashMap` превратится в связный список. Поэтому возврат константы является допустимым, но абсолютно не рекомендуемым антипаттерном — так иногда делают в учебных целях, чтобы намеренно продемонстрировать деградацию производительности.

22. Правила переопределения метода clone()

Метод clone() определён в классе Object и объявлен как protected, то есть по умолчанию он недоступен из внешнего кода. Чтобы разрешить клонирование объектов своего класса, нужно выполнить несколько шагов.

Во-первых, класс должен реализовать интерфейс Cloneable. Этот интерфейс является маркерным (не содержит методов) и служит сигналом для JVM о том, что данный класс разрешает клонирование. Если вызвать clone() на объекте класса, не реализующего Cloneable, будет выброшено CloneNotSupportedException.

Во-вторых, нужно переопределить метод clone() и сделать его public (переопределение с расширением модификатора доступа — единственный разрешённый вариант изменения модификатора при переопределении). Базовая реализация Object.clone() выполняет мелкое копирование (shallow copy): создаёт новый объект того же класса и копирует все поля побитово. Для примитивных полей это означает копирование значений. Для ссылочных полей это означает копирование ссылок, то есть и оригинал, и копия будут ссылаться на одни и те же вложенные объекты.

Если мелкое копирование нас устраивает (например, все поля — примитивы), то достаточно просто вызвать super.clone(). Если же нужно глубокое копирование (deep copy), где и вложенные объекты тоже копируются независимо, нужно вручную клонировать каждое ссылочное поле.

Проблемы с clone(): механизм клонирования в Java считается непродуманным и вызывает много вопросов. Метод Object.clone() создаёт экземпляр без вызова конструктора, что может нарушать инварианты класса. Интерфейс Cloneable не содержит метод clone(), что делает его контракт неочевидным. Глубокое клонирование при сложных графах объектов трудно реализовать корректно. Поэтому на практике для копирования объектов часто предпочитают использовать конструктор копирования (copy constructor, метод принимающий объект того же типа и создающий новый) или фабричный

метод копирования — это более гибкие и понятные альтернативы.

23. Отличия `finally` и `finalize`. Для чего используется `final`?

Несмотря на схожесть звучания, это три совершенно разных понятия в Java.

Ключевое слово `final` используется в трёх контекстах. Для переменной `final` означает, что после инициализации её значение нельзя изменить — переменная становится константой (однако если это ссылочная переменная, состояние объекта, на который она указывает, изменить можно; нельзя только изменить саму ссылку). Для метода `final` означает, что метод не может быть переопределён в подклассах. Для класса `final` означает, что от класса нельзя наследоваться. Также поле `final` экземпляра должно быть инициализировано либо при объявлении, либо в конструкторе.

`finally` — это блок в конструкции `try-catch-finally`. Блок `finally` гарантированно выполняется после блока `try` и возможных блоков `catch`, независимо от того, было ли выброшено исключение, было ли оно поймано, или даже если в блоке `try` есть оператор `return`. Блок `finally` используется для освобождения ресурсов: закрытие файловых потоков, соединений с базами данных, сетевых соединений. Единственные случаи, когда `finally` не выполняется: вызов `System.exit()`, фатальная ошибка JVM, или если поток был принудительно убит. Начиная с Java 7 для управления ресурсами рекомендуется использовать конструкцию `try-with-resources`, которая автоматически закрывает ресурсы, реализующие интерфейс `AutoCloseable`.

`finalize()` — это метод класса `Object`, который теоретически должен был вызываться сборщиком мусора перед уничтожением объекта, давая объекту возможность освободить ресурсы. На практике этот метод крайне непредсказуем: нет гарантий, когда именно он будет вызван (GC может откладывать это на неопределённое время), и нет гарантий, что он вообще будет вызван до завершения JVM. Использование `finalize()` для освобождения ресурсов (например, закрытия соединений) — серьёзный антипаттерн. Метод `finalize()` был объявлен устаревшим в Java 9 и удалён в Java 18. Вместо него

следует использовать try-with-resources или Cleaner API (введённый в Java 9).

24. JavaBeans: требования к классам Bean-компонентов, соглашения об именах

JavaBeans — это стандарт для создания повторно используемых программных компонентов в Java, впервые описанный в спецификации JavaBeans API. Класс, соответствующий спецификации JavaBeans, называется бином (bean) и должен удовлетворять следующим требованиям.

Во-первых, класс должен иметь публичный конструктор без параметров (no-arg constructor). Это позволяет различным фреймворкам и инструментам (IDE, сериализаторы, фреймворки внедрения зависимостей) создавать экземпляры бина без знания специфики конструктора.

Во-вторых, все свойства класса (поля, доступные для чтения/записи) должны быть приватными, а доступ к ним — через публичные методы геттеров и сеттеров, следующие соглашению об именах. Для свойства name типа String геттер должен называться getName() и возвращать String, сеттер — setName(String name) и возвращать void. Для булевых свойств геттер может иметь префикс is вместо get: для свойства active — isActive(). Если свойство доступно только для чтения, определяется только геттер; только для записи — только сеттер (редко).

В-третьих, класс должен быть сериализуемым: реализовывать интерфейс java.io.Serializable. Это позволяет сохранять состояние бина (передавать по сети, сохранять на диск). При этом принято объявлять поле private static final long serialVersionUID = 1L; для управления версионностью сериализации.

В-четвёртых, класс может поддерживать события через механизм listener'ов (регистрация/удаление слушателей), хотя это требование актуально прежде всего для JavaBeans в контексте визуальных компонентов.

Сегодня концепция JavaBeans широко используется в различных

фреймворках (Spring, Hibernate, JSF), которые требуют соблюдения соглашений об именах геттеров/сеттеров для автоматического маппинга свойств.

25. Ассоциации. Композиция и агрегация

Ассоциация, агрегация и композиция — это типы отношений между классами в ООП, отражающие степень связанности объектов.

Ассоциация — это общий термин для обозначения любой связи между объектами двух классов. Один объект использует или взаимодействует с другим. Ассоциация может быть однонаправленной (А знает о В, но В не знает об А) или двунаправленной.

Агрегация (aggregation) — это тип ассоциации, представляющий отношение "целое — часть" (has-a), при котором части могут существовать независимо от целого. Жизненный цикл частей не определяется жизненным циклом целого. Классический пример: Университет содержит Кафедры, а Кафедра содержит Преподавателей. Если Университет прекращает существование, Преподаватели всё равно продолжают существовать. На диаграммах UML агрегация обозначается незакрашенным ромбом у "целого". В коде агрегация обычно реализуется как поле-ссылка, объект которого создаётся вне данного класса и передаётся через конструктор или сеттер. Пример: класс Team содержит список объектов Player, которые могут существовать и вне команды.

Композиция (composition) — это более сильный тип агрегации, при котором части не могут существовать без целого. Жизненный цикл части полностью определяется жизненным циклом целого: когда целое создаётся — создаются части, когда целое уничтожается — уничтожаются части. Пример: Дом содержит Комнаты. Комнаты не могут существовать без Дома. Другой пример: объект класса Order содержит объекты OrderItem — элементы заказа, которые не имеют смысла вне заказа. На диаграммах UML обозначается закрашенным ромбом. В коде части обычно создаются непосредственно в

конструкторе целого или при инициализации полей.

Ключевое различие: при агрегации часть может принадлежать нескольким целым или существовать самостоятельно; при композиции часть принадлежит строго одному целому и погибает вместе с ним.

26. Отличия абстрактного класса от интерфейса. Модификаторы полей интерфейса

До Java 8 разница между абстрактным классом и интерфейсом была более очевидной, но даже сейчас она принципиальна.

Абстрактный класс объявляется с ключевым словом `abstract` и может содержать как абстрактные методы (без реализации), так и конкретные методы (с реализацией). Абстрактный класс может иметь поля с любыми модификаторами доступа и любых типов (не только константы). Он может иметь конструкторы (которые вызываются из конструкторов подклассов). Класс может наследовать только от одного абстрактного класса (`extends`). Абстрактный класс может иметь состояние.

Интерфейс объявляется ключевым словом `interface`. До Java 8 интерфейс мог содержать только абстрактные методы (без реализации) и константы. Начиная с Java 8 интерфейсы могут содержать методы с реализацией через `default` и `static` ключевые слова. Начиная с Java 9 добавились `private` методы в интерфейсах. Класс может реализовывать произвольное количество интерфейсов (`implements`), что является способом достижения множественного наследования поведения в Java. Интерфейсы не могут иметь конструкторов и не могут хранить изменяемое состояние (только константы).

Поля интерфейса по умолчанию имеют модификаторы `public static final`. То есть любое поле, объявленное в интерфейсе, автоматически является публичной статической константой, даже если явно не написать эти модификаторы. Это означает, что поля интерфейса — это всегда константы, и их нельзя изменить.

Методы интерфейса по умолчанию (без явного модификатора) являются `public abstract`. Методы с ключевым словом `default` являются `public default`. Начиная с Java 9 в интерфейсах допустимы `private` методы — вспомогательные методы для повторного использования кода внутри `default` методов.

Когда использовать что: абстрактный класс хорошо подходит, когда есть общее состояние и общая базовая реализация для группы связанных классов. Интерфейс подходит, когда нужно определить контракт (набор методов) без привязки к иерархии наследования, или когда нужно множественное наследование.

27. Когда применять интерфейс, а когда абстрактный класс

Выбор между интерфейсом и абстрактным классом — это вопрос проектирования, который зависит от конкретной ситуации.

Интерфейс следует предпочесть, когда вы хотите определить контракт (что должен уметь делать класс), не предопределяя, как именно. Интерфейс подходит, когда несвязанные классы должны реализовать одно и то же поведение — например, интерфейс `Comparable` реализуют `String`, `Integer`, и пользовательские классы, не имеющие ничего общего в иерархии. Интерфейс нужен, когда хочется использовать преимущества множественного наследования — класс может реализовывать любое количество интерфейсов. Интерфейс предпочтителен, когда вы проектируете API для использования сторонними разработчиками — это даёт максимальную гибкость. Принцип SOLID (Interface Segregation Principle) говорит, что лучше иметь много маленьких специализированных интерфейсов, чем один большой.

Абстрактный класс следует предпочесть, когда есть общая реализация или общее состояние, которое имеет смысл разделить между подклассами, и когда вы хотите избежать дублирования кода. Абстрактный класс подходит, когда классы связаны иерархически и "является разновидностью" (is-a) —

например, `Animal` как абстрактный класс для `Dog` и `Cat`. Абстрактный класс нужен, когда требуется определить конструктор с параметрами, который будут вызывать подклассы. Абстрактный класс подходит, когда нужен шаблонный метод (Template Method pattern) — базовый алгоритм определён в абстрактном классе, а отдельные шаги переопределяются в подклассах.

Хорошая практика: программируйте через интерфейсы (`List list = new ArrayList()` вместо `ArrayList list = new ArrayList()`), это делает код гибким. Если нужно предоставить некоторую общую реализацию — создайте абстрактный класс-скелет (abstract skeletal implementation), как это сделано в Java со классами `AbstractList`, `AbstractMap` и подобными.

28. Бывают ли интерфейсы без методов? Для чего?

Да, интерфейсы без методов существуют, и они называются маркерными интерфейсами (marker interfaces или tagging interfaces). Их единственная цель — "пометить" класс, указав принадлежность к определённой категории или наличие определённого свойства, без добавления каких-либо методов.

Примеры маркерных интерфейсов в стандартной библиотеке Java:

`java.io.Serializable` — указывает, что объекты этого класса могут быть сериализованы (преобразованы в байтовый поток). Механизм сериализации проверяет через рефлексия, реализует ли класс этот интерфейс, и если нет — выбрасывает `NotSerializableException`.

`java.lang.Cloneable` — указывает, что класс разрешает вызов метода `clone()` для своих объектов.

`java.util.RandomAccess` — указывает, что реализующий список поддерживает эффективный произвольный доступ (доступ по индексу за $O(1)$), что позволяет алгоритмам выбирать оптимальную стратегию обхода.

Как маркерные интерфейсы используются: код проверяет наличие

интерфейса через оператор `instanceof` или метод `isAssignableFrom()` рефлексии. Например, `if (object instanceof Serializable) { ... }`.

Альтернативой маркерным интерфейсам являются аннотации. Аннотации более гибки (могут содержать параметры) и не занимают место в иерархии типов. Поэтому при проектировании нового кода предпочтительнее использовать аннотации вместо маркерных интерфейсов. Однако маркерные интерфейсы имеют одно преимущество перед аннотациями: они участвуют в системе типов, и `instanceof` является более строгой и производительной проверкой, чем рефлексивная проверка аннотаций.

29. Перегрузка и переопределение. Можно ли менять модификатор доступа?

Перегрузка (overloading) — это возможность иметь в одном классе несколько методов с одинаковым именем, но разными списками параметров (разное количество параметров, разные типы или разный порядок). Перегрузка разрешается на этапе компиляции (compile-time/static polymorphism) на основе статического типа аргументов. Возвращаемый тип, модификаторы доступа и выбрасываемые исключения не являются частью сигнатуры метода для целей перегрузки. Пример: `void print(String s)`, `void print(int i)`, `void print(String s, int count)` — все три метода с именем `print` в одном классе.

Переопределение (overriding) — это предоставление в подклассе новой реализации метода, унаследованного от суперкласса, с той же сигнатурой (одинаковое имя и список параметров). Переопределение разрешается во время выполнения (runtime/dynamic polymorphism) на основе реального типа объекта. Переопределённый метод помечается аннотацией `@Override`, которая является подсказкой компилятору: если с такой аннотацией метод на самом деле ничего не переопределяет (например, из-за опечатки в имени), компилятор выдаст ошибку.

Можно ли менять модификатор доступа? При переопределении можно изменить модификатор доступа, но только в сторону расширения (менее

ограниченный). Нельзя сузить доступ. Правило: уровень доступа переопределённого метода должен быть не ниже уровня доступа метода в суперклассе. Например, если метод в суперклассе `protected`, в подклассе его можно переопределить как `protected` или `public`, но нельзя как `private` или `package-private`. Это связано с принципом подстановки Лисков (LSP): объект подкласса должен быть взаимозаменяем с объектом суперкласса, и если суперкласс предоставляет `protected` доступ, подкласс не может его ограничить.

При перегрузке модификатор доступа можно менять произвольно — перегруженные методы являются независимыми методами, поэтому каждый может иметь любой модификатор.

30. Перегрузка и переопределение: изменение возвращаемого типа и типа параметров

При переопределении и возвращаемый тип: начиная с Java 5 при переопределении разрешено изменять возвращаемый тип на ковариантный (`covariant return type`). Это означает, что возвращаемый тип переопределённого метода может быть подклассом возвращаемого типа оригинального метода. Например, если в суперклассе метод возвращает `Animal`, в подклассе его можно переопределить так, чтобы он возвращал `Dog` (подкласс `Animal`). Это удобно и безопасно: раз метод объявлен возвращающим `Animal`, то возврат `Dog` (который является `Animal`) не нарушает контракт. При этом возвращаемый тип не может быть расширен (нельзя вернуть менее специфичный тип, чем в суперклассе). Для примитивов ковариантность не применяется.

При переопределении: изменение типа параметров — это уже не переопределение, а перегрузка. Если вы измените тип хотя бы одного параметра, компилятор воспримет это как новый перегруженный метод, а не переопределение унаследованного. Если при этом стоит аннотация `@Override`, компилятор выдаст ошибку, потому что метод с такой сигнатурой не существует в суперклассе.

При перегрузке: изменение возвращаемого типа — нельзя перегружать метод, изменив только возвращаемый тип при одинаковых параметрах. Два метода `int foo()` и `String foo()` в одном классе вызовут ошибку компиляции, потому что по одному только имени и параметрам компилятор не может определить, какой из них вызвать. Возвращаемый тип не является частью сигнатуры для определения перегрузки.

При перегрузке: изменение типа параметров — именно это и является основой перегрузки. Можно менять типы параметров, их количество, порядок. Каждая такая комбинация создаёт отдельный перегруженный метод.

31. Как передаются переменные в методы — по значению или по ссылке?

В Java переменные всегда передаются в методы по значению (pass by value). Это утверждение справедливо для всех типов переменных, однако его смысл различается для примитивов и ссылочных типов.

Для примитивных типов всё очевидно: передаётся копия самого значения. Если внутри метода изменить параметр, оригинальная переменная не изменится:

Java

```
void increment(int x) { x++; } // изменяет только локальную копию
int a = 5;
increment(a);
System.out.println(a); // выведет 5, а не 6
```

Для ссылочных типов передаётся копия ссылки. То есть метод получает свою копию ссылки, которая указывает на тот же объект в памяти. Это означает, что через эту ссылку метод может изменять состояние объекта (вызывать его методы, менять поля), и эти изменения будут видны снаружи — потому что обе ссылки (оригинал и копия) указывают на один и тот же объект. Однако

если внутри метода присвоить параметру-ссылке новый объект, это не изменит оригинальную ссылку:

Java

```
void changeObject(Person p) {  
    p.setName("Bob"); // это изменит объект, на который указывают обе ссылки  
    p = new Person("Charlie"); // это изменит только локальную копию ссылки  
}  
Person person = new Person("Alice");  
changeObject(person);  
System.out.println(person.getName()); // выведет "Bob", потому что объект был  
изменён
```

Итого: Java — pass by value, но для объектов значением является ссылка. Некоторые называют это "pass by value of the reference", что точно описывает механизм. В Java нет pass by reference в том смысле, в каком он есть в C++ (передача указателя на переменную, которая позволила бы изменить саму переменную снаружи).

32. Что такое конструктор по умолчанию?

Конструктор по умолчанию (default constructor) — это конструктор без параметров, который компилятор Java автоматически добавляет в класс, если в классе не определён ни один конструктор явно. Если же программист написал хотя бы один конструктор (с параметрами или без), компилятор уже не добавляет конструктор по умолчанию автоматически.

Конструктор по умолчанию, добавляемый компилятором, является public, если класс public, или package-private в противном случае. Тело конструктора по умолчанию содержит только вызов super() — конструктора суперкласса без параметров. Все поля объекта при этом инициализируются значениями по умолчанию: числовые — 0, boolean — false, char — '\u0000', ссылки — null.

Это важно понимать в контексте наследования: если в суперклассе есть

только конструктор с параметрами (и нет конструктора без параметров), то подкласс обязан явно вызвать один из конструкторов суперкласса через `super(...)`. Иначе компилятор попытается вставить вызов `super()` (без аргументов) автоматически, не найдёт такого конструктора и выдаст ошибку компиляции.

33. Свойства конструктора. Способы его вызова

Конструктор — это специальный метод для инициализации объекта. Он имеет ряд ключевых свойств. Имя конструктора должно совпадать с именем класса. Конструктор не имеет возвращаемого типа (даже `void`) — если написать `void ClassName()`, это будет обычный метод, а не конструктор. Конструкторов в классе может быть несколько (перегрузка конструкторов). Конструктор вызывается при создании объекта оператором `new`. Конструктор может выбрасывать исключения. Конструктор не наследуется и не переопределяется.

Способы вызова конструктора. Основной способ — через оператор `new`: `MyClass obj = new MyClass(param1, param2);`. При этом JVM выделяет память для объекта, инициализирует все поля значениями по умолчанию, затем вызывает конструктор.

Вызов конструктора из другого конструктора того же класса — через `this(...)`. Такой вызов должен быть первой строкой конструктора. Это позволяет реализовать цепочку конструкторов и избежать дублирования кода:

Java

```
public MyClass() { this("default"); }  
public MyClass(String name) { this.name = name; }
```

Вызов конструктора суперкласса из конструктора подкласса — через `super(...)`. Тоже должен быть первой строкой. Если `super()` не написан явно, компилятор добавляет его автоматически (без аргументов).

Через рефлексию: `Constructor<MyClass> c = MyClass.class.getConstructor();`
`MyClass obj = c.newInstance();` — используется в фреймворках.

Через `clone()` — создание копии существующего объекта, минуя конструктор (что является особенностью механизма клонирования).

Через `ObjectInputStream.readObject()` при десериализации — тоже без вызова обычного конструктора (хотя если класс реализует `Externalizable`, конструктор без аргументов вызывается).

34. Логические блоки. Как работают и какие бывают?

Логические блоки инициализации — это блоки кода, заключённые в фигурные скобки `{ }` и размещённые непосредственно в теле класса (не внутри метода или конструктора).

Блок инициализации экземпляра (`instance initializer block`) — это просто блок `{ ... }` внутри класса. Он выполняется при каждом создании нового объекта данного класса, до вызова конструктора (точнее, код из блока копируется компилятором в каждый конструктор перед его основным кодом, но после вызова `super()`). Блоков может быть несколько, они выполняются в порядке объявления в коде. Используются, когда нужна инициализация, общая для всех конструкторов, и нет желания выносить её в отдельный метод:

Java

```
class MyClass {  
    int value;  
    { value = 10; System.out.println("Instance block"); } // блок инициализации  
    экземпляра  
    MyClass() { System.out.println("Constructor"); }  
}
```

Статический блок инициализации (`static initializer block`) — это блок, помеченный ключевым словом `static`: `static { ... }`. Он выполняется один раз

при первой загрузке класса в JVM (при первом обращении к классу), ещё до создания любых экземпляров. Используется для инициализации статических полей, которые требуют сложной логики (например, заполнение статической коллекции, загрузка JDBC-драйвера, чтение файла конфигурации). Статических блоков тоже может быть несколько, выполняются последовательно:

Java

```
class Config {  
    static Map<String, String> settings;  
    static {  
        settings = new HashMap<>();  
        settings.put("host", "localhost");  
        settings.put("port", "8080");  
    }  
}
```

Порядок инициализации при создании объекта: статические поля и статические блоки суперкласса (при первой загрузке) → статические поля и статические блоки подкласса (при первой загрузке) → поля экземпляра суперкласса и блоки инициализации экземпляра суперкласса → конструктор суперкласса → поля экземпляра подкласса и блоки инициализации экземпляра подкласса → конструктор подкласса.

35. Mutable и Immutable классы. Примеры. Как создать Immutable класс. Record

Mutable (изменяемый) класс — это класс, состояние объектов которого может быть изменено после создания. Примеры из Java: `StringBuilder`, `ArrayList`, `HashMap`, `Date`, `Calendar`, пользовательские классы с геттерами и сеттерами.

Immutable (неизменяемый) класс — это класс, состояние объектов которого не может быть изменено после создания. Любая "модифицирующая" операция создаёт новый объект. Примеры из Java: `String`, `Integer`, `Long`, `Double`

и другие классы-обёртки, BigInteger, BigDecimal, LocalDate, LocalTime, LocalDateTime (java.time API), UUID.

Преимущества неизменяемых объектов: они потокобезопасны без какой-либо синхронизации (несколько потоков могут одновременно читать один объект без риска гонки), их можно безопасно использовать как ключи в HashMap (хэш-код не изменится), их можно свободно разделять и кешировать (пул строк — яркий пример), они упрощают рассуждения о программе (состояние предсказуемо).

Как создать Immutable класс:

Объявить класс как final (чтобы подклассы не могли нарушить неизменяемость).

Все поля объявить как private final.

Не предоставлять сеттеры.

Если поля содержат ссылки на изменяемые объекты (например, ArrayList, Date), делать защитные копии при инициализации в конструкторе (defensive copy) и возвращать защитные копии из геттеров.

Не "утекать" ссылки на изменяемые внутренние поля.

Java

```
public final class ImmutablePoint {  
    private final int x;  
    private final int y;  
    public ImmutablePoint(int x, int y) { this.x = x; this.y = y; }  
    public int getX() { return x; }  
    public int getY() { return y; }  
    public ImmutablePoint translate(int dx, int dy) { return new ImmutablePoint(x +  
dx, y + dy); }  
}
```

Record (записи) — это специальный тип классов, введённый как стабильная фича в Java 16 (JEP 395). Record — это компактный способ объявить

неизменяемый класс-держатель данных. Компилятор автоматически генерирует: приватные финальные поля для каждого компонента, канонический конструктор, геттеры (с именами, совпадающими с именами полей, без префикса get), методы equals(), hashCode() и toString() на основе всех компонентов:

Java

```
public record Point(int x, int y) { }
```

// использование:

```
Point p = new Point(3, 4);
```

```
System.out.println(p.x()); // геттер называется x(), а не getX()
```

Record неявно финальный, его поля неявно финальны. Record не может явно наследоваться от других классов (кроме Record), но может реализовывать интерфейсы. Record можно снабдить компактным конструктором для валидации:

Java

```
public record Range(int min, int max) {
```

```
    public Range { if (min > max) throw new IllegalArgumentException(); }
```

```
}
```

36. static — что такое? Что будет при изменении через объект?

Всегда ли одинаково?

Ключевое слово static в Java означает, что поле, метод или блок принадлежат классу, а не конкретному экземпляру. Статические члены существуют в единственном экземпляре (на уровне класса) и создаются при загрузке класса JVM.

Статическое поле (class variable) — одно поле, общее для всех объектов данного класса. Изменение этого поля через один объект изменяет его для всех объектов. Пример: static int count = 0; — если это счётчик созданных объектов, одно и то же значение видно через любой экземпляр и через сам

класс.

Что будет, если изменить статическое поле через объект? Технически это возможно, но является плохим стилем и вводит в заблуждение. Если написать `obj.staticField = 10;`, это компилируется и работает — изменяется то же самое статическое поле класса. То же самое, что `ClassName.staticField = 10;`. Изменение видно через все другие объекты и через обращение к классу напрямую. Компилятор обычно выдаёт предупреждение, что обращение к статическому полю через экземпляр является нежелательным стилем — рекомендуется всегда обращаться к статическим полям и методам через имя класса.

Всегда ли статическое поле содержит одинаковое значение для всех объектов? В рамках одного `ClassLoader`'а — да, статическое поле одно для всех объектов класса. Однако если один и тот же класс загружен несколькими разными `ClassLoader`'ами, каждый `ClassLoader` будет иметь свою "версию" класса со своей статической переменной (это используется, например, в серверах приложений для изоляции разных веб-приложений). Также в многопоточной среде без синхронизации разные потоки могут видеть устаревшие значения из кеша процессора, если поле не объявлено `volatile`.

37. Может ли статический метод быть перегружен нестатическим? Переопределён?

Перегрузка: да, статический метод может быть перегружен нестатическим в том же классе, потому что перегрузка основана на сигнатуре (имя + параметры). Если нестатический метод имеет другой список параметров — это просто перегрузка, и она работает нормально. Однако если попытаться создать нестатический метод с точно такой же сигнатурой, как у статического — это будет ошибкой компиляции.

Переопределение: статический метод не может быть переопределён в обычном смысле. Если в подклассе объявить статический метод с той же сигнатурой — это называется сокрытием метода (`method hiding`), а не

переопределением. При сокрытии метод выбирается на основе статического типа переменной (тип, указанный при объявлении), а не реального типа объекта. То есть полиморфизм здесь не работает:

Java

```
class Parent { static void hello() { System.out.println("Parent"); } }  
class Child extends Parent { static void hello() { System.out.println("Child"); } }
```

```
Parent obj = new Child();  
obj.hello();      // выведет "Parent" — выбирается по статическому типу Parent  
Child.hello();    // выведет "Child"  
Parent.hello();   // выведет "Parent"
```

Нестатический метод не может переопределить статический (и наоборот). Если в суперклассе метод статический, а в подклассе нестатический с той же сигнатурой — ошибка компиляции. Аннотация `@Override` не применима к сокрытию статических методов.

38. Внутренние классы. Какие бывают и для каких целей используются?

В Java существует несколько видов вложенных (nested) классов, каждый со своими характеристиками.

Внутренний класс (inner class, нестатический вложенный класс) — объявляется внутри другого класса без ключевого слова `static`. Экземпляр внутреннего класса неявно содержит ссылку на экземпляр внешнего класса и может обращаться ко всем его полям и методам (включая приватные). Для создания внутреннего класса необходим экземпляр внешнего: `Outer outer = new Outer(); Outer.Inner inner = outer.new Inner();`. Используется, когда логика внутреннего класса тесно связана с конкретным экземпляром внешнего класса. Недостаток: скрытая ссылка на внешний класс может препятствовать сборке мусора (memory leak), если внутренний класс "живёт" дольше внешнего.

Статический вложенный класс (static nested class) — объявляется с ключевым словом `static`. Не имеет неявной ссылки на экземпляр внешнего класса и может создаваться независимо: `Outer.Nested obj = new Outer.Nested();`. Может обращаться только к статическим членам внешнего класса. Используется для логической группировки классов, которые связаны с внешним классом, но не нуждаются в доступе к его экземплярным полям. Предпочтительнее нестатического внутреннего класса, если доступ к экземпляру внешнего не нужен.

Локальный класс (local class) — объявляется внутри тела метода, блока инициализации или конструктора. Виден только внутри метода, где объявлен. Может обращаться к переменным метода, только если они `effectively final` (не изменяются после инициализации). Используется крайне редко, в ситуациях, когда нужен класс, специфичный только для данного метода.

Анонимный класс (anonymous class) — класс без имени, объявляемый и создаваемый одновременно. Подробнее в следующем вопросе.

39. Анонимные классы. Практическое применение. Для каких классов нельзя создать?

Анонимный класс — это класс без имени, который объявляется и инициализируется одновременно, как часть выражения. Анонимный класс неявно расширяет некоторый класс или реализует некоторый интерфейс:

Java

```
Runnable r = new Runnable() {  
    @Override  
    public void run() { System.out.println("Running!"); }  
};  
new Thread(r).start();
```

Анонимный класс может обращаться к переменным внешнего scope, если они

effectively final. Анонимный класс не может иметь явный конструктор (только инициализаторы экземпляра). Анонимный класс может реализовать только один интерфейс или расширить один класс (нельзя одновременно).

Практическое применение анонимных классов: реализация callback'ов, обработчиков событий (особенно в Swing/AWT: `button.addActionListener(new ActionListener() { ... })`), сравнителей (`Collections.sort(list, new Comparator<String>() { ... })`), реализация абстрактных классов для конкретного локального использования. Начиная с Java 8 многие случаи использования анонимных классов заменены лямбда-выражениями (когда нужно реализовать функциональный интерфейс).

Для каких классов нельзя создать анонимный класс: нельзя создать анонимный подкласс `final` класса (например, `new String() { ... }` — ошибка, потому что `String` является `final`). Нельзя создать анонимный класс, реализующий более одного интерфейса одновременно (через один синтаксис). Также нельзя создать анонимный класс, наследующий `enum` (перечисления финальны по природе).

40. Generics — что это, для чего, что происходит при компиляции. Wildcards

Generics (обобщения) — это механизм параметрического полиморфизма, введённый в Java 5, позволяющий создавать классы, интерфейсы и методы, работающие с типами-параметрами. Главная цель generics — обеспечить типобезопасность во время компиляции и устранить необходимость в явных приведениях типов. До generics написать `List<String>` было невозможно — использовался просто `List`, хранящий `Object`, и при извлечении элемента нужно было явно приводить тип, что могло вызывать `ClassCastException` во время выполнения.

С generics компилятор проверяет типы: `List<String> list = new ArrayList<>(); list.add(42);` — ошибка компиляции. Нельзя случайно добавить не тот тип.

Type Erasure (стирание типов) — это то, что происходит с generics при компиляции. JVM ничего не знает о generics во время выполнения. Компилятор Java использует информацию о типах generics только для проверки типобезопасности, а затем удаляет параметры типов из байт-кода и заменяет их на Object (или на верхнюю границу, если она указана). Вставляет необходимые касты автоматически. Это обеспечивает обратную совместимость с кодом, написанным до Java 5. Следствия type erasure: нельзя создать new T(), нельзя создать new T[10], нельзя использовать instanceof с параметром типа, нельзя перегружать методы, отличающиеся только параметрами типа (void foo(List<String>) и void foo(List<Integer>) после erasure станут одинаковыми).

Wildcards — символ ? в generics, обозначающий "неизвестный тип". Используется для повышения гибкости методов:

List<?> — список неизвестного типа. Из него можно читать (как Object), но нельзя добавлять элементы (кроме null).

List<? extends Animal> — верхняя ограниченная wildcard (upper bounded). Список чего-то, что является Animal или его подклассом. Из такого списка можно безопасно читать как Animal (producer), но нельзя добавлять элементы, потому что неизвестен точный тип.

List<? super Dog> — нижняя ограниченная wildcard (lower bounded). Список чего-то, что является Dog или его суперклассом. В такой список можно добавлять Dog (и подклассы), но при чтении получается Object.

Принцип PECS (Producer Extends, Consumer Super): если коллекция является источником данных (producer), используйте extends; если коллекция является получателем данных (consumer), используйте super.

41. Enum — что такое, область применения, некорректное использование

Enum (перечисление) — это специальный тип данных в Java, введенный в Java 5, позволяющий определить набор именованных констант. Enum является полноценным классом (неявно наследует от java.lang.Enum), является финальным (нельзя наследоваться от enum), и каждая константа перечисления является единственным экземпляром своего класса.

Enum может содержать поля, конструкторы, методы и даже абстрактные методы. Конструктор enum всегда private:

Java

```
public enum Planet {  
    MERCURY(3.303e+23, 2.4397e6),  
    VENUS(4.869e+24, 6.0518e6),  
    EARTH(5.976e+24, 6.37814e6);  
  
    private final double mass;  
    private final double radius;  
    Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }  
    double surfaceGravity() { return 6.67300E-11 * mass / (radius * radius); }  
}
```

Enum также имеет полезные встроенные методы: values() возвращает массив всех констант, valueOf(String) возвращает константу по имени, name() возвращает имя константы, ordinal() возвращает порядковый номер (0-based), compareTo() сравнивает по ordinal.

Область применения: дни недели, месяцы, стороны света, состояния (статусы заказа: NEW, IN_PROGRESS, DONE), типы (тип транспортного средства: CAR, TRUCK, BIKE), коды операций, константы конфигурации. Enum хорошо подходит для использования в switch. Enum можно использовать в EnumSet и EnumMap — специализированных высокопроизводительных коллекциях.

Некорректное использование: не стоит использовать `enum` для большого изменяемого набора значений (`enum` не предназначен для хранения данных из базы данных, которые могут меняться). Не стоит полагаться на `ordinal()` для хранения в БД или сериализации — добавление новых констант сдвинет `ordinal`. Не стоит использовать `enum` там, где вместо него лучше подойдет паттерн Стратегия или полиморфизм. Также неправильно сравнивать `enum` через `equals()` — для `enum` рекомендуется `==`, потому что каждая константа — единственный экземпляр (и `equals` для `enum` использует `==` внутри, но явное `==` нагляднее).

42. Отличия `Comparator` и `Comparable`

`Comparable<T>` — интерфейс из пакета `java.lang`, содержащий единственный метод `int compareTo(T o)`. Класс, реализующий `Comparable`, определяет своё естественное (натуральное) упорядочение. Логика сравнения встроена в сам класс. Примеры: `String`, `Integer`, `LocalDate` — все реализуют `Comparable`. Метод `compareTo` должен возвращать отрицательное число, если `this` меньше `o`, ноль — если равны, положительное — если `this` больше. Если класс реализует `Comparable`, его объекты можно сортировать с помощью `Collections.sort(list)` без дополнительных аргументов, и они могут автоматически упорядочиваться в `TreeSet` и `TreeMap`.

`Comparator<T>` — интерфейс из пакета `java.util`, содержащий метод `int compare(T o1, T o2)`. Компаратор — это отдельный объект, определяющий логику сравнения снаружи класса. Это позволяет иметь несколько различных порядков сортировки для одного класса, или сортировать классы, не реализующие `Comparable`, или сортировать объекты сторонних классов, исходный код которых изменить нельзя. С Java 8 `Comparator` является функциональным интерфейсом и может создаваться через лямбды: `Comparator<Person> byAge = (p1, p2) -> p1.getAge() - p2.getAge();`. Также в Java 8 добавлены удобные статические фабричные методы: `Comparator.comparing(Person::getName)`, `Comparator.comparingInt(Person::getAge)`, методы для цепочки `.thenComparing()` и для реверса `.reversed()`.

Когда что использовать: если у класса есть одно, очевидное и неизменное "естественное" упорядочение (как у чисел или строк) — реализуйте Comparable. Если нужна гибкость (разные варианты сортировки, или логика сортировки меняется) — используйте Comparator. На практике оба подхода часто комбинируются.

43. Класс Optional. Борьба с null. Хорошие и плохие примеры. orElse vs orElseGet

Optional<T> (пакет java.util) — это контейнер, который либо содержит ненулевое значение (Optional.of(value)), либо пустой (Optional.empty()). Он был введён в Java 8 прежде всего как возвращаемый тип методов, которые могут не возвращать результат, делая эту возможность явной и принуждая вызывающий код её обработать.

Хорошее использование: как возвращаемый тип метода, который может не найти результат:

Java

```
public Optional<User> findUserById(int id) {  
    return Optional.ofNullable(database.get(id)); // ofNullable принимает  
    // возможный null  
}  
// использование:  
findUserById(42)  
    .map(User::getName)  
    .orElse("Unknown");
```

Плохое / некорректное использование: использовать Optional как поле класса (это избыточно, сложнее сериализовывать, и Optional не реализует Serializable). Использовать Optional как параметр метода (лучше перегрузить метод или принять null). Вызывать optional.get() без предварительной проверки isPresent() — это выбросит NoSuchElementException, что ничуть не

лучше `NullPointerException`. Использовать `Optional` для каждого потенциально нулевого значения — это избыточно и создаёт накладные расходы. Паттерн `if (opt.isPresent()) { opt.get() }` — лучше использовать `ifPresent()` или `map()`.

Отличие `orElse()` от `orElseGet()`: это ключевое отличие. `orElse(T value)` принимает уже вычисленное значение — выражение, переданное в `orElse()`, всегда вычисляется, даже если `Optional` содержит значение и это вычисление не нужно. `orElseGet(Supplier<T> supplier)` принимает функцию (лямбду) и вызывает её только если `Optional` пустой. Поэтому если получение значения по умолчанию является дорогостоящей операцией (обращение к БД, создание тяжёлого объекта), нужно использовать `orElseGet()`. Если значение по умолчанию — простая константа — разницы практически нет, но `orElseGet()` идиоматически правильнее для производительного кода.

44. Принципы SOLID, YAGNI, KISS, DRY

Это набор фундаментальных принципов проектирования программного обеспечения.

SOLID — акроним из пяти принципов объектно-ориентированного проектирования.

S — **Single Responsibility Principle** (Принцип единственной ответственности): каждый класс должен иметь одну и только одну причину для изменения. Класс должен делать что-то одно и делать это хорошо. Если класс управляет бизнес-логикой, работой с базой данных и формированием отчётов одновременно — это нарушение SRP.

O — **Open/Closed Principle** (Принцип открытости/закрытости): программные сущности должны быть открыты для расширения, но закрыты для модификации. Новое поведение добавляется через наследование, интерфейсы или паттерны (*Strategy*, *Decorator*), а не через изменение существующего кода.

L — **Liskov Substitution Principle** (Принцип подстановки Лисков): объекты

подтипа должны быть взаимозаменяемы с объектами супертипа без нарушения корректности программы. Если `Square extends Rectangle`, то код, работающий с `Rectangle`, должен корректно работать и с `Square`. Нарушение LSP — классическая ловушка ненадлежащего наследования.

I — Interface Segregation Principle (Принцип разделения интерфейсов): клиент не должен зависеть от методов, которые он не использует. Лучше иметь несколько маленьких специализированных интерфейсов, чем один большой "толстый" интерфейс. Это предотвращает ситуацию, когда классы вынуждены реализовывать ненужные им методы.

D — Dependency Inversion Principle (Принцип инверсии зависимостей): модули верхнего уровня не должны зависеть от модулей нижнего уровня — оба должны зависеть от абстракций. Абстракции не должны зависеть от деталей — детали должны зависеть от абстракций. Это основа внедрения зависимостей (Dependency Injection) и IoC-контейнеров.

DRY (Don't Repeat Yourself — Не повторяйся): каждый фрагмент знания или логики должен иметь единственное, однозначное, авторитетное представление в системе. Дублирование кода — источник ошибок: при изменении логики нужно помнить об обновлении всех копий. Решение: выносить повторяющийся код в методы, классы, утилиты.

KISS (Keep It Simple, Stupid — Делай проще): системы работают лучше, если они просты, а не сложны. Не нужно усложнять решение сверх необходимого. Предпочитайте простой понятный код сложному "умному". Сложность — враг поддерживаемости.

YAGNI (You Aren't Gonna Need It — Тебе это не понадобится): не реализуйте функциональность, пока она реально не нужна. Не проектируйте "на будущее" фичи, которые ещё не востребованы. Это избыточный код, который усложняет систему, требует тестирования и поддержки, но не приносит пользы прямо сейчас.

45. ClassLoader. Static переменные при разных ClassLoader. Синглтон и несколько ClassLoader

ClassLoader (загрузчик классов) — это компонент JVM, отвечающий за поиск и загрузку байт-кода классов в память JVM во время выполнения. ClassLoader реализует принцип ленивой загрузки: класс загружается в память не при старте программы, а при первом обращении к нему. Загрузка класса включает три фазы: Loading (нахождение байт-кода и создание объекта Class в памяти), Linking (верификация байт-кода, подготовка статических полей, разрешение символических ссылок) и Initialization (выполнение статических инициализаторов и присвоение значений статическим полям).

В Java существует иерархия стандартных ClassLoader'ов. Bootstrap ClassLoader — встроен в JVM (написан на нативном коде), загружает классы из java.base модуля (ядро Java: java.lang, java.util и другие). Platform ClassLoader (до Java 9 — Extension ClassLoader) — загружает классы платформенных модулей. Application (System) ClassLoader — загружает классы из classpath приложения. Каждый ClassLoader (кроме Bootstrap) имеет родителя и использует принцип делегирования (parent delegation model): прежде чем загрузить класс самостоятельно, ClassLoader сначала просит родителя, и только если родитель не нашёл класс — загружает сам. Это гарантирует, что стандартные классы Java всегда загружаются Bootstrap ClassLoader'ом и не могут быть подменены.

Статические переменные при разных ClassLoader'ax: в JVM класс идентифицируется парой (имя класса + загрузивший его ClassLoader). Если один и тот же класс загружен двумя разными ClassLoader'ами, JVM считает их двумя разными классами. У каждого из них будут свои независимые статические переменные. Изменение статической переменной в классе, загруженном одним ClassLoader'ом, не влияет на "тот же" класс, загруженный другим ClassLoader'ом — они полностью изолированы. Именно этот механизм используется в серверах приложений (Tomcat, JBoss и другие) для изоляции разных веб-приложений, каждое из которых загружается своим

ClassLoader'ом, даже если они используют одни и те же библиотеки.

Синглтон и несколько ClassLoader'ов: синглтон (Singleton) — это паттерн, гарантирующий единственность экземпляра в JVM. Стандартная реализация синглтона (статическое поле `private static final INSTANCE`) гарантирует уникальность в рамках одного ClassLoader'a. Если класс синглтона загружен двумя разными ClassLoader'ами, каждый из них будет иметь свою независимую статическую переменную `INSTANCE`, и значит будет создано два экземпляра синглтона. Это означает, что в одной JVM формально могут существовать несколько экземпляров класса, объявленного как синглтон, если он загружен несколькими ClassLoader'ами. Это реальная проблема в окружениях серверов приложений. Для истинно уникального синглтона в рамках всей JVM его нужно загружать через Bootstrap ClassLoader, что на практике трудно реализуемо. Поэтому при разработке корпоративных приложений нужно понимать эту особенность и не полагаться на синглтоны там, где важна абсолютная уникальность в многоклассозагрузочных окружениях.

Java I/O, Exceptions, JUnit, Многопоточность — Подробные Ответы

46. Какие бывают потоки ввода/вывода?

В Java система ввода/вывода (I/O) построена вокруг понятия потока (stream) — последовательности данных, которая читается из источника или записывается в приёмник. Вся система I/O находится в пакете `java.io`, а начиная с Java 1.4 появилось также NIO в пакете `java.nio`.

Потоки делятся по нескольким критериям.

По направлению:

Потоки ввода (Input streams) — читают данные из источника (файл, сеть, массив байтов, клавиатура). Потоки вывода (Output streams) — записывают данные в приёмник.

По типу данных:

Байтовые потоки (byte streams) — работают с данными в виде байтов (8 бит).

Базовые абстрактные классы: `InputStream` для чтения и `OutputStream` для записи. Используются для работы с любыми бинарными данными: изображения, аудио, видео, сериализованные объекты. Основные реализации: `FileInputStream` / `FileOutputStream` — работа с файлами; `ByteArrayInputStream` / `ByteArrayOutputStream` — работа с массивом байтов в памяти; `BufferedInputStream` / `BufferedOutputStream` — буферизованные обёртки, значительно ускоряющие работу за счёт буферизации; `DataInputStream` / `DataOutputStream` — работа с примитивными Java-типами; `ObjectInputStream` / `ObjectOutputStream` — сериализация/десериализация объектов; `PrintStream` — форматированный вывод (именно этот класс стоит за `System.out`).

Символьные потоки (character streams) — работают с данными в виде символов (chars, 16 бит, Unicode). Базовые абстрактные классы: `Reader` и `Writer`. Автоматически обрабатывают кодировку символов (преобразование байтов в символы и обратно). Основные реализации: `FileReader` / `FileWriter` — чтение/запись символов из файла (используют кодировку платформы по умолчанию); `BufferedReader` / `BufferedWriter` — буферизованные обёртки с дополнительными методами (например, `readLine()`); `InputStreamReader` / `OutputStreamWriter` — мост между байтовыми и символьными потоками, позволяющий указать нужную кодировку; `StringReader` / `StringWriter` — работа со строками; `PrintWriter` — форматированный вывод с автоматическим сбросом буфера.

По уровню (прямые и обёрточные): Прямые потоки (node streams) напрямую связаны с источником данных (файл, массив). Потоки-обёртки (filter/wrapper streams) оборачивают другие потоки и добавляют функциональность (буферизация, форматирование, подсчёт строк). Это паттерн Декоратор — потоки можно "оборачивать" многократно: `new BufferedReader(new InputStreamReader(new FileInputStream("file.txt"), "UTF-8"))`.

NIO (New I/O, пакет `java.nio`): Начиная с Java 1.4 появился NIO с концепцией каналов (Channel) и буферов (Buffer). Каналы — двунаправленные (в отличие от потоков), поддерживают неблокирующий I/O. Буферы — контейнеры для

данных фиксированного размера. Селекторы (Selector) позволяют одному потоку управлять несколькими каналами. Java 7 добавила NIO.2 с новым API для работы с файловой системой через Path, Files, Paths — значительно более удобным, чем старый File.

47. Сериализация: что это, зачем нужна, transient, static, serialVersionUID

Сериализация — это процесс преобразования состояния объекта в последовательность байтов для сохранения или передачи. Десериализация — обратный процесс восстановления объекта из последовательности байтов.

Сериализация применяется в следующих ситуациях: сохранение состояния объекта на диск (персистентность); передача объектов по сети (RMI — Remote Method Invocation, сетевые приложения); кеширование объектов; копирование объектов через сериализацию/десериализацию (deep copy); сессии в веб-приложениях (объекты сессии сериализуются).

Для того чтобы объект можно было сериализовать стандартным механизмом Java, его класс должен реализовывать интерфейс `java.io.Serializable`. Это маркерный интерфейс — он не содержит методов. Сериализация выполняется через `ObjectOutputStream.writeObject(obj)`, десериализация — через `ObjectInputStream.readObject()`. При десериализации конструктор не вызывается — JVM восстанавливает объект напрямую из байтов. Если в иерархии наследования суперкласс не реализует `Serializable`, его поля не сериализуются, и при десериализации конструктор суперкласса без параметров вызывается.

Ключевое слово `transient` — поля, помеченные `transient`, исключаются из сериализации. При десериализации такие поля получают значения по умолчанию (0, null, false). Используется для: паролей и конфиденциальных данных (не должны сохраняться), полей, которые не имеют смысла после перезапуска (например, соединение с БД), полей, которые можно вычислить из других (кешированные значения), полей не-сериализуемых типов (если

поле нельзя сделать `Serializable`, оно должно быть `transient`, иначе сериализация упадёт с `NotSerializableException`).

Сериализация `static`-полей: статические поля не сериализуются. Статические поля принадлежат классу, а не объекту, и сериализация сохраняет состояние объекта. Поэтому при десериализации статические поля получают текущее значение, определённое классом (либо из статического инициализатора, либо значение по умолчанию), а не то, что было при сериализации. Это логично, потому что статические поля и так должны быть одинаковы для всех объектов класса.

`serialVersionUID` — это поле типа `private static final long serialVersionUID = 1L;`, которое является уникальным идентификатором версии класса для целей сериализации. Когда JVM сериализует объект, она записывает `serialVersionUID` в поток байтов. При десериализации JVM сравнивает `serialVersionUID` в потоке с `serialVersionUID` текущей версии класса. Если они не совпадают — выбрасывается `InvalidClassException`, защищая от загрузки несовместимых версий. Если не объявить `serialVersionUID` явно, компилятор сгенерирует его автоматически на основе структуры класса (полей, методов). Это опасно: любое изменение класса (добавление поля, изменение подписи метода) изменит автоматически сгенерированный UID, и сохранённые ранее объекты не смогут быть десериализованы. Поэтому рекомендуется всегда объявлять `serialVersionUID` явно.

48. Сериализация объекта в XML вместо байт-кода

Да, это абсолютно возможно, и существует несколько подходов.

Стандартный механизм сериализации Java (`ObjectOutputStream`) сохраняет объект в бинарный (байтовый) формат, специфичный для Java. Но есть много ситуаций, когда нужен более читаемый, переносимый или совместимый с другими языками формат, такой как XML или JSON.

JAXB (Java Architecture for XML Binding) — стандартный Java-способ работы

с XML. Входил в состав Java SE до Java 11, затем был перемещён в отдельный модуль (jakarta.xml.bind). С помощью аннотаций `@XmlRootElement`, `@XmlElement`, `@XmlAttribute` можно разметить класс, а затем использовать `Marshaller` для записи объекта в XML и `Unmarshaller` для чтения. Это по сути XML-сериализация.

`XStream` — популярная сторонняя библиотека, которая преобразует объекты Java в XML и обратно с минимальными настройками. Не требует специальных аннотаций или реализации интерфейсов.

Jackson с модулем XML — библиотека Jackson, широко известная для JSON-сериализации, имеет модуль `jackson-dataformat-xml` для работы с XML.

JSON-сериализация как альтернатива XML: библиотеки Jackson (`ObjectMapper`), Gson, Moshi позволяют сериализовать объекты в JSON-строку или файл. JSON сегодня гораздо более распространён, чем XML, в API и микросервисах.

Таким образом, понятие "сериализация" шире стандартного Java-механизма и включает любое преобразование объекта в другое представление (JSON, XML, YAML, Protocol Buffers и другие). Главное отличие нестандартной сериализации от стандартной — она не зависит от `Serializable`, более читаема и переносима, но требует сторонних библиотек или дополнительной настройки.

49. Иерархия исключений

Иерархия исключений в Java является частью общей иерархии классов и берёт начало от класса `Throwable`.

На вершине стоит класс `Throwable` — это базовый класс для всего, что можно выбросить с помощью `throw` и поймать в блоке `catch`. У `Throwable` есть два прямых подкласса: `Error` и `Exception`.

Error — это серьёзные проблемы, которые обычно невозможно или бессмысленно обрабатывать в прикладном коде. Они сигнализируют о критических сбоях в работе JVM или системы. Примеры: `OutOfMemoryError` (закончилась память кучи), `StackOverflowError` (переполнение стека вызовов), `VirtualMachineError`, `AssertionError`, `NoClassDefFoundError`. Error и его подклассы являются `unchecked` — компилятор не требует их обработки.

Exception — это исключения, которые представляют проблемы на уровне приложения и обычно могут быть обработаны. Делится на две ветви.

Первая ветвь — `checked exceptions` (проверяемые исключения): все подклассы `Exception`, кроме `RuntimeException` и его потомков. Компилятор требует их явной обработки — метод должен либо ловить такое исключение (`try-catch`), либо объявлять его в сигнатуре (`throws`). Примеры: `IOException`, `FileNotFoundException`, `SQLException`, `ClassNotFoundException`, `CloneNotSupportedException`, `InterruptedException`.

Вторая ветвь — `RuntimeException` и его потомки (`unchecked exceptions`, непроверяемые): компилятор не требует их обработки. Обычно сигнализируют о программных ошибках — ошибках в логике программы, которые можно избежать корректным кодом. Примеры: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ClassCastException`, `IllegalArgumentException`, `IllegalStateException`, `ArithmeticException` (деление на ноль), `NumberFormatException`, `UnsupportedOperationException`, `ConcurrentModificationException`.

Класс `Throwable` и все его наследники содержат полезные методы: `getMessage()` — возвращает сообщение об ошибке, `getCause()` — возвращает причину (вложенное исключение), `printStackTrace()` — выводит трассировку стека, `getStackTrace()` — возвращает массив элементов трассировки.

50. Checked и unchecked исключения: отличия

`Checked exceptions` (проверяемые) — это исключения, которые компилятор

Java требует обязательно обработать. Если метод выбрасывает checked-исключение, вызывающий код должен либо поймать его в блоке try-catch, либо объявить throws в своей сигнатуре (перебросив ответственность выше). Философия checked исключений состоит в том, что это ситуации, которые программист должен предусмотреть и обработать: файл не найден, нет соединения с БД, некорректный формат данных из внешнего источника. Типичные представители: IOException, SQLException, ParseException.

Unchecked exceptions (непроверяемые) — компилятор не требует их обработки. Это либо RuntimeException и его потомки, либо Error. Философия состоит в том, что это либо ошибки программиста (NullPointerException — ты не проверил на null, ArrayIndexOutOfBoundsException — ты вышел за границу массива), либо неустраняемые системные проблемы (Error). Обращивать каждое из них в каждом методе было бы крайне громоздко и нечитаемо.

Споры о том, когда использовать checked, а когда unchecked, не утихают в сообществе Java-разработчиков. Многие современные фреймворки (Spring, например) предпочитают unchecked исключения, оборачивая checked в runtime-варианты. Аргументы против checked: они загромождают сигнатуры методов, нарушают принцип инкапсуляции (детали реализации в виде конкретных исключений "протекают" через API), приводят к антипаттерну "проглатывания" исключений (пустой catch-блок), сложно обрабатываются в функциональном коде (лямбды не могут бросать checked исключения без обёртки). Аргументы за checked: они явно документируют возможные сбои, заставляют программиста задуматься об обработке ошибок.

51. Работа try-catch-finally. Без catch. Порядок catch. Multicatch

Блок try-catch-finally — основной механизм обработки исключений в Java.

try: в этот блок помещается код, который потенциально может выбросить исключение. Когда в блоке try выбрасывается исключение, выполнение

немедленно прерывается и управление передаётся в соответствующий блок `catch`.

`catch`: блок (или несколько блоков) для перехвата исключений определённого типа. Каждый `catch` указывает тип исключения, которое он обрабатывает. JVM ищет первый `catch`-блок, тип которого совместим с выброшенным исключением (тот же класс или суперкласс). Как только подходящий `catch` найден, выполняется его код, и остальные `catch`-блоки пропускаются.

`finally`: этот блок выполняется всегда после `try` и `catch`, независимо от того, было ли выброшено исключение, было ли оно поймано, или даже если в `try` или `catch` есть оператор `return`. Используется для освобождения ресурсов. Единственные случаи, когда `finally` не выполняется: вызов `System.exit()`, крэш JVM (фатальная ошибка), или поток был убит принудительно.

Может ли блок работать без `catch`? Да! Конструкция `try-finally` (без `catch`) является абсолютно допустимой. Это используется, когда нужно гарантировать выполнение финального кода (освобождение ресурса), но обрабатывать исключение в данном методе не нужно — исключение "пройдёт сквозь" и будет выброшено вызывающему коду:

Java

```
try {  
    doSomething();  
} finally {  
    closeResource(); // выполнится в любом случае  
}
```

Порядок блоков `catch`: блоки `catch` должны располагаться от более специфичного к более общему типу исключения. Если поставить `catch(Exception e)` перед `catch(IOException e)`, то второй блок никогда не выполнится, и компилятор выдаст ошибку. Всегда сначала ловите потомков, затем предков:

Java

```
try { ... }  
catch (FileNotFoundException e) { ... } // сначала более специфичное  
catch (IOException e) { ... }          // затем более общее  
catch (Exception e) { ... }           // и самое общее в конце
```

Multi-catch (множественный catch), введённый в Java 7: позволяет в одном блоке catch поймать несколько разных типов исключений, разделив их символом |. Это устраняет дублирование кода обработки:

Java

```
try { ... }  
catch (IOException | SQLException | ParseException e) {  
    logger.error("Ошибка: ", e);  
}
```

Переменная e в multi-catch является implicitly final — ей нельзя присвоить новое значение внутри блока.

52. Блок try-with-resources

Try-with-resources был введён в Java 7 (JEP в рамках Project Coin) и является синтаксическим сахаром для автоматического закрытия ресурсов. Ресурс объявляется в скобках после try, и JVM гарантирует вызов метода close() на этом ресурсе после выхода из блока try, в любом случае — даже если выброшено исключение.

Для того чтобы ресурс можно было использовать в try-with-resources, его класс должен реализовывать интерфейс java.lang.AutoCloseable (или его подинтерфейс java.io.Closeable), содержащий единственный метод void close() throws Exception.

Java

```
try (BufferedReader reader = new BufferedReader(new FileReader("file.txt"));
    Connection conn = dataSource.getConnection()) {
    // работа с ресурсами
    String line = reader.readLine();
} catch (IOException e) {
    // обработка исключения
}
// reader и conn автоматически закрыты здесь
```

Можно объявить несколько ресурсов, разделив их точкой с запятой. Ресурсы закрываются в обратном порядке объявления (последний объявленный закрывается первым).

Важная деталь о подавленных исключениях (suppressed exceptions): если и в блоке try, и в методе close() выбрасываются исключения, основным считается исключение из try, а исключение из close() добавляется к нему как "подавленное" (addSuppressed()). Получить подавленные исключения можно через e.getSuppressed(). В старом коде с try-finally второе исключение полностью перекрывало первое, что затрудняло отладку.

Начиная с Java 9 переменную ресурса можно объявить за пределами try, если она effectively final: try (existingResource) { ... } — без повторного объявления типа.

53. Что такое Error. Перехват Error: есть ли смысл, причины и действия

Error — это подкласс Throwable, представляющий серьёзные проблемы, которые обычно являются неустранимыми с точки зрения приложения. Error сигнализирует о проблемах на уровне JVM или среды выполнения, а не логики приложения.

Основные виды Error:

`OutOfMemoryError` — JVM не может выделить память для нового объекта (куча переполнена).

`StackOverflowError` — стек вызовов переполнен (бесконечная рекурсия).

`NoClassDefFoundError` — класс был найден при компиляции, но недоступен при выполнении (удалён из classpath).

`AssertionError` — нарушено утверждение (оператор `assert`).

`VirtualMachineError` — внутренняя ошибка JVM.

`ThreadDeath` — поток был остановлен через устаревший метод `Thread.stop()`.

Есть ли смысл в перехвате `Error`? В подавляющем большинстве случаев — нет. `Error` сигнализирует о ситуации, из которой приложение не может корректно восстановиться. Перехватывать `OutOfMemoryError` бессмысленно: если памяти нет, вы скорее всего не сможете ничего сделать, не выделяя новой памяти. `StackOverflowError` при перехвате может быть повторно выброшен при попытке обработки.

Однако есть исключения — редкие случаи, когда перехват имеет смысл. Например, в контейнерах серверов приложений и фреймворках, которые должны продолжать работу даже если один запрос привёл к `Error` — тогда `Error` перехватывается, логируется и поток обработки запроса прекращается, но сервер остаётся жив. Также при перехвате `AssertionError` в тестовых фреймворках.

Если всё же перехватывать, то ловить нужно конкретный тип `Error`, а не `Throwable` или `Error` в целом. После перехвата — логировать как можно меньшим количеством кода (чтобы самим не вызвать новый `Error`) и завершать работу через `System.exit()`.

54. OutOfMemoryError vs StackOverflowError. Как избежать?

StackOverflowError возникает, когда стек вызовов (call stack) переполняется. Каждый вызов метода добавляет фрейм в стек. Когда глубина рекурсии или цепочка вызовов превышает максимально допустимый размер стека — выбрасывается StackOverflowError. Наиболее частая причина — бесконечная рекурсия (метод прямо или косвенно вызывает сам себя без условия выхода). Размер стека каждого потока регулируется JVM-флагом -Xss (например, -Xss2m — 2 мегабайта на стек потока).

Как избежать: проверять базовый случай рекурсии, убедиться, что рекурсия всегда завершится. Заменять глубокую рекурсию итеративным алгоритмом (например, используя явный стек на Deque). Использовать хвостовую рекурсию (хотя JVM HotSpot не оптимизирует её автоматически). Увеличить размер стека через -Xss, если рекурсия обоснованно глубока.

OutOfMemoryError возникает, когда JVM не может выделить память для нового объекта в куче (heap). Существуют разные варианты этой ошибки с разными сообщениями: OutOfMemoryError: Java heap space — куча переполнена; OutOfMemoryError: GC overhead limit exceeded — GC тратит слишком много времени на сборку мусора, освобождая слишком мало памяти; OutOfMemoryError: Metaspace — переполнено метапространство (слишком много загруженных классов); OutOfMemoryError: Direct buffer memory — нехватка памяти для NIO direct buffers.

Причины OutOfMemoryError: утечки памяти (объекты удерживаются ссылками и не могут быть собраны GC), создание огромных структур данных, загрузка слишком большого файла в память целиком, бесконечное добавление объектов в статические коллекции.

Как избежать: использовать профилировщики памяти (VisualVM, YourKit, JProfiler) для обнаружения утечек. Увеличить размер кучи через флаги -Xms (начальный) и -Xmx (максимальный). Обработать данные потоком, а не загружая в память целиком. Использовать слабые ссылки (WeakReference,

SoftReference) для кешей. Вовремя закрывать ресурсы. Не хранить ненужные объекты в статических коллекциях.

55. Оператор throw. Как работает? Свойства

Оператор throw используется для явного выброса исключения. Синтаксис: throw выражение;, где выражение должно иметь тип Throwable или его подкласса.

При выполнении throw текущий поток немедленно прекращает нормальное выполнение. JVM начинает поиск обработчика исключения, просматривая стек вызовов снизу вверх: сначала в текущем методе, затем в вызывающем, затем в вызывающем вызывающего и так далее. Если подходящий catch-блок найден — выполняется. Если стек исчерпан и обработчик не найден — поток завершается с выводом трассировки стека в System.err.

Если метод выбрасывает checked-исключение, это должно быть объявлено в сигнатуре через throws:

Java

```
public void readFile(String path) throws IOException {  
    throw new IOException("Файл не найден: " + path);  
}
```

Оператор throw также используется для пробрасывания исключения (rethrowing): поймав исключение в catch, можно выбросить его снова (throw e;) или обернуть в другое исключение (throw new ServiceException("Ошибка", e;)). Это называется exception chaining — исходное исключение передаётся как cause нового.

Начиная с Java 7 если вы ловите исключение определённого типа и перебрасываете переменную без изменений, компилятор достаточно умён, чтобы вывести точный тип выброшенного исключения (а не Exception), что позволяет объявлять в throws более точный тип.

После оператора `throw` в том же блоке код недостижим — компилятор выдаст ошибку `unreachable statement`.

56. Механизм `assert`. `AssertionError`: когда возникает, смысл

Оператор `assert` в Java (введён в Java 1.4) предназначен для проверки утверждений — условий, которые, по мнению разработчика, всегда должны быть истинными в данной точке программы. Синтаксис: `assert условие;` или `assert условие : сообщение;`.

Если условие истинно — ничего не происходит. Если условие ложно — выбрасывается `AssertionError` (наследник `Error`) с опциональным сообщением. `AssertionError` является `unchecked` и наследует от `Error`, что означает, что его не нужно объявлять в `throws` и что он не предназначен для перехвата в обычном коде.

Критически важная деталь: по умолчанию `assertions` отключены в JVM. Для их включения нужно запускать JVM с флагом `-ea` (`enable assertions`) или `-enableassertions`. В продакшн-окружении `assertions` обычно отключены. Включать их рекомендуется во время разработки и тестирования.

`Assertions` используются для: проверки предусловий (в начале метода: "этот параметр не должен быть `null`"), проверки постусловий (в конце метода: "результат не может быть отрицательным"), проверки инвариантов (состояние объекта после операции), документирования ожиданий (это самодокументирующийся код).

Чего не стоит делать с `assertions`: не используйте `assert` для проверки параметров публичных методов (используйте явные `if` + `throw IllegalArgumentException`, потому что `assertions` могут быть отключены), не используйте `assert` для кода с побочными эффектами (потому что при отключённых `assertions` этот код не выполнится).

В контексте JUnit слово "assert" имеет другое значение — методы `Assertions.assertEquals()`, `Assertions.assertTrue()` и другие являются методами тестового фреймворка и не связаны с оператором `assert`.

57. JUnit. Что это такое? Принципы написания

JUnit — это самый популярный фреймворк для написания и запуска автоматических тестов на Java. Текущая версия — JUnit 5 (также называемая JUnit Jupiter), которая состоит из трёх модулей: JUnit Platform (платформа запуска тестов), JUnit Jupiter (новый API для написания тестов), JUnit Vintage (поддержка старых тестов JUnit 3 и 4).

Принципы написания тестов в JUnit 5:

Тестовый класс — обычный Java-класс, не нужно наследовать ни от каких базовых классов. Тестовые методы помечаются аннотацией `@Test`. Методы могут быть пакетно-приватными (в JUnit 5 не нужно объявлять `public`).

Основные аннотации: `@Test` — тестовый метод; `@BeforeEach` — выполняется перед каждым тестовым методом (настройка состояния, `setup`); `@AfterEach` — выполняется после каждого тестового метода (очистка, `teardown`); `@BeforeAll` — выполняется один раз перед всеми тестами в классе (должен быть `static`); `@AfterAll` — один раз после всех тестов (`static`); `@Disabled` — отключить тест; `@DisplayName` — читаемое имя теста; `@ParameterizedTest` — параметризованный тест с разными входными данными; `@RepeatedTest` — повторить тест N раз; `@Nested` — вложенные тестовые классы для группировки.

Проверки выполняются через методы класса `Assertions`: `assertEquals(expected, actual)`, `assertNotNull(obj)`, `assertTrue(condition)`, `assertThrows(Exception.class, () -> methodCall())`, `assertAll()` — группировка нескольких проверок.

Принципы FIRST для хороших тестов: Fast (быстрые), Independent (независимые друг от друга), Repeatable (воспроизводимые при каждом

запуске), Self-validating (сами сообщают о результате), Timely (написанные вовремя, в TDD — до кода). Также принцип AAA (Arrange-Act-Assert): подготовить данные, выполнить действие, проверить результат.

58. Stub vs Mock: в чём разница?

Stubs и Mocks — это тестовые дублёры (test doubles), которые заменяют реальные зависимости в тестах, но служат разным целям.

Stub (заглушка) — это объект, который возвращает заранее подготовленные данные в ответ на вызовы методов. Stub предоставляет нужные данные тестируемому коду, но тест не проверяет, как именно и сколько раз был вызван stub. Stub отвечает на вопрос "что вернуть", а не "был ли вызван". Пример: заглушка для репозитория, которая всегда возвращает фиксированного пользователя, чтобы тест мог проверить бизнес-логику.

Mock (имитация) — это объект, который не только возвращает данные, но и записывает все взаимодействия с собой и позволяет проверить, что нужные методы были вызваны нужное количество раз с нужными аргументами. Mock отвечает на вопрос "как был использован?". После выполнения теста делаются проверки (verify) на mock-объекте.

Концептуально: stub — это state-based testing (проверяем результирующее состояние), mock — это behaviour-based testing / interaction testing (проверяем взаимодействие между объектами).

В Mockito один и тот же объект, созданный через @Mock или Mockito.mock(), можно использовать и как stub (настраивая возвращаемые значения через when().thenReturn()) и как mock (проверяя вызовы через verify()). Разделение stub/mock в Mockito — концептуальное, а не на уровне разных классов.

59. @Mock, @Spy, @InjectMocks: разница и принцип работы

@Mock создаёт полную имитацию объекта. Все методы mock-объекта по умолчанию возвращают значения по умолчанию: null для объектов, 0 для

числовых типов, false для boolean, пустые коллекции для коллекций. Реальный код класса не вызывается. Нужно явно настраивать поведение через `when(mock.method()).thenReturn(value)`.

`@Spy` создаёт "шпиона" на основе реального объекта. В отличие от `Mock`, `Spy` вызывает реальный код методов класса по умолчанию. Можно выборочно "заглушить" отдельные методы через `doReturn(value).when(spy).method()` (именно в таком порядке, потому что `when(spy.method())` вызвал бы реальный код). `Spy` полезен, когда нужно протестировать реальную реализацию класса, но изолировать несколько конкретных методов (например, те, что обращаются к внешним ресурсам).

`@InjectMocks` создаёт реальный экземпляр тестируемого класса и внедряет в него `Mock` и `Spy` объекты, аннотированные `@Mock` и `@Spy`. Mockito пытается внедрить зависимости следующими способами по порядку: через конструктор (если подходящий конструктор найден), через сеттеры (ищет сеттеры, принимающие типы моков), через поля напрямую (через рефлексия, даже приватные). Это позволяет тестировать реальный код тестируемого класса, заменив только его зависимости на моки.

Java

```
@ExtendWith(MockitoExtension.class)
```

```
class UserServiceTest {
```

```
    @Mock UserRepository userRepository; // полная имитация
```

```
    @Spy EmailService emailService; // реальный объект со шпионом
```

```
    @InjectMocks UserService userService; // реальный UserService с  
    внедрёнными моками
```

```
}
```

60. Жизненный цикл Mockito в JUnit 5. Как заставить работать

@Mock

В JUnit 5 аннотации Mockito (`@Mock`, `@Spy`, `@InjectMocks`) не работают "из коробки" — нужно явно подключить поддержку Mockito одним из способов.

Способ 1 — аннотация `@ExtendWith(MockitoExtension.class)` на тестовом классе. Это самый рекомендуемый и чистый способ. `MockitoExtension` является `JUnit 5 Extension`, который интегрируется в жизненный цикл `JUnit` и инициализирует все аннотированные поля перед каждым тестом:

Java

```
@ExtendWith(MockitoExtension.class)
class MyServiceTest {
    @Mock MyRepository repo;
    @InjectMocks MyService service;

    @Test
    void testSomething() {
        when(repo.find(1)).thenReturn(new Entity());
        // ...
        verify(repo).find(1);
    }
}
```

Способ 2 — `MockitoAnnotations.openMocks(this)` в методе `@BeforeEach`. Это ручной способ инициализации аннотаций, нужный если нельзя использовать `Extension`:

Java

```
@BeforeEach
void setup() {
    MockitoAnnotations.openMocks(this);
}
```

Жизненный цикл: По умолчанию `JUnit 5` создаёт новый экземпляр тестового класса для каждого тестового метода. `Mockito Extension` создаёт новые моки перед каждым тестом и сбрасывает их состояние после. Это гарантирует

изоляция тестов — взаимодействия и настройки одного теста не влияют на другой.

61. Пирамида тестирования

Пирамида тестирования (Test Pyramid) — это концептуальная модель, описывающая, какие типы тестов и в каком соотношении должны присутствовать в проекте. Идея предложена Майком Коном.

Основание пирамиды — Unit-тесты (модульные): самый большой слой, их должно быть большинство (60-70%). Тестируют минимальную единицу кода — один метод или один класс — изолированно от остальных зависимостей (через моки). Очень быстрые (миллисекунды), дешёвые в написании, легко находят место ошибки. Дают быструю обратную связь разработчику.

Средний слой — Integration-тесты (интеграционные): их должно быть меньше (20-30%). Тестируют взаимодействие нескольких компонентов вместе: например, сервис + репозиторий + реальная база данных, или несколько модулей. Медленнее unit-тестов, требуют инфраструктуры (БД, очередь сообщений). Выявляют проблемы интеграции компонентов.

Верхушка пирамиды — End-to-End тесты (E2E / UI-тесты): их должно быть мало (5-10%). Тестируют всю систему целиком с точки зрения пользователя — от UI до базы данных. Самые медленные (минуты), хрупкие (ломаются из-за любого изменения UI), дорогие в поддержке. Но дают уверенность, что основные пользовательские сценарии работают.

Антипаттерн — "перевернутая пирамида" или "мороженое": много E2E тестов, мало unit. Это приводит к медленным, нестабильным и дорогим в поддержке наборам тестов. Также антипаттерн — "бублик" (много E2E и unit, мало интеграционных). Правильно сбалансированная пирамида обеспечивает быструю обратную связь, высокое покрытие и разумную стоимость поддержки.

62. Процесс и поток. Состояния потока

Процесс — это экземпляр запущенной программы. Каждый процесс имеет собственное изолированное адресное пространство памяти, ресурсы ОС (файловые дескрипторы, сетевые соединения), минимум один поток выполнения. Процессы изолированы друг от друга — один процесс не может напрямую обратиться к памяти другого. Взаимодействие между процессами (IPC) осуществляется через специальные механизмы: файлы, сокеты, pipes, разделяемая память.

Поток (thread) — это наименьшая единица выполнения внутри процесса. Все потоки одного процесса разделяют общую память (heap, Metaspace, статические поля), но каждый имеет собственный стек вызовов и PC-регистр. Потоки внутри одного процесса могут напрямую обращаться к общим данным — это делает их взаимодействие эффективным, но требует синхронизации для предотвращения гонок данных.

Состояния потока в Java определяются перечислением Thread.State:

NEW — поток создан (new Thread(...)), но метод start() ещё не вызван.

RUNNABLE — поток выполняется или готов к выполнению (ожидает процессорного времени от планировщика ОС). В Java оба состояния (executing и ready) объединены в RUNNABLE.

BLOCKED — поток заблокирован, ожидая получить монитор объекта (например, пытается войти в synchronized-блок, занятый другим потоком).

WAITING — поток бесконечно ждёт уведомления от другого потока. В это состояние поток переходит при вызове Object.wait() без таймаута, Thread.join() без таймаута, LockSupport.park().

TIMED_WAITING — поток ждёт с таймаутом. Переходит при: Thread.sleep(millis), Object.wait(millis), Thread.join(millis),

LockSupport.parkNanos(), LockSupport.parkUntil().

TERMINATED — поток завершил выполнение метода run() или был прерван исключением.

63. Как создать поток? Способы запуска

В Java есть несколько способов создания и запуска потоков.

Способ 1 — наследование от Thread: создать подкласс Thread и переопределить метод run(). Минус: класс не может наследовать ничего другого (Java не поддерживает множественное наследование), и смешивает задачу ("что делать") с механизмом ("как запустить поток").

Java

```
class MyThread extends Thread {  
    @Override  
    public void run() { System.out.println("Работает в потоке: " + getName()); }  
}
```

new MyThread().start(); // start() создаёт системный поток и вызывает run()

Способ 2 — реализация Runnable: реализовать интерфейс Runnable и передать экземпляр конструктору Thread. Предпочтительный способ, отделяет задачу от потока. Runnable — функциональный интерфейс, поэтому можно использовать лямбду:

Java

```
Thread t = new Thread(() -> System.out.println("Лямбда в потоке"));  
t.start();
```

Способ 3 — Callable и Future: Callable<V> похож на Runnable, но может возвращать результат и выбрасывать checked исключения. Используется совместно с ExecutorService:

Java

```
Callable<Integer> task = () -> { return 42; };
```

```
Future<Integer> future = executorService.submit(task);
```

```
Integer result = future.get(); // блокирует, пока результат не готов
```

Способ 4 — `ExecutorService` (рекомендуемый для производственного кода): вместо ручного создания потоков использовать пул потоков. `Executors.newFixedThreadPool(4)` создаёт пул из 4 потоков. Задачи (`Runnable` или `Callable`) передаются через `submit()` или `execute()`.

Способ 5 — `CompletableFuture` (Java 8+): для асинхронного программирования с богатым API для составления цепочек операций.

Важно: `start()` создаёт новый системный поток и асинхронно вызывает `run()`. Вызов `run()` напрямую — это просто обычный вызов метода в текущем потоке, никакого нового потока не создаётся.

64. Как корректно завершить работу потока

Принудительного надёжного способа "убить" поток извне в Java не существует — метод `Thread.stop()` был объявлен устаревшим ещё в Java 1.1 из-за серьёзных проблем безопасности (он мог оставить разделяемые объекты в неконсистентном состоянии).

Правильный способ 1 — флаг прерывания (`volatile boolean`):

Java

```
class WorkerThread extends Thread {  
    private volatile boolean running = true;
```

```
    @Override
```

```
    public void run() {  
        while (running) {
```

```

        doWork();
    }
}

public void stopThread() { running = false; }
}

```

Поток сам проверяет флаг в своём цикле и завершается. Ключевое слово `volatile` необходимо для видимости изменения флага в разных потоках.

Правильный способ 2 — механизм прерывания потока (`Thread.interrupt()`):

Это стандартный и предпочтительный Java-механизм. `thread.interrupt()` устанавливает флаг прерывания потока. Поток должен сам проверять его через `Thread.currentThread().isInterrupted()` или через обработку `InterruptedException`. Многие блокирующие методы (`sleep()`, `wait()`, `join()`) выбрасывают `InterruptedException` при установке флага прерывания, что позволяет потоку корректно завершиться из заблокированного состояния:

Java

```

@Override
public void run() {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            doWork();
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // восстановить флаг прерывания
            break; // корректно завершиться
        }
    }
}
}

```

Важно: при перехвате `InterruptedException` не "проглатывать" его — нужно

либо повторно прерывать поток (`Thread.currentThread().interrupt()`), либо пробрасывать исключение выше.

65. Как выполнить набор команд в отдельном потоке

Самый простой способ — обернуть код в лямбду `Runnable` и передать в `Thread`:

Java

```
Thread t = new Thread() -> {  
    command1();  
    command2();  
    command3();  
});  
t.start();
```

Для повторного использования и управления ресурсами — использовать `ExecutorService`:

Java

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
executor.submit() -> {  
    command1();  
    command2();  
    command3();  
});  
executor.shutdown(); // больше не принимает задачи, завершает  
выполняющиеся
```

Если нужен результат — `Callable` и `Future`:

Java

```
Future<Result> future = executor.submit() -> {
```

```
command1();  
return computeResult();  
});  
Result result = future.get(); // ожидаем завершения и получаем результат
```

Для сложной асинхронной логики — `CompletableFuture`:

Java

```
CompletableFuture.runAsync(() -> command1())  
    .thenRun(() -> command2())  
    .thenRun(() -> command3())  
    .join(); // ожидать завершения
```

66. Как работают методы `wait()` и `notify()/notifyAll()`

Методы `wait()`, `notify()` и `notifyAll()` определены в классе `Object` и являются механизмом координации потоков через монитор объекта. Ключевое требование: эти методы можно вызывать только внутри синхронизированного блока (`synchronized`), иначе выбрасывается `IllegalMonitorStateException`.

`wait()`: вызывающий поток освобождает монитор объекта и переходит в состояние `WAITING`. Поток остаётся в этом состоянии, пока другой поток не вызовет `notify()` или `notifyAll()` на том же объекте, или пока не истечёт таймаут (для `wait(long millis)`). После пробуждения поток пытается снова получить монитор и продолжить выполнение.

`notify()`: пробуждает один случайный поток, ожидающий на данном объекте. Монитор при этом не освобождается — он освобождается, когда вызывающий поток покидает `synchronized`-блок. Пробуждённый поток конкурирует с другими потоками за получение монитора.

`notifyAll()`: пробуждает все потоки, ожидающие на данном объекте. Все они конкурируют за монитор, и только один получает его — остальные продолжают ждать.

Важная деталь — spurious wakeups (ложные пробуждения): поток может быть разбужен без вызова notify(). Поэтому условие ожидания нужно проверять в цикле while, а не if:

Java

```
synchronized (lock) {  
    while (!condition) { // while, не if!  
        lock.wait();  
    }  
    // здесь условие точно выполнено  
}
```

67. Отличие wait() с параметром от wait() без параметра

wait() без параметра (или wait(0)) — поток ждёт бесконечно до тех пор, пока не будет явно уведомлён через notify() или notifyAll(). Поток может выйти из ожидания только при уведомлении или при прерывании (InterruptedException). Переходит в состояние WAITING.

wait(long millis) — поток ждёт не более указанного количества миллисекунд. Если за это время не пришло уведомление, поток самостоятельно пробуждается и пытается снова получить монитор. Переходит в состояние TIMED_WAITING. Существует также wait(long millis, int nanos) для более точного указания времени ожидания.

Таким образом, wait(millis) — это ожидание с таймаутом. После его истечения поток не знает, был ли он уведомлён или проснулся по таймауту, поэтому нужно снова проверить условие через while (!condition).

68. Отличия Thread.sleep() от Thread.yield()

Thread.sleep(long millis) — статический метод, который переводит текущий поток в состояние TIMED_WAITING на указанное время. Поток не выполняется и не конкурирует за CPU в течение этого времени. Монитор (lock), если он был захвачен, не освобождается — это принципиальное

отличие от `wait()`. По истечении времени поток переходит обратно в состояние `RUNNABLE` и ожидает, когда планировщик снова выделит ему CPU. Может быть прерван через `Thread.interrupt()`, выбросив `InterruptedException`.

`Thread.yield()` — статический метод, который даёт подсказку планировщику ОС, что текущий поток готов уступить процессорное время. После вызова `yield()` поток переходит из `RUNNING` обратно в `RUNNABLE`, и планировщик может переключиться на другой поток той же или более высокой приоритетности. Однако это лишь рекомендация — планировщик может проигнорировать её и оставить текущий поток выполняться. `yield()` не вызывает `InterruptedException`, не принимает параметров. На практике используется крайне редко — в основном для настройки производительности в специфических сценариях.

Ключевое отличие: `sleep()` гарантированно приостанавливает поток на указанное время, `yield()` лишь предлагает уступить, без гарантий.

69. Отличия `sleep(long millis)` от `wait(long millis)`

Несмотря на то что оба метода приостанавливают поток на указанное время, они принципиально различаются.

`Thread.sleep(long millis)`: статический метод класса `Thread`. Не требует монитора — может вызываться в любом месте. Не освобождает монитор объекта, если поток держит блокировку. Поток переходит в состояние `TIMED_WAITING`. После истечения времени автоматически просыпается. Всегда выбрасывает `InterruptedException` при прерывании.

`Object.wait(long millis)`: метод экземпляра класса `Object`. Должен вызываться только в `synchronized`-блоке, иначе `IllegalMonitorStateException`. Освобождает монитор объекта на время ожидания, позволяя другим потокам войти в синхронизированный блок. Поток переходит в состояние `TIMED_WAITING`. Может быть разбужен досрочно через `notify()/notifyAll()`. Также выбрасывает

InterruptedException.

Итого: главное отличие — освобождение монитора. `wait()` является механизмом межпоточной координации и работает с монитором. `sleep()` — просто пауза в выполнении без влияния на блокировки.

70. Как работает метод `join()`

`Thread.join()` — это метод, который заставляет текущий поток ждать, пока другой поток (тот, на котором вызван `join()`) завершит выполнение. После завершения целевого потока вызывающий поток продолжает работу.

Java

```
Thread t = new Thread() -> {  
    // долгая работа  
    computeResult();  
});  
t.start();  
t.join(); // текущий поток ждёт завершения t  
// здесь t гарантированно завершился
```

Существует версия с таймаутом `join(long millis)` — текущий поток ждёт не более указанного времени. После этого продолжает выполнение независимо от того, завершился ли другой поток. Важно проверять `t.isAlive()` после такого `join()`, чтобы понять, завершился ли поток или нет.

Метод `join()` может выбросить `InterruptedException`, если поток ожидания был прерван. Внутренне `join()` реализован через `wait()` на объекте потока.

Практическое применение: запустить несколько задач параллельно и дождаться их всех перед продолжением. В современном коде для этого чаще используют `CompletableFuture.allOf()` или `ExecutorService.invokeAll()`.

71. Синхронизация. Ключевое слово `synchronized`. Способы

синхронизации

Синхронизация — это механизм управления доступом нескольких потоков к общим ресурсам (данным, объектам) для предотвращения состояния гонки (race condition). Без синхронизации несколько потоков, одновременно читающих и записывающих общие данные, могут получить некорректные результаты из-за непредсказуемого порядка операций.

Ключевое слово `synchronized` работает через механизм монитора (mutex). Каждый объект в Java имеет связанный с ним монитор. `synchronized` гарантирует, что в любой момент времени не более одного потока выполняет синхронизированный код для данного монитора.

Формы использования `synchronized`:

Синхронизированный метод экземпляра — монитором является `this`:

Java

```
public synchronized void increment() { count++; }
```

Синхронизированный статический метод — монитором является Class-объект класса:

Java

```
public static synchronized void staticIncrement() { staticCount++; }
```

Синхронизированный блок — можно явно указать объект-монитор, что даёт более тонкий контроль:

Java

```
synchronized (lockObject) { count++; }
```

Другие способы синхронизации:

`volatile` — гарантирует видимость изменений переменной для всех потоков, но не атомарность составных операций.

`java.util.concurrent.locks.Lock (ReentrantLock)` — более гибкая альтернатива `synchronized` с таймаутом ожидания, прерываемой блокировкой, поддержкой `Condition`.

`ReentrantReadWriteLock` — позволяет нескольким потокам одновременно читать, но только одному писать.

`StampedLock (Java 8+)` — оптимистичная блокировка для сценариев, где чтений намного больше, чем записей.

Классы из `java.util.concurrent.atomic` — атомарные операции без блокировок.

`Semaphore`, `CountDownLatch`, `CyclicBarrier` — высокоуровневые синхронизаторы.

Конкурентные коллекции — `ConcurrentHashMap`, `CopyOnWriteArrayList` и другие.

72. По каким объектам синхронизируются статические и нестатические методы

Нестатический `synchronized`-метод синхронизируется по `this` — то есть по конкретному экземпляру объекта, на котором вызывается метод. Два разных объекта одного класса имеют разные мониторы, поэтому их синхронизированные методы не блокируют друг друга. Два потока, работающие с одним объектом, будут конкурировать за его монитор.

Статический `synchronized`-метод синхронизируется по `ClassName.class` — по объекту `Class`, представляющему данный класс в JVM. Этот объект один для всего класса, независимо от количества экземпляров. Поэтому два потока, вызывающих статический синхронизированный метод из любых мест, будут

конкурировать за один монитор.

Важное следствие: нестатический и статический синхронизированные методы одного класса синхронизируются по разным объектам (один по `this`, другой по `Class`). Это значит, что они могут выполняться одновременно и не блокируют друг друга — это нужно учитывать при проектировании потокобезопасных классов.

73. Отличия `synchronized` от `Lock`

`synchronized` — встроенный языковой механизм. `Lock` (`java.util.concurrent.locks.Lock`, основная реализация `ReentrantLock`) — более мощный и гибкий API из пакета `java.util.concurrent`.

Структура: `synchronized` — ключевое слово, структура языка (блок или метод). `Lock` — объект, блокировка вручную через `lock.lock()` и обязательный `lock.unlock()` (обычно в `finally`).

Попытка захвата с таймаутом: `synchronized` блокирует поток навсегда до получения монитора. `ReentrantLock` имеет метод `tryLock()` (немедленный возврат `false`, если занято) и `tryLock(time, unit)` (ожидание с таймаутом) — поток может отступить, если не смог получить блокировку.

Прерываемое ожидание: `synchronized` не реагирует на прерывание во время ожидания монитора (поток в состоянии `BLOCKED` нельзя прервать через `interrupt()`). `Lock.lockInterruptibly()` позволяет прервать ожидающий поток через `interrupt()`.

Справедливость (fairness): `synchronized` не гарантирует порядок предоставления монитора потокам. `ReentrantLock(true)` создаёт справедливую блокировку — потоки получают её в порядке очереди (FIFO). Справедливая блокировка медленнее.

Условия (Condition): `synchronized` имеет один `wait()/notify()`. `Lock`

предоставляет `lock.newCondition()` — можно создать несколько `Condition` для одного `Lock`, что позволяет более точно управлять уведомлениями.

`ReadWriteLock`: для `synchronized` нет встроенного разделения `read/write`. `ReentrantReadWriteLock` позволяет нескольким читателям одновременно, что улучшает пропускную способность в `read-heavy` сценариях.

74. Есть ли у `Lock` аналог `wait/notify`?

Да, аналогом является `Condition` интерфейс из `java.util.concurrent.locks`. Объект `Condition` создаётся через `lock.newCondition()` и привязывается к конкретному `Lock`.

Методы `Condition`:

`await()` — аналог `Object.wait()`. Освобождает блокировку и ждёт сигнала.

`await(long time, TimeUnit unit)` — ожидание с таймаутом.

`signal()` — аналог `Object.notify()`. Пробуждает один ожидающий поток.

`signalAll()` — аналог `Object.notifyAll()`.

Преимущество перед `wait/notify`: для одного `Lock` можно создать несколько `Condition`. Например, для буферизованной очереди: один `Condition notFull` (ждут производители, когда очередь полна) и один `Condition notEmpty` (ждут потребители, когда очередь пуста). Это позволяет уведомлять только нужных ожидающих, а не всех сразу.

Java

```
ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();
// производитель:
lock.lock();
try {
    while (isFull()) notFull.await();
```

```
    add(item);
    notEmpty.signal();
} finally { lock.unlock(); }
```

75. Deadlock, livelock, race condition, starvation. Реализация deadlock

Race condition (состояние гонки) — ситуация, когда результат программы зависит от непредсказуемого порядка выполнения операций в нескольких потоках. Классический пример: `i++` в нескольких потоках без синхронизации — операция `i++` не атомарна (это три операции: прочитать, увеличить, записать), и потоки могут перезаписать изменения друг друга.

Deadlock (взаимная блокировка) — ситуация, когда два или более потока бесконечно ждут друг друга: поток А держит ресурс 1 и ждёт ресурс 2, поток В держит ресурс 2 и ждёт ресурс 1. Никто не может продолжить. Программа зависает.

Java

// Реализация deadlock:

```
Object lock1 = new Object();
```

```
Object lock2 = new Object();
```

```
Thread t1 = new Thread(() -> {
    synchronized (lock1) {
        System.out.println("T1 захватил lock1");
        try { Thread.sleep(100); } catch (InterruptedException e) {}
        synchronized (lock2) { System.out.println("T1 захватил lock2"); }
    }
});
```

```
Thread t2 = new Thread(() -> {
    synchronized (lock2) {
        System.out.println("T2 захватил lock2");
    }
});
```

```
try { Thread.sleep(100); } catch (InterruptedException e) {}  
synchronized (lock1) { System.out.println("T2 захватил lock1"); }  
}  
});  
t1.start(); t2.start(); // оба потока зависнут
```

Для предотвращения deadlock: всегда захватывать блокировки в одном и том же порядке; использовать tryLock() с таймаутом; уменьшать количество одновременно удерживаемых блокировок.

Livelock — потоки не заблокированы, они активно выполняются, но не могут продвинуться вперёд, потому что постоянно реагируют на действия друг друга. Аналогия: два человека, встретившиеся в узком коридоре, оба вежливо отступают вправо, затем снова вправо, и так бесконечно. Решение: добавить случайную задержку или приоритизацию.

Starvation (голодание) — поток бесконечно ждёт ресурс, который постоянно захватывается другими потоками с более высоким приоритетом или более активными. Поток "голодает" — никогда не получает процессорное время. Решение: справедливые блокировки (ReentrantLock(true)), правильная расстановка приоритетов.

76. Semaphore, CyclicBarrier, CountdownLatch: отличия от Lock

Все три класса находятся в java.util.concurrent и являются синхронизаторами — инструментами координации потоков. В отличие от Lock, который обеспечивает взаимное исключение (только один поток в секции), эти классы реализуют более высокоуровневые паттерны координации.

Semaphore (семафор) — контролирует количество потоков, одновременно имеющих доступ к ресурсу. Инициализируется с числом "разрешений" (permits). acquire() уменьшает счётчик (ждёт, если 0), release() увеличивает. Если Semaphore(1) — это аналог мьютекса (как Lock). Применяется для ограничения параллелизма: например, не более 10 одновременных подключений к БД, пул ресурсов. Отличие от Lock: может быть освобождён

другим потоком (а не тем, что захватил).

`CountDownLatch` — позволяет одному или нескольким потокам ждать, пока другие не выполнят определённые действия. Инициализируется с счётчиком. `countDown()` уменьшает счётчик, `await()` блокирует, пока счётчик не достигнет 0. Одноразовый — счётчик нельзя сбросить. Применяется: главный поток запускает N рабочих потоков и ждёт их завершения; или несколько потоков ждут готовности стартового сигнала.

`CyclicBarrier` — позволяет нескольким потокам ждать друг друга в "точке встречи" (барьере). Когда нужное количество потоков вызвали `await()` — все они разблокируются одновременно. Многоразовый — после прохождения барьера он автоматически сбрасывается для следующего использования. Опционально можно указать задачу `Runnable`, которая выполняется после прохождения барьера. Применяется для фазовых вычислений, где все потоки должны завершить одну фазу, прежде чем начинать следующую.

Отличие от `Lock`: `Lock` — для взаимного исключения. Эти синхронизаторы — для координации и оркестровки работы нескольких потоков.

77. `volatile`. Отличие от `AtomicInteger`. Классы `java.util.concurrent.atomic`

`volatile` — ключевое слово, гарантирующее видимость изменений переменной для всех потоков. Без `volatile` каждый поток может кэшировать значение переменной в своём регистре или кеше процессора. Изменение в одном потоке может быть невидимо другому потоку неопределённое время. `volatile` гарантирует, что при записи значение немедленно записывается в основную память, а при чтении — всегда берётся из основной памяти. Также `volatile` устанавливает барьер памяти (`memory barrier`), предотвращая переупорядочивание инструкций.

Ограничение `volatile`: оно гарантирует видимость, но не атомарность составных операций. Операция `volatileVar++` состоит из трёх шагов:

прочитать, увеличить, записать. Два потока могут одновременно прочитать одно и то же значение, увеличить и записать — одно из увеличений потеряется. `volatile` подходит для флагов (простые чтение/запись одного значения), но не для счётчиков или других составных операций.

`AtomicInteger` (и другие классы из `java.util.concurrent.atomic`) — решают именно эту проблему. Они обеспечивают атомарность операций без использования блокировок, через инструкции процессора CAS (Compare-And-Swap). `AtomicInteger.incrementAndGet()` — это атомарная операция "увеличить и вернуть" без блокировок. Это и быстрее `synchronized`, и безопаснее `volatile` для составных операций.

Другие классы пакета `java.util.concurrent.atomic`:

`AtomicLong` — атомарный `long`. `AtomicBoolean` — атомарный `boolean`. `AtomicReference<V>` — атомарная ссылка на объект. `AtomicIntegerArray`, `AtomicLongArray`, `AtomicReferenceArray` — атомарные массивы. `AtomicIntegerFieldUpdater`, `AtomicLongFieldUpdater` — атомарное обновление полей объектов. `LongAdder`, `DoubleAdder` (Java 8+) — высокопроизводительные счётчики для случаев высокой конкуренции (менее точные, но быстрее, чем `AtomicLong` при многих потоках). `LongAccumulator`, `DoubleAccumulator` — обобщённые аккумуляторы с произвольной функцией. `Java Collections`, `Functional Programming`, `JDBC`, `SQL`, `Design Patterns`

JAVA COLLECTIONS

1. Основные интерфейсы коллекций и их иерархия

Вся система коллекций в Java строится на нескольких ключевых интерфейсах, которые образуют две независимые иерархии — иерархию `Collection` и иерархию `Map`.

Иерархия `Collection`:

На вершине стоит интерфейс `Iterable<T>`, который даёт возможность использовать объект в цикле `for-each`. Он содержит единственный метод

`iterator()`. От него наследует `Collection<E>` — корневой интерфейс для всех коллекций (за исключением `Map`). `Collection` добавляет методы `add()`, `remove()`, `size()`, `contains()`, `iterator()`, `toArray()` и другие базовые операции.

От `Collection` наследуют три основных интерфейса.

`List<E>` — упорядоченная последовательность элементов, допускающая дубликаты. Элементы хранятся в порядке добавления (или в порядке индексации), к каждому элементу можно обратиться по индексу. Основные реализации: `ArrayList`, `LinkedList`, `Vector`, `Stack`, `CopyOnWriteArrayList`. Применять `List` стоит, когда важен порядок элементов и нужен доступ по индексу, или когда допускаются дубликаты.

`Set<E>` — коллекция уникальных элементов (не допускает дубликатов). Не гарантирует порядок (если не использовать специальные реализации). Основные реализации: `HashSet` (на основе хэш-таблицы, $O(1)$ для `add/remove/contains`), `LinkedHashSet` (сохраняет порядок вставки), `TreeSet` (хранит в отсортированном порядке, $O(\log n)$ для операций), `EnumSet` (эффективная реализация для `enum`). Применять `Set` стоит, когда нужна уникальность элементов, или для быстрой проверки наличия элемента.

`Queue<E>` — очередь, обычно FIFO (first-in-first-out). Добавляет методы `offer()`, `poll()`, `peek()`. Подинтерфейс `Deque<E>` (double-ended queue) поддерживает операции с обоих концов и может работать как стек (LIFO). Реализации: `LinkedList`, `ArrayDeque`, `PriorityQueue`, `ArrayBlockingQueue`, `LinkedBlockingQueue`. Применять `Queue/Deque` стоит для задач и очередей, алгоритмов BFS/DFS, буферов между производителями и потребителями.

Когда какую коллекцию использовать: если нужна быстрая проверка уникальности — `HashSet`; если нужен отсортированный набор — `TreeSet`; если нужен быстрый доступ по индексу и преимущественно чтение — `ArrayList`; если нужны частые вставки/удаления в середину — `LinkedList`; если нужна очередь с приоритетом — `PriorityQueue`.

Если часто добавлять и удалять элементы: зависит от того, в какое место. Если в конец — ArrayList быстр (амортизированно $O(1)$). Если в середину или начало, или случайное место — LinkedList ($O(1)$ для вставки если есть итератор, но $O(n)$ для нахождения позиции). Если из произвольного места и порядок не важен — HashSet или LinkedList. На практике ArrayList обычно быстрее LinkedList даже для вставок из-за лучшей локальности кеша процессора.

2. Что такое интерфейс Map? Представляет ли он коллекцию?

Map<K,V> — интерфейс, представляющий структуру данных "ключ-значение" (словарь). Каждый ключ уникален, каждому ключу соответствует ровно одно значение. Map предоставляет методы put(key, value), get(key), remove(key), containsKey(key), containsValue(value), keySet(), values(), entrySet().

Представляет ли Map коллекцию? Формально — нет. Map не наследует от интерфейса Collection и не является его частью. Это отдельная иерархия. Однако Map тесно связан с коллекциями: map.keySet() возвращает Set<K>, map.values() возвращает Collection<V>, map.entrySet() возвращает Set<Map.Entry<K,V>>. Поэтому Map часто рассматривается как часть Java Collections Framework, хотя технически находится за пределами иерархии Collection.

Основные реализации: HashMap (хэш-таблица, $O(1)$ для операций, не гарантирует порядок), LinkedHashMap (сохраняет порядок вставки или порядок обращения), TreeMap (красно-чёрное дерево, $O(\log n)$, ключи в естественном или заданном порядке), Hashtable (устаревший потокобезопасный вариант), ConcurrentHashMap (современный потокобезопасный), EnumMap (для enum-ключей), WeakHashMap (ключи — слабые ссылки, GC может их удалить).

3. Сравнение ArrayList и LinkedList

Обе реализации интерфейса List, но внутренняя структура принципиально отличается.

ArrayList внутри использует динамический массив (Object[]). При создании без параметров создаётся массив ёмкостью 10. При заполнении выполняется расширение: создаётся новый массив ёмкостью $\text{oldCapacity} * 1.5$ и данные копируются.

Характеристики ArrayList: доступ по индексу — $O(1)$ (прямое обращение к массиву); добавление в конец — амортизированно $O(1)$ (иногда $O(n)$ при расширении); добавление/удаление в середину — $O(n)$ (нужно сдвигать элементы); поиск — $O(n)$; итерация — очень быстрая (хорошая локальность кеша, элементы расположены в памяти последовательно); потребление памяти — компактное (только массив ссылок).

LinkedList внутри является двусвязным списком. Каждый элемент хранится в узле (Node), содержащем ссылки на предыдущий и следующий узел плюс само значение. LinkedList реализует также Deque.

Характеристики LinkedList: доступ по индексу — $O(n)$ (нужно пройти от начала или конца до нужного индекса); добавление/удаление в начало или конец — $O(1)$; добавление/удаление по позиции итератора — $O(1)$ (но нахождение позиции — $O(n)$); итерация — медленнее, чем ArrayList (плохая локальность кеша, узлы разбросаны по памяти); потребление памяти — больше (каждый узел хранит два дополнительных указателя).

На практике ArrayList почти всегда предпочтительнее. Локальность кеша процессора (cache locality) даёт ArrayList огромное преимущество в реальных сценариях. LinkedList имеет смысл только если постоянно добавляете/удаляете элементы в начало или нужны операции двунаправленной очереди.

4. Сравнение HashMap и Hashtable

HashMap — введён в Java 2 как часть Collections Framework. Не синхронизирован, не потокобезопасен. Допускает один null-ключ и множество null-значений. Высокая производительность в однопоточном окружении. Не гарантирует порядок элементов.

Hashtable — один из оригинальных классов Java 1.0, является устаревшим (legacy) классом. Все методы синхронизированы (synchronized), что делает его потокобезопасным, но медленным — синхронизируется весь объект целиком. Не допускает null-ключей и null-значений (выбрасывает NullPointerException). Наследует от Dictionary (устаревший класс).

Ключевые отличия: HashMap быстрее в однопоточном коде; Hashtable потокобезопасен но медленен; HashMap допускает null-ключ, Hashtable — нет; HashMap — итерируется через Iterator, Hashtable — через Enumeration (устаревший интерфейс) и Iterator.

Для потокобезопасной работы в современном коде следует использовать ConcurrentHashMap вместо Hashtable — он обеспечивает лучшую производительность за счёт сегментированной (а начиная с Java 8 — ещё более тонкой) блокировки.

5. Особенности Set. Сравнение HashSet и TreeSet. Как добавляются объекты

Интерфейс Set гарантирует уникальность элементов — добавить один и тот же элемент дважды невозможно. При попытке добавить дубликат метод add() возвращает false и коллекция не изменяется. Уникальность определяется через hashCode() и equals().

HashSet внутри реализован на основе HashMap — элементы Set хранятся как ключи в HashMap, а значением является константный объект-заглушка (PRESENT). Не гарантирует порядок итерации. Допускает null. Операции add, remove, contains работают за $O(1)$ в среднем. Для работы элементов в HashSet крайне важно правильно переопределить hashCode() и equals() в их

классах.

Как добавляется объект в `HashSet`: вычисляется `hashCode()` объекта, определяется корзина (`bucket`), в корзине проверяется наличие объекта с таким же `hashCode()`, и если найден — вызывается `equals()` для проверки. Если `equals()` возвращает `true` — объект считается дубликатом, не добавляется. Если `false` или корзина пуста — объект добавляется.

`TreeSet` внутри реализован на основе `TreeMap` (красно-чёрное дерево). Хранит элементы в отсортированном порядке — либо по естественному порядку (элементы должны реализовывать `Comparable`), либо по переданному `Comparator`. Не допускает `null` (нельзя сравнить `null` с другим элементом). Операции работают за $O(\log n)$. Дополнительные методы: `first()`, `last()`, `headSet()`, `tailSet()`, `subSet()`, `floor()`, `ceiling()`, `higher()`, `lower()`.

Как добавляется объект в `TreeSet`: вызывается метод `compareTo()` (или `compare()` `Comparator'a`) для сравнения с существующими элементами и нахождения правильной позиции в красно-чёрном дереве. Если `compareTo()` возвращает 0 — объект считается дубликатом. Поэтому в `TreeSet` не нужен `hashCode()/equals()` — уникальность определяется через `compareTo()/compare()`.

`LinkedHashSet` — промежуточный вариант: хэш-таблица + двусвязный список. Производительность близка к `HashSet` ($O(1)$), но сохраняет порядок вставки.

6. Принцип работы `HashMap`. Изменения в Java 8

Внутренняя структура `HashMap` до Java 8: массив корзин (`buckets`) `Node[] table`. Каждая корзина содержит связный список из `Node` объектов. `Node` хранит: `hash` (хэш ключа), `key`, `value`, `next` (ссылка на следующий узел в цепочке).

Операция `put(key, value)`: вычисляется `hash = key.hashCode()`, затем

применяется дополнительное перемешивание ($h \wedge (h \ggg 16)$) для лучшего распределения. Индекс корзины: $index = hash \& (capacity - 1)$. Если корзина пуста — создаётся новый Node. Если не пуста — перебираются элементы цепочки: если найден ключ с таким же hash и equals() вернул true — значение заменяется. Если не найден — Node добавляется в начало (до Java 8) или конец цепочки.

Параметры HashMap: начальная ёмкость (default 16) и коэффициент загрузки loadFactor (default 0.75). Когда количество элементов превысит $capacity * loadFactor$, происходит resize() — размер таблицы удваивается, и все элементы перехэшируются.

Изменения в Java 8: главное нововведение — при достижении длины цепочки в одной корзине 8 элементов (TREEIFY_THRESHOLD = 8) и при размере таблицы не менее 64, связный список в этой корзине преобразуется в красно-чёрное дерево. Это улучшает производительность в худшем случае с $O(n)$ до $O(\log n)$ для операций в одной корзине (актуально при плохой хэш-функции или намеренных коллизиях). Когда дерево уменьшается до 6 элементов (UNTREEIFY_THRESHOLD = 6) — оно преобразуется обратно в список. Также в Java 8 элементы при resize перекладываются в новый массив с использованием битов хэша более эффективно. И итерация по entrySet() стала несколько быстрее.

7. Отличие ArrayList от Vector

Vector — один из оригинальных классов Java 1.0, является аналогом ArrayList, но с синхронизацией. Все методы Vector объявлены synchronized. Это делает его потокобезопасным, но значительно медленнее в однопоточном коде из-за накладных расходов на синхронизацию каждого вызова. При расширении Vector по умолчанию удваивает ёмкость ($capacity * 2$), тогда как ArrayList увеличивает на 50% ($capacity * 1.5$). Vector также имеет устаревшие методы (addElement, elementAt, removeElement) из интерфейса Enumeration.

ArrayList — введён в Java 2 в составе Collections Framework. Не

синхронизирован, не потокобезопасен. Значительно быстрее Vector в однопоточной среде. Использует стандартный интерфейс Iterator и ListIterator. Является рекомендуемым выбором в современном коде.

Оба класса реализуют List, RandomAccess, Cloneable, Serializable. Для потокобезопасного динамического массива в современном коде следует использовать CopyOnWriteArrayList или обернуть ArrayList через Collections.synchronizedList().

8. Коллекции из пакета concurrent

Пакет java.util.concurrent содержит высокопроизводительные потокобезопасные реализации коллекций.

ConcurrentHashMap — потокобезопасная замена Hashtable и синхронизированного HashMap. До Java 8 использовала сегментированную блокировку (16 сегментов, каждый — независимый замок). С Java 8 перешла на более тонкую блокировку: блокируется только отдельная корзина (bucket) при записи через synchronized-блок, чтение вообще не блокируется (через volatile). Это даёт высокую пропускную способность. Не допускает null-ключей и null-значений. Поддерживает атомарные операции: putIfAbsent(), computeIfAbsent(), compute(), merge().

CopyOnWriteArrayList — потокобезопасная реализация List. При каждой модификации (add, set, remove) создаётся полная копия внутреннего массива. Чтение (итерация) абсолютно безопасно без блокировок — читается снимок массива на момент начала итерации. Это делает CopyOnWriteArrayList идеальным для сценариев, где чтений намного больше, чем записей. Запись дорогостоящая — $O(n)$. Итераторы не поддерживают remove().

CopyOnWriteArraySet — аналогично, на базе CopyOnWriteArrayList.

ConcurrentLinkedQueue и ConcurrentLinkedDeque — неблокирующие очереди на основе алгоритма CAS. Очень высокая производительность в

многопоточной среде без блокировок.

Блокирующие очереди — реализуют интерфейс `BlockingQueue`: `ArrayBlockingQueue` (ограниченная, на массиве), `LinkedBlockingQueue` (может быть ограниченной или нет, на связном списке), `PriorityBlockingQueue` (с приоритетом), `SynchronousQueue` (каждая операция вставки ожидает соответствующей операции чтения), `DelayQueue` (элементы доступны только по истечении задержки). Применяются в паттерне "производитель-потребитель".

`ConcurrentSkipListMap` и `ConcurrentSkipListSet` — потокобезопасные аналоги `TreeMap` и `TreeSet` на основе skip-list (структура данных с несколькими уровнями связей). $O(\log n)$ для операций, поддерживают сортировку, полностью неблокирующие.

9. Что происходит при добавлении элемента в `ArrayList`

При создании `new ArrayList<>()` без аргументов создаётся пустой массив-заглушка `DEFAULTCAPACITY_EMPTY_ELEMENTDATA`. При первом добавлении элемента создаётся массив ёмкостью 10 (`DEFAULT_CAPACITY`).

При каждом вызове `add(element)` выполняется следующее: проверяется, есть ли место в массиве (`size < elementData.length`). Если место есть — элемент просто записывается в `elementData[size]` и `size` увеличивается на 1. Если места нет — выполняется расширение `grow()`: вычисляется новая ёмкость `newCapacity = oldCapacity + (oldCapacity >> 1)` — то есть старая ёмкость плюс её половина (умножение на 1.5). Затем вызывается `Arrays.copyOf(elementData, newCapacity)` — создаётся новый массив нужной ёмкости и данные копируются. После этого добавляется новый элемент.

Операция `add(int index, E element)` для вставки в середину: сначала вызывается `ensureCapacity()` при необходимости, затем `System.arraycopy()` для сдвига элементов вправо, начиная с `index`, затем записывается новый элемент

в позицию `index`. Это операция $O(n)$ из-за сдвига.

`removeAll()` и `remove(index)` аналогично используют `System.arraycopy()` для сдвига элементов влево, заполняя образовавшуюся дыру. Массив при удалении не уменьшается автоматически — для уменьшения нужно вызвать `trimToSize()`.

10. Преобразование потоко-небезопасной коллекции в потокобезопасную

Способ 1 — `Collections.synchronizedXxx()`: утилитный класс `Collections` предоставляет статические методы-обёртки, которые возвращают потокобезопасную обёртку вокруг переданной коллекции. Все методы обёртки синхронизированы через `synchronized(mutex)`. Примеры: `Collections.synchronizedList(list)`, `Collections.synchronizedMap(map)`, `Collections.synchronizedSet(set)`. Важно: при итерации по такой коллекции нужно вручную синхронизироваться:

Java

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
synchronized (syncList) {
    Iterator<String> it = syncList.iterator();
    while (it.hasNext()) { ... }
}
```

Это простое решение, но производительность ниже, чем у специализированных `concurrent`-коллекций, из-за блокировки всего объекта.

Способ 2 — использовать `ConcurrentHashMap`, `CopyOnWriteArrayList` и другие коллекции из `java.util.concurrent` — предпочтительный подход в современном коде. Они предоставляют лучшую пропускную способность за счёт более тонкой блокировки или `lock-free` алгоритмов.

Способ 3 — `Collections.unmodifiableXxx()`: создаёт неизменяемую обёртку.

Если коллекция не изменяется — она потокобезопасна для чтения.

FUNCTIONAL PROGRAMMING & STREAM API

11. Функциональные интерфейсы: определение, default и static методы, применение

Функциональный интерфейс — это интерфейс, содержащий ровно один абстрактный метод (SAM — Single Abstract Method). Аннотация `@FunctionalInterface` является опциональной, но рекомендуемой — она заставляет компилятор проверять, что интерфейс действительно функциональный, и выдаёт ошибку если абстрактных методов больше одного.

Функциональные интерфейсы являются целевыми типами для лямбда-выражений и ссылок на методы. Компилятор использует тип функционального интерфейса для вывода типа лямбды (type inference).

Функциональный интерфейс может содержать любое количество default и static методов — они не нарушают статус функционального интерфейса, потому что не являются абстрактными. default методы позволяют добавлять поведение к интерфейсу без нарушения совместимости. Например, `Function<T,R>` имеет default методы `andThen()` и `compose()` для компоновки функций, а `Predicate<T>` имеет `and()`, `or()`, `negate()` для логических операций над предикатами.

Все методы, унаследованные от `Object` (`equals`, `hashCode`, `toString`) тоже не нарушают правило SAM — они не считаются абстрактными методами интерфейса для данного правила.

Применение функциональных интерфейсов: они являются основой функционального стиля программирования в Java, используются с лямбдами и Stream API, в паттернах Strategy, Command, Observer, при создании колбэков и обработчиков событий.

12. Лямбда-выражение. Замыкания. Синтаксис. Характеристики

Лямбда-выражение — это краткая форма записи анонимной функции, которая реализует единственный абстрактный метод функционального интерфейса. Лямбда не является объектом в классическом смысле — она является экземпляром функционального интерфейса, создаваемым более эффективным способом, чем анонимный класс (через `invokedynamic` и `LambdaMetafactory`).

Синтаксис: (параметры) -> выражение или (параметры) -> { операторы; }. Если параметр один — скобки вокруг него опциональны: `x -> x * x`. Если параметров нет — пустые скобки обязательны: `() -> "hello"`. Тип параметров обычно выводится компилятором (type inference), но можно указать явно: `(String s) -> s.toUpperCase()`. Если тело — одно выражение, оно является возвращаемым значением, `return` не нужен. Если тело — блок из нескольких операторов, `return` обязателен (если метод не `void`).

Замыкания (closures): лямбда может обращаться к переменным из окружающего контекста (из метода, в котором объявлена). Однако эти переменные должны быть *effectively final* (фактически финальными) — они не должны изменяться после инициализации (не обязательно явно пометить `final`, но изменять нельзя). Это связано с тем, что лямбда может жить дольше метода, в котором создана (например, если она передаётся куда-то ещё). Поля экземпляра (`this.field`) и статические поля не подпадают под это ограничение — лямбда захватывает ссылку `this`.

Характеристики: лямбды не создают новый `scope` для `this` (в отличие от анонимных классов, где `this` указывает на анонимный класс, в лямбде `this` указывает на объемлющий объект). Лямбды не могут объявлять свою `checked`-исключение, если функциональный интерфейс не объявляет её в сигнатуре.

13. Function, Supplier, Predicate, Consumer: применение

Все эти интерфейсы находятся в пакете `java.util.function`.

`Function<T, R>` — принимает объект типа `T`, возвращает объект типа `R`. Метод `R apply(T t)`. Используется для преобразований: `Function<String, Integer> length = String::length;`. Методы `andThen(Function after)` и `compose(Function before)` позволяют составлять цепочки. `UnaryOperator<T>` — специализация `Function<T,T>` (тип входа и выхода совпадают). `BiFunction<T,U,R>` — принимает два аргумента.

`Supplier<T>` — не принимает аргументов, возвращает объект типа `T`. Метод `T get()`. Используется для ленивой инициализации, фабрик, генерации значений: `Supplier<List<String>> listFactory = ArrayList::new;`. В `Optional.orElseGet(Supplier)` — альтернативное значение вычисляется только при необходимости.

`Predicate<T>` — принимает объект типа `T`, возвращает `boolean`. Метод `boolean test(T t)`. Используется для фильтрации, проверки условий. В `Stream API` метод `filter()` принимает `Predicate`. Методы `and()`, `or()`, `negate()` позволяют комбинировать предикаты: `Predicate<String> notEmpty = Predicate.not(String::isEmpty)`. `BiPredicate<T,U>` — принимает два аргумента.

`Consumer<T>` — принимает объект типа `T`, ничего не возвращает (`void`). Метод `void accept(T t)`. Используется для операций с побочными эффектами: вывод, логирование, сохранение. В `Stream API` метод `forEach()` принимает `Consumer`. Метод `andThen()` позволяет связывать консьюмеры. `BiConsumer<T,U>` — принимает два аргумента.

Также важны: `IntFunction`, `LongFunction`, `DoubleFunction`, `ToIntFunction`, `IntUnaryOperator`, `IntBinaryOperator` и другие примитивные специализации, позволяющие избежать автобоксинга.

14. Ссылка на метод: что это такое?

Ссылка на метод (`method reference`) — это компактный синтаксис для создания лямбда-выражения, которое просто вызывает уже существующий метод. Используется оператор `::`. По сути, ссылка на метод — это сахарный

синтаксис для конкретного вида лямбд, где лямбда только перенаправляет вызов к методу.

Ссылка на метод — это не ссылка на объект в смысле Java-ссылок на объекты в памяти. Ссылка на метод является экземпляром функционального интерфейса, как и лямбда. Однако в случае ссылки на метод экземпляра конкретного объекта (`instance::method`) объект действительно "захватывается" в замыкании.

Существуют четыре вида ссылок на методы:

Ссылка на статический метод: `ClassName::staticMethod`. Эквивалентна лямбде `(args) -> ClassName.staticMethod(args)`. Пример: `Function<String, Integer> parse = Integer::parseInt;`

Ссылка на метод экземпляра конкретного объекта: `instance::instanceMethod`. Эквивалентна `(args) -> instance.instanceMethod(args)`. Конкретный объект захватывается. Пример: `Supplier<String> supplier = someString::toUpperCase;`

Ссылка на метод экземпляра произвольного объекта определённого типа: `ClassName::instanceMethod`. Первый параметр лямбды становится объектом, на котором вызывается метод: `(obj, args) -> obj.instanceMethod(args)`. Пример: `Function<String, String> upper = String::toUpperCase;` (эквивалентно `s -> s.toUpperCase()`)

Ссылка на конструктор: `ClassName::new`. Эквивалентна `(args) -> new ClassName(args)`. Пример: `Supplier<List<String>> factory = ArrayList::new;`

15. Чем Stream отличается от коллекции?

Коллекция и Stream — принципиально разные концепции, хотя Stream обычно создаётся на основе коллекции.

Коллекция — это структура данных, хранящая элементы в памяти. Все

элементы присутствуют одновременно. Коллекция — это данные. Коллекцию можно итерировать многократно. Элементы в коллекции можно добавлять, удалять, изменять.

Stream — это конвейер операций над данными, а не структура хранения. Stream не хранит данные — он описывает, какие операции нужно применить к данным. Несколько ключевых отличий:

Stream одноразовый — после выполнения терминальной операции Stream не может быть использован повторно. Попытка использовать "потреблённый" stream вызовет `IllegalStateException`.

Stream ленивый — промежуточные операции (`filter`, `map`, `sorted`) не выполняются немедленно. Они лишь строят конвейер. Вычисления начинаются только при вызове терминальной операции.

Stream не модифицирует источник — операции над Stream не изменяют исходную коллекцию, а создают новый Stream с преобразованными данными.

Stream может быть бесконечным — `Stream.iterate(0, x -> x + 1)` создаёт бесконечный поток, что невозможно для коллекции.

Stream поддерживает параллельное выполнение через `parallelStream()` или `stream().parallel()` — операции автоматически распараллеливаются на пул потоков `ForkJoinPool`.

16. Промежуточные и терминальные операции. Lazy evaluation

Промежуточные операции (intermediate operations) — операции, которые возвращают новый Stream и позволяют строить конвейер. Они ленивые — не выполняются, пока не будет вызвана терминальная операция. Основные промежуточные операции: `filter(Predicate)` — фильтрация элементов; `map(Function)` — преобразование каждого элемента; `flatMap(Function)` — "развёртывание" вложенных потоков; `distinct()` — уникальные элементы;

`sorted()` / `sorted(Comparator)` — сортировка; `limit(n)` — ограничение количества; `skip(n)` — пропуск первых `n` элементов; `peek(Consumer)` — отладочный просмотр без изменения; `mapToInt`, `mapToLong`, `mapToDouble` — преобразование в примитивные потоки; `takeWhile(Predicate)`, `dropWhile(Predicate)` (Java 9+).

Терминальные операции (terminal operations) — операции, которые запускают выполнение всего конвейера и возвращают результат (не `Stream`). После вызова терминальной операции `Stream` считается потреблённым. Основные: `forEach(Consumer)` — выполнить действие для каждого элемента; `collect(Collector)` — собрать в коллекцию или другой результат; `count()` — подсчёт элементов; `reduce(BinaryOperator)` — свёртка; `findFirst()`, `findAny()` — найти элемент (возвращают `Optional`); `anyMatch()`, `allMatch()`, `noneMatch()` — проверка условия; `min()`, `max()` — минимум/максимум; `toArray()` — в массив; `sum()`, `average()` (для примитивных потоков).

Lazy evaluation — ленивые вычисления означают, что JVM оптимизирует выполнение конвейера. Например, для `stream.filter(x -> x > 5).findFirst()` JVM прекратит обработку, как только найдёт первый подходящий элемент — не нужно обрабатывать весь поток. Для `limit(n)` обрабатываются только первые `n` элементов. Это делает `Stream API` эффективным инструментом даже для больших или бесконечных источников данных.

17. `map()` vs `flatMap()`

`map(Function<T, R> mapper)` — применяет функцию к каждому элементу потока и возвращает поток из результатов. Отображение "один к одному": каждый элемент входного потока превращается ровно в один элемент выходного потока. Например, `stream.map(String::length)` превратит поток строк в поток их длин.

`flatMap(Function<T, Stream<R>> mapper)` — применяет функцию к каждому элементу, функция возвращает `Stream<R>`, и все эти потоки сплющиваются (объединяются) в один общий поток. Отображение "один ко многим": каждый

элемент входного потока может превратиться в ноль, один или несколько элементов выходного потока.

Наглядный пример: есть список строк, нужно получить все слова:

Java

```
List<String> sentences = List.of("hello world", "foo bar baz");
```

```
// map даёт поток из массивов строк: Stream<String[]>  
Stream<String[]> withMap = sentences.stream()  
    .map(s -> s.split(" "));
```

```
// flatMap "разворачивает" массивы в единый поток слов: Stream<String>  
Stream<String> withFlatMap = sentences.stream()  
    .flatMap(s -> Arrays.stream(s.split(" ")));
```

```
// результат: "hello", "world", "foo", "bar", "baz"
```

flatMap используется всякий раз, когда каждый элемент отображается в коллекцию или поток, и нужно работать с этими элементами "плоско", а не с вложенными коллекциями. Типичные сценарии: развёртывание вложенных списков, получение всех элементов из коллекции коллекций, работа с Optional-потоками (flatMap для Optional "разворачивает" Optional внутри Optional).

JDBC

18. Создание Connection. Последовательность действий для запроса к БД. Заккрытие

Для работы с базой данных через JDBC нужно выполнить следующую последовательность действий.

Шаг 1 — Регистрация драйвера. До Java 6 нужно было явно вызывать Class.forName("com.mysql.cj.jdbc.Driver"). Начиная с Java 6 через механизм Service Provider Interface (SPI) драйвер загружается автоматически, если

JAR-файл драйвера находится в classpath и содержит файл META-INF/services/java.sql.Driver.

Шаг 2 — Получение Connection. Через DriverManager.getConnection(url, user, password), где URL имеет формат jdbc:postgresql://localhost:5432/mydb. Или через DataSource.getConnection() — предпочтительный способ в реальных приложениях, где DataSource управляет пулом соединений (HikariCP, C3P0, DBCP).

Шаг 3 — Создание Statement или PreparedStatement. Statement stmt = connection.createStatement() для простых запросов. PreparedStatement pstmt = connection.prepareStatement(sql) для параметризованных запросов.

Шаг 4 — Выполнение запроса. executeQuery(sql) для SELECT, возвращает ResultSet. executeUpdate(sql) для INSERT/UPDATE/DELETE, возвращает количество затронутых строк. execute(sql) — универсальный, для любого запроса.

Шаг 5 — Обработка ResultSet. Итерирование через while (rs.next()), извлечение данных через rs.getString("column"), rs.getInt("column") и другие методы.

Шаг 6 — Заккрытие ресурсов. Ресурсы нужно закрывать в порядке, обратном открытию: сначала ResultSet, затем Statement/PreparedStatement, затем Connection. Современный подход — использовать try-with-resources: Connection, PreparedStatement, ResultSet все реализуют AutoCloseable, что гарантирует закрытие даже при исключении:

Java

```
try (Connection conn = dataSource.getConnection();  
    PreparedStatement ps = conn.prepareStatement("SELECT * FROM users  
WHERE id = ?")) {
```

```

ps.setInt(1, userId);
try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        // обработка
    }
}
}

```

19. Statement vs PreparedStatement. Хранение запроса. Statement без параметров

Statement — для простых статических запросов без параметров. Каждый вызов `execute()` отправляет полный SQL-текст на сервер, где он парсится, компилируется и выполняется заново.

PreparedStatement — для параметризованных запросов с плейсхолдерами ?. При создании `prepareStatement(sql)` SQL отправляется на сервер, где компилируется и сохраняется план выполнения (`execution plan`). При последующих вызовах `execute()` передаются только значения параметров, а план выполнения переиспользуется. Это даёт ряд преимуществ.

Первое — производительность: план выполнения создаётся один раз. Если PreparedStatement вызывается многократно с разными параметрами (например, в цикле для пакетной вставки), сервер не тратит время на парсинг каждый раз. Эффект особенно значителен для сложных запросов.

Второе — защита от SQL-инъекций: параметры, переданные через `setString()`, `setInt()` и другие методы, автоматически экранируются. Они передаются как данные, а не как часть SQL-текста.

Третье — читаемость и удобство: код чище, нет конкатенации строк.

Будет ли тот же эффект от Statement, если формировать запрос в строке? Нет. Если вы делаете `Statement.execute("SELECT * WHERE id = " + id)`, это каждый раз разный SQL-текст (с разными значениями `id`), и база данных не

может кешировать план выполнения — каждый раз полная обработка. Более того, это уязвимо к SQL-инъекциям. Эффект кеширования плана достигается только при использовании PreparedStatement с параметрами ?.

20. Зачем нужен CallableStatement?

CallableStatement используется для вызова хранимых процедур (stored procedures) и хранимых функций (stored functions) в базе данных. Хранимые процедуры — это предварительно скомпилированный SQL-код, хранящийся непосредственно в СУБД.

Синтаксис вызова: CallableStatement cs = connection.prepareCall("{call procedure_name(?, ?, ?)}"); или "{? = call function_name(?, ?)}" для функций с возвращаемым значением.

CallableStatement поддерживает три типа параметров: IN-параметры (входные, устанавливаются через setXxx()), OUT-параметры (выходные, регистрируются через registerOutParameter() и читаются через getXxx() после выполнения), INOUT-параметры (одновременно входные и выходные).

Применяется, когда бизнес-логика инкапсулирована в хранимых процедурах на стороне БД, для вызова процедур, выполняющих сложные операции (batch обработка, расчёты на стороне БД), при работе с legacy-системами, где основная логика в базе данных.

21. Отличие executeUpdate() от executeQuery()

executeQuery(String sql) — предназначен для выполнения SELECT-запросов. Возвращает ResultSet с результатами запроса. Если передать не-SELECT запрос, поведение зависит от драйвера — обычно выбрасывается исключение.

executeUpdate(String sql) — предназначен для выполнения INSERT, UPDATE, DELETE и DDL-операций (CREATE TABLE, DROP TABLE и другие). Возвращает int — количество строк, затронутых операцией (для DML). Для

DDL-операций возвращает 0. Не возвращает ResultSet.

`execute(String sql)` — универсальный метод. Возвращает `boolean`: `true` если первый результат — это ResultSet (был SELECT), `false` если первый результат — количество обновлённых строк или нет результата. Используется для запросов, тип которых неизвестен заранее, или для запросов, возвращающих несколько результатов.

Для `PreparedStatement` аналогичные методы вызываются без аргументов (`executeQuery()`, `executeUpdate()`, `execute()`), потому что SQL уже задан при создании.

22. Транзакции. Принципы ACID

Транзакция — это группа операций с базой данных, которые выполняются как единое целое: либо все успешно завершаются, либо все откатываются, оставляя БД в исходном состоянии.

В JDBC по умолчанию включён режим автокоммита (`autoCommit = true`) — каждый SQL-оператор автоматически фиксируется как отдельная транзакция. Для явного управления транзакциями: `connection.setAutoCommit(false)`. Затем группа операций, `connection.commit()` для фиксации или `connection.rollback()` для отката при ошибке.

ACID — акроним из четырёх свойств, гарантирующих надёжность транзакций.

Atomicity (Атомарность): транзакция либо выполняется полностью, либо не выполняется вообще. Если на каком-то этапе произошла ошибка — все изменения отменяются (`rollback`). Нет промежуточных состояний.

Consistency (Согласованность): транзакция переводит базу данных из одного корректного состояния в другое корректное состояние. Все ограничения целостности (`constraints`) должны быть соблюдены до и после транзакции.

Isolation (Изолированность): одновременно выполняющиеся транзакции не влияют на результаты друг друга. Уровень изоляции определяет, насколько транзакции видят изменения других.

Durability (Долговечность): после успешной фиксации (commit) изменения сохраняются постоянно, даже в случае сбоя системы. Обычно реализуется через журнал транзакций (write-ahead log).

23. Уровни изоляции транзакций. Виды чтения. Точки сохранения

Проблемы параллельных транзакций:

Dirty Read (грязное чтение) — транзакция читает данные, изменённые другой транзакцией, которая ещё не зафиксирована. Если вторая транзакция откатится, первая прочитала "несуществующие" данные.

Non-Repeatable Read (неповторяющееся чтение) — транзакция читает одну и ту же строку дважды и получает разные результаты, потому что другая транзакция изменила и зафиксировала эту строку между двумя чтениями.

Phantom Read (фантомное чтение) — транзакция выполняет один и тот же запрос дважды и получает разный набор строк, потому что другая транзакция вставила или удалила строки между двумя выполнениями запроса.

Уровни изоляции (от наименее до наиболее строгого):

READ_UNCOMMITTED — самый низкий уровень. Возможны dirty read, non-repeatable read, phantom read. Транзакции видят незафиксированные изменения других. Максимальная производительность, минимальная безопасность.

READ_COMMITTED — исключает dirty read. Транзакция видит только зафиксированные данные. Возможны non-repeatable read и phantom read.

Уровень по умолчанию в большинстве СУБД (PostgreSQL, Oracle, SQL Server).

REPEATABLE_READ — исключает dirty read и non-repeatable read. Если строка прочитана в транзакции, повторное чтение вернёт те же данные. Возможны phantom read. Уровень по умолчанию в MySQL InnoDB.

SERIALIZABLE — самый высокий уровень. Исключает все три проблемы. Транзакции выполняются так, будто они последовательные. Максимальная безопасность, минимальная производительность из-за блокировок.

В JDBC уровень изоляции устанавливается через: `connection.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED)`.

Точки сохранения (Savepoint): позволяют откатить транзакцию не полностью, а только до определённой точки. `Savepoint sp = connection.setSavepoint("savepoint1");` — создать точку. `connection.rollback(sp)` — откатиться до точки. `connection.releaseSavepoint(sp)` — удалить точку.

24. SQL Injection: как работает и как защищаться

SQL Injection — это уязвимость, при которой злоумышленник вставляет вредоносный SQL-код в пользовательский ввод, который затем выполняется базой данных. Это одна из самых опасных и распространённых веб-уязвимостей.

Как работает: представьте код `String sql = "SELECT * FROM users WHERE name = '" + userInput + "'";`. Если пользователь вводит `admin' OR '1'=1`, итоговый SQL становится `SELECT * FROM users WHERE name = 'admin' OR '1'=1'`, что вернёт всех пользователей. Более опасный вариант: `'; DROP TABLE users; --` может удалить таблицу.

Защита:

Главный метод — `PreparedStatement` с параметрами. Значения параметров никогда не интерпретируются как SQL-код. `setString()` экранирует специальные символы: `ps.setString(1, userInput)`. Это делает инъекцию невозможной.

Валидация ввода — проверять и санитизировать все данные от пользователя на соответствие ожидаемому формату (whitelist подход — разрешать только то, что точно безопасно).

ORM-фреймворки (Hibernate, JPA) — при правильном использовании (JPQL/HQL с параметрами, Criteria API) автоматически защищают от SQL-инъекций.

Принцип наименьших привилегий — пользователь БД, от имени которого работает приложение, должен иметь только необходимые права (SELECT, INSERT, UPDATE на конкретные таблицы), но не DROP, CREATE или права суперпользователя.

Хранимые процедуры — при правильном использовании (с параметрами, без динамического SQL внутри) тоже защищают.

SQL

1. Нормализация

Нормализация — это процесс организации данных в базе данных с целью устранения избыточности (дублирования данных) и аномалий обновления (update anomalies), обеспечения целостности данных. Нормализация осуществляется путём разбиения таблиц и определения связей между ними.

1NF (Первая нормальная форма): каждая ячейка таблицы содержит атомарное (неделимое) значение. Нет повторяющихся групп или массивов в ячейке. Каждая строка уникальна (есть первичный ключ). Нарушение: хранить несколько телефонов в одной ячейке через запятую.

2NF (Вторая нормальная форма): соответствует 1NF и все не-ключевые атрибуты полностью функционально зависят от всего первичного ключа (нет частичных зависимостей). Актуально только при составном первичном ключе. Нарушение: в таблице заказов с ключом (order_id, product_id) хранить название продукта — оно зависит только от product_id, а не от всего ключа.

3NF (Третья нормальная форма): соответствует 2NF и нет транзитивных зависимостей (не-ключевой атрибут зависит от другого не-ключевого атрибута). Нарушение: в таблице сотрудников хранить department_name — оно зависит от department_id, а не напрямую от первичного ключа сотрудника.

BCNF (Нормальная форма Бойса-Кодда): усиленная 3NF — для каждой нетривиальной функциональной зависимости $X \rightarrow Y$, X должен быть суперключом.

На практике обычно нормализуют до 3NF — это разумный баланс между целостностью данных и производительностью. Иногда денормализацию применяют намеренно (добавляют избыточность) для ускорения чтения в OLAP-системах.

2. Типы связей в базе данных. Many-to-many

One-to-One (один к одному): одна запись в таблице A соответствует ровно одной записи в таблице B. Реализуется добавлением внешнего ключа в одну из таблиц с ограничением UNIQUE. Пример: пользователь и его профиль.

One-to-Many (один ко многим): одна запись в A соответствует многим записям в B. Самая распространённая связь. Реализуется внешним ключом в таблице B, указывающим на первичный ключ A. Пример: один автор может написать много книг. В таблице книг — поле author_id (foreign key).

Many-to-Many (многие ко многим): одна запись в A соответствует многим

записям в B, и одна запись в B соответствует многим записям в A. Не может быть реализована напрямую в реляционных БД. Решение — промежуточная (junction/associative) таблица. Например, студенты и курсы: таблица student_courses с полями student_id (FK → students) и course_id (FK → courses). Первичным ключом может быть пара (student_id, course_id). В промежуточной таблице можно хранить дополнительные атрибуты связи (дата записи, оценка).

3. Primary Key (первичный ключ)

Первичный ключ (Primary Key, PK) — это столбец или набор столбцов, значения которых однозначно идентифицируют каждую строку в таблице. Первичный ключ гарантирует: уникальность (не может быть двух строк с одинаковым значением PK), ненулевость (PK не может быть NULL), неизменность (значение PK не должно изменяться).

Первичный ключ может быть простым (один столбец) или составным (несколько столбцов). По каждому первичному ключу автоматически создаётся уникальный индекс для быстрого поиска.

Виды первичных ключей: суррогатный — искусственно созданный идентификатор без бизнес-смысла (auto-increment INT, UUID), не зависит от бизнес-данных. Натуральный — атрибут, имеющий бизнес-смысл (email, ИНН, ISBN) — рискованно, потому что бизнес-правила могут меняться. Рекомендуется использовать суррогатные ключи.

4. Foreign Key (внешний ключ)

Внешний ключ (Foreign Key, FK) — это столбец или набор столбцов, значения которых ссылаются на первичный ключ (или уникальный ключ) другой таблицы. Внешний ключ обеспечивает ссылочную целостность (referential integrity) — гарантирует, что нельзя создать "висячую" ссылку на несуществующую запись.

Действия при удалении/обновлении родительской записи (referential actions):

CASCADE — автоматически удалить/обновить зависимые записи; SET NULL — установить FK в NULL; SET DEFAULT — установить значение по умолчанию; RESTRICT/NO ACTION — запретить удаление/обновление, если есть зависимые записи (поведение по умолчанию).

FK может быть NULL (запись не обязательно должна быть связана с родителем). FK не обязан быть уникальным — это основа связи "один ко многим".

5. Индексы в базе данных

Индекс — это специальная структура данных, создаваемая для ускорения поиска строк в таблице по значениям определённых столбцов. Без индекса поиск выполняется через полное сканирование таблицы (full table scan) — $O(n)$. С индексом — $O(\log n)$ для B-tree индекса.

Как работает: наиболее распространённый тип — B-tree (сбалансированное дерево). Индекс хранит пары (значение ключа, указатель на строку в таблице), отсортированные по значению ключа. Это позволяет быстро находить строки по значению и выполнять операции диапазона. Также существуют Hash-индексы ($O(1)$ для точного совпадения, но не для диапазонов), Bitmap-индексы (для столбцов с малой кардинальностью), Full-text индексы (для поиска по тексту), пространственные индексы.

Преимущества: значительное ускорение SELECT-запросов с условием по индексированному столбцу, ускорение операций JOIN, сортировки ORDER BY и группировки GROUP BY по индексированным столбцам.

Недостатки: занимают дополнительное место на диске; замедляют операции INSERT, UPDATE, DELETE — при изменении данных индексы тоже обновляются; слишком много индексов — обратный эффект.

Где хранятся: индексы хранятся отдельно от данных таблицы, обычно в том же файловом пространстве базы данных (tablespace). В PostgreSQL — в

отдельных файлах, в MySQL InnoDB — индексы сохраняются в .ibd файлах вместе с данными таблицы.

Индексы создаются автоматически для PRIMARY KEY и UNIQUE-ограничений. Остальные создаются явно: CREATE INDEX idx_name ON table(column).

6. Типы JOIN

INNER JOIN — возвращает только строки, для которых есть совпадение в обеих таблицах. Строки без совпадения исключаются.

LEFT (OUTER) JOIN — возвращает все строки левой таблицы и совпадающие строки правой. Для строк левой таблицы, не имеющих совпадения справа, столбцы правой таблицы заполняются NULL.

RIGHT (OUTER) JOIN — зеркально противоположен LEFT JOIN. Возвращает все строки правой таблицы и совпадающие строки левой. На практике почти всегда можно заменить LEFT JOIN с переставленными таблицами.

FULL (OUTER) JOIN — возвращает все строки обеих таблиц. Где нет совпадения — NULL. Объединение LEFT и RIGHT JOIN.

CROSS JOIN — декартово произведение: каждая строка первой таблицы соединяется с каждой строкой второй. Если в таблице A — N строк, в таблице B — M строк, результат содержит N×M строк. Редко используется, осторожно с большими таблицами.

SELF JOIN — таблица соединяется сама с собой. Используется для иерархических данных (например, сотрудник — менеджер в одной таблице).

7. HAVING: для чего используется, отличия от WHERE

WHERE — фильтрует строки до группировки. Применяется к отдельным

строкам. Не может использовать агрегатные функции (COUNT, SUM, AVG и другие), потому что агрегация ещё не выполнена. WHERE выполняется для каждой строки.

HAVING — фильтрует группы после группировки (GROUP BY). Применяется к результатам агрегации. Может использовать агрегатные функции. HAVING выполняется для каждой группы.

SQL

```
SELECT department_id, COUNT(*) as emp_count, AVG(salary) as avg_salary
FROM employees
WHERE salary > 30000      -- фильтруем строки до группировки
GROUP BY department_id
HAVING COUNT(*) > 5      -- фильтруем группы после группировки
      AND AVG(salary) > 50000; -- нельзя было бы написать в WHERE
```

Порядок выполнения SQL: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. HAVING без GROUP BY применяется ко всей таблице как к одной группе.

8. View (представление): зачем нужно и какие поля там будут

View (представление) — это именованный сохранённый SQL-запрос, который ведёт себя как виртуальная таблица. View не хранит данные — при обращении к view выполняется лежащий в его основе запрос. Создаётся: CREATE VIEW view_name AS SELECT

Зачем нужно: упрощение сложных запросов (скрывает сложные JOIN-ы и агрегации за простым именем), безопасность (можно предоставить пользователю доступ к view с ограниченным набором столбцов, скрыв конфиденциальные данные), абстракция (клиентский код работает с view, а структура таблиц может изменяться без изменения клиентского кода), повторное использование запросов.

Какие поля будут в view: ровно те, которые определены в SELECT-запросе view. Это может быть подмножество столбцов из таблицы, вычисляемые столбцы (выражения, агрегаты), столбцы из нескольких таблиц (через JOIN), переименованные столбцы (через AS).

Materialized View (не стандарт SQL, но поддерживается PostgreSQL, Oracle) — view, которое реально хранит данные снимка. Нужно явное обновление (REFRESH MATERIALIZED VIEW). Используется для ускорения тяжёлых аналитических запросов.

9. Агрегирующие функции. Примеры

Агрегирующие функции — функции, которые вычисляют единственное значение из набора строк. Применяются к группе строк (при использовании GROUP BY) или ко всем строкам таблицы (без GROUP BY).

COUNT(*) — количество строк. COUNT(column) — количество строк, где column не NULL. COUNT(DISTINCT column) — количество уникальных значений.

SUM(column) — сумма значений (для числовых типов). Игнорирует NULL.

AVG(column) — среднее значение. Игнорирует NULL.

MIN(column) — минимальное значение.

MAX(column) — максимальное значение.

STRING_AGG(column, separator) (PostgreSQL) / GROUP_CONCAT(column) (MySQL) — конкатенация строк из группы.

ARRAY_AGG(column) (PostgreSQL) — агрегация значений в массив.

Все агрегирующие функции (кроме COUNT(*)) игнорируют NULL-значения.

Используются в SELECT совместно с GROUP BY — не-агрегированные столбцы в SELECT должны быть в GROUP BY.

10. Query Execution Plan: что это и как посмотреть

Query Execution Plan (план выполнения запроса) — это описание того, как именно СУБД планирует выполнить SQL-запрос: какие индексы использовать, в каком порядке соединять таблицы, какие алгоритмы применять. Оптимизатор СУБД анализирует запрос и статистику по данным, чтобы выбрать наиболее эффективный план.

Для просмотра плана используются команды (зависит от СУБД):

PostgreSQL: EXPLAIN SELECT ... — показывает план без выполнения. EXPLAIN ANALYZE SELECT ... — выполняет запрос и показывает реальное время выполнения каждого шага. EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ... — детальный план с информацией о буферах.

MySQL: EXPLAIN SELECT ... или EXPLAIN FORMAT=JSON SELECT ... EXPLAIN ANALYZE SELECT ... (MySQL 8.0+).

Oracle: EXPLAIN PLAN FOR SELECT ... + SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY).

SQL Server: SET SHOWPLAN_TEXT ON или кнопка "Include Actual Execution Plan" в SSMS.

В плане выполнения нужно обращать внимание на: Seq Scan / Full Table Scan (полное сканирование — может быть медленным), Index Scan / Index Seek (использование индекса — обычно быстро), Join алгоритмы (Nested Loop, Hash Join, Merge Join), estimated vs actual rows (расхождение говорит об устаревшей статистике), cost (оценочная стоимость операции). На основе плана можно выявить узкие места и добавить нужные индексы.

DESIGN PATTERNS

1. Зачем нужны паттерны? Примеры из проекта

Паттерны проектирования (Design Patterns) — это типовые решения часто встречающихся проблем проектирования программного обеспечения. Это не готовый код, а описание подхода (шаблон), который можно адаптировать к конкретной ситуации.

Зачем нужны: во-первых, они представляют проверенные решения — не нужно изобретать велосипед для типичных задач. Во-вторых, они обеспечивают общий язык — когда разработчики говорят "здесь нужен Singleton" или "используй Factory", все понимают, о чём речь. В-третьих, они улучшают структуру кода — паттерны ведут к более гибкому, расширяемому и поддерживаемому коду. В-четвёртых, они снижают связность (coupling) и повышают сцепленность (cohesion).

Паттерны делятся на три категории: Порождающие (Creational) — создание объектов; Структурные (Structural) — организация классов и объектов; Поведенческие (Behavioral) — взаимодействие между объектами.

2. Паттерны в Java SE

Java SE сама по себе является отличным примером применения паттернов. Singleton — `Runtime.getRuntime()`, `System`. Factory Method — `Calendar.getInstance()`, `NumberFormat.getInstance()`. Abstract Factory — `DocumentBuilderFactory`. Builder — `StringBuilder`, `Stream.Builder`, `ProcessBuilder`. Prototype — `Object.clone()`. Decorator — потоки ввода/вывода (`BufferedInputStream` оборачивает `FileInputStream`). Adapter — `Arrays.asList()`, `InputStreamReader`. Iterator — `Iterator`, `Enumeration`. Observer — `listeners` в `Swing`, `EventListener`. Strategy — `Comparator`. Template Method — `AbstractList`. Proxy — динамические прокси через рефлексия (`java.lang.reflect.Proxy`). Composite — `javax.swing.JComponent`. Command — `Runnable`, `Callable`.

3. Factory Method и Builder

Factory Method — паттерн, определяющий интерфейс для создания объекта, но позволяющий подклассам (или отдельным фабричным методам) решать, объект какого класса создавать. Инкапсулирует логику создания объекта. Клиент работает с продуктом через интерфейс, не зная о конкретной реализации. В Java SE: `Calendar.getInstance()` возвращает конкретную реализацию `Calendar` в зависимости от локали. `Collections.unmodifiableList()`. `Paths.get()`.

Builder — паттерн для пошагового создания сложных объектов. Особенно полезен, когда у объекта много опциональных параметров (альтернатива телескопическим конструкторам). Предоставляет fluent API (method chaining). `StringBuilder` — мутабельный строитель строк. `Stream.Builder`. `ProcessBuilder` для создания процессов ОС. В современном коде часто используется с библиотеками Lombok (`@Builder`) для автогенерации:

Java

```
Person person = Person.builder()
    .name("Alice").age(30).email("alice@example.com")
    .build();
```

4. Singleton

Singleton — паттерн, гарантирующий существование единственного экземпляра класса в пределах одного `ClassLoader` и предоставляющий глобальную точку доступа к нему.

В Java SE: `Runtime.getRuntime()` — единственный экземпляр среды выполнения. В веб-приложениях `Servlet` и `ServletContext` также являются синглтонами в пределах веб-приложения.

Наилучшая реализация Singleton — через `enum` (потокобезопасна, защита от рефлексии и сериализации):

Java

```
public enum Singleton { INSTANCE; public void doWork() { ... } }
```

Реализация через статическое поле (Eager initialization — создаётся при загрузке класса):

Java

```
public class Singleton {  
    private static final Singleton INSTANCE = new Singleton();  
    private Singleton() {}  
    public static Singleton getInstance() { return INSTANCE; }  
}
```

Реализация Double-Checked Locking (Lazy initialization, потокобезопасная):

Java

```
public class Singleton {  
    private static volatile Singleton instance;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) { instance = new Singleton(); }  
            }  
        }  
        return instance;  
    }  
}
```

5. Как создать второй объект Singleton и как воспрепятствовать

Способы создать второй экземпляр Singleton:

Через рефлексию: `Constructor<Singleton> c = Singleton.class.getDeclaredConstructor(); c.setAccessible(true); Singleton s2 =`

`c.newInstance()`; — обходит приватный конструктор. Защита: в конструкторе проверять, что экземпляр уже создан, и выбрасывать исключение при повторном вызове.

Через сериализацию и десериализацию: при десериализации создаётся новый объект в обход конструктора. Защита: реализовать метод `protected Object readResolve() { return INSTANCE; }` — он заменяет десериализованный объект на существующий.

Через разные `ClassLoader`'ы: каждый `ClassLoader` имеет свои статические поля, поэтому класс, загруженный двумя разными `ClassLoader`'ами, создаст два "синглтона".

Через клонирование: если `Singleton` реализует `Cloneable`. Защита: переопределить `clone()` и выбрасывать `CloneNotSupportedException`.

`Enum Singleton` автоматически защищён от рефлексии (`IllegalArgumentException` при попытке использовать конструктор через рефлексию), от сериализации (`enum`-константы сериализуются по имени и десериализуются в ту же константу), и от клонирования (`enum` нельзя клонировать).

6. Prototype (`clone()`)

Prototype — паттерн, позволяющий копировать объекты, не вдаваясь в детали их реализации. Вместо создания нового объекта с нуля — клонирование существующего.

В Java реализуется через `Object.clone()` и интерфейс `Cloneable`. Актуален, когда создание объекта "с нуля" дорогостоящее (много вычислений, обращений к БД), но копирование дешевле. Или когда нужно создать объект, аналогичный существующему, с небольшими изменениями.

Shallow copy — `super.clone()` копирует поля побитово. Для примитивов и

иммутабельных объектов достаточно. Deep copy — нужно вручную клонировать изменяемые ссылочные поля.

Альтернативы clone(): конструктор копирования (new MyClass(original)), фабричный метод MyClass.copyOf(original), сериализация/десериализация для глубокого копирования. Механизм clone() считается проблематичным (не вызывает конструктор, Cloneable не содержит метод clone()) — конструктор копирования предпочтительнее.

7. Command

Command — поведенческий паттерн, инкапсулирующий запрос как объект. Позволяет параметризовать методы с разными запросами, ставить запросы в очередь, логировать их, поддерживать отмену операций (undo).

Компоненты: Command (интерфейс с методом execute()), ConcreteCommand (конкретная команда), Invoker (выполняет команды), Receiver (объект, над которым выполняется действие).

В Java: Runnable и Callable — реализация Command (задача инкапсулируется и передаётся исполнителю). ExecutorService — invoker. Транзакционные системы используют паттерн Command с undo для журналирования операций. В GUI-приложениях — действия меню и кнопок как команды.

8. Composite

Composite — структурный паттерн, позволяющий компоновать объекты в древовидные структуры и работать с одиночными объектами и их группами единообразно через общий интерфейс.

Компоненты: Component (общий интерфейс), Leaf (конечный элемент без "детей"), Composite (элемент, содержащий список дочерних Component и делегирующий им операции).

В Java SE: java.nio.file.Path — путь составлен из компонентов.

`javax.swing.JComponent` — контейнеры содержат компоненты, компоненты могут быть контейнерами. DOM (Document Object Model) — `Node` в `org.w3c.dom`: `Document`, `Element`, `Text` — все наследуют `Node`. `java.awt.Container` содержит `Component`-объекты.

9. Chain of Responsibility

Chain of Responsibility — поведенческий паттерн, передающий запрос по цепочке обработчиков. Каждый обработчик решает, обработать запрос самому или передать следующему.

В Java SE: цепочка `Filter`-ов в `Servlet API` (каждый `Filter` обрабатывает запрос и передаёт следующему через `FilterChain.doFilter()`). Закрывание вложенных `InputStream`-ов (закрывание внешнего потока закрывает внутренние по цепочке). Логгеры в `java.util.logging` — `Logger` передаёт записи родительскому логгеру по иерархии. Обработчики исключений.

10. State

State — поведенческий паттерн, позволяющий объекту изменять своё поведение при изменении внутреннего состояния. Объект будет казаться изменившим свой класс.

Вместо огромных `if-else` или `switch` по состоянию — отдельный класс для каждого состояния, реализующий общий интерфейс. Переключение состояний происходит в методах самих состояний или в контексте.

В Java SE: `Thread.State` — перечисление состояний потока (`NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`). Поток ведёт себя по-разному в каждом состоянии. Паттерн State — концептуальная основа автоматов состояний, используемых в `workflow`-системах, игровой логике, парсерах.

11. Iterator

Iterator — поведенческий паттерн, предоставляющий способ

последовательного обхода элементов агрегированного объекта без раскрытия его внутренней структуры.

В Java: интерфейс `java.util.Iterator<E>` с методами `hasNext()`, `next()`, `remove()`. `ListIterator<E>` — двунаправленный итератор с дополнительными методами `hasPrevious()`, `previous()`, `add()`, `set()`. `Enumeration` — устаревший аналог `Iterator` (метод `hasMoreElements()`, `nextElement()`), используется в `Vector`, `Hashtable`, `Stack`. `ResultSet` в JDBC — навигация по результатам запроса через `next()`, также `absolute()`, `previous()`, `first()`, `last()` для прокручиваемых `ResultSet`.

12. Proxy

Proxy — структурный паттерн, предоставляющий суррогатный объект (прокси), контролирующий доступ к другому объекту. Прокси реализует тот же интерфейс, что и реальный объект.

Виды прокси: `Virtual Proxy` (ленивая инициализация тяжёлого объекта), `Remote Proxy` (доступ к объекту в другом адресном пространстве), `Protection Proxy` (контроль доступа), `Caching Proxy`, `Logging Proxy`.

В Java: Динамические прокси через рефлексию — `java.lang.reflect.Proxy.newProxyInstance()` создаёт прокси-объект в рантайме. Прокси реализует указанные интерфейсы, все вызовы перехватываются `InvocationHandler`. Это основа многих фреймворков: Spring AOP, Hibernate lazy loading, транзакционные прокси в Spring. Начиная с Java 15 — `java.lang.reflect.Proxy` для интерфейсов, для классов — библиотеки CGLib или ByteBuddy (используются Spring'ом).

13. Observer

Observer — поведенческий паттерн, определяющий зависимость "один ко многим" между объектами: при изменении состояния одного объекта (`Subject/Observable`) все зависящие от него объекты (`Observer/Listener`) уведомляются автоматически.

В Java SE: `java.util.EventListener` — маркерный интерфейс для слушателей событий. В Swing: `ActionListener`, `MouseListener`, `KeyListener` регистрируются на компонентах. `java.util.Observer` и `java.util.Observable` — устаревшая реализация (deprecated с Java 9). В современном коде используют: события через реактивные потоки (Flow API в Java 9 — `Publisher/Subscriber/Processor`), библиотеки RxJava, ProjectReactor, паттерн EventBus.

14. Wrapper/Adapter/Decorator

Три паттерна схожи структурно (один объект оборачивает другой), но различаются целью.

Adapter — преобразует интерфейс одного класса в интерфейс, ожидаемый клиентом. Делает несовместимые классы совместимыми. В Java: `Arrays.asList()` адаптирует массив к интерфейсу `List`. `InputStreamReader` адаптирует `InputStream` (байтовый) к `Reader` (символьный). `Collections.enumeration(list)` адаптирует `Iterator` к `Enumeration`.

Decorator — динамически добавляет объекту новые обязанности, не изменяя его класс. Реализует тот же интерфейс, что и декорируемый объект, и делегирует ему работу, добавляя своё поведение. В Java: вся система I/O — `BufferedInputStream(new FileInputStream(...))`, `DataOutputStream(new FileOutputStream(...))`. `Collections.unmodifiableList()`, `Collections.synchronizedList()` — декораторы, добавляющие поведение.

Проху — контролирует доступ к объекту. Структурно похож на Decorator, но цель иная: не добавить функциональность, а контролировать доступ.

Wrapper — общий термин для паттернов, где объект "оборачивается" в другой. Классы-обёртки (`Integer`, `Double`) в Java — это Wrapper над примитивами.

15. Immutable и Record

Immutable — паттерн (и принцип проектирования) создания объектов, чьё

состояние не может изменяться после инициализации. Преимущества: потокобезопасность без синхронизации, простота рассуждений, безопасное кеширование. Реализация: `final` класс, `private final` поля, нет сеттеров, защитные копии изменяемых полей в конструкторе и геттерах. Примеры в Java: `String`, классы-обёртки, `LocalDate`, `BigInteger`.

`Record` (Java 16+) — специальный тип класса, предназначенный для неизменяемых классов-держателей данных. Компилятор автоматически генерирует компактный конструктор, геттеры (имя без `get`-префикса), `equals()`, `hashCode()`, `toString()`. Неявно `final`. Может реализовывать интерфейсы, иметь дополнительные методы и статические поля, но не может наследоваться от классов и иметь изменяемые поля экземпляра:

Java

```
public record Person(String name, int age) {  
    // компактный конструктор для валидации:  
    public Person { if (age < 0) throw new IllegalArgumentException(); }  
}
```

16. MVC

MVC (Model-View-Controller) — архитектурный паттерн, разделяющий приложение на три компонента.

Model (Модель) — данные и бизнес-логика. Не зависит от View и Controller. Уведомляет наблюдателей об изменениях (паттерн Observer).

View (Представление) — отображение данных пользователю. Зависит от Model (читает данные), но не содержит бизнес-логики. В веб: HTML-шаблоны, JSP, Thymeleaf.

Controller (Контроллер) — обрабатывает ввод пользователя, обновляет Model, выбирает View для отображения. В Spring MVC: методы, аннотированные `@Controller` + `@RequestMapping`.

Производные паттерны: MVP (Model-View-Presenter) — View пассивен, Presenter управляет и View, и Model. MVVM (Model-View-ViewModel) — ViewModel содержит состояние View и логику представления, двусторонняя привязка. Используется в Angular, React (partial).

17. DAO vs Repository

DAO (Data Access Object) — паттерн, инкапсулирующий логику доступа к одному источнику данных (таблице, файлу). DAO работает с абстракцией хранилища и скрывает детали SQL/JDBC от бизнес-логики. Обычно реализует методы CRUD для конкретной сущности: `UserDAO.findById()`, `UserDAO.save()`, `UserDAO.delete()`. DAO тесно привязан к конкретной технологии хранения.

Repository — паттерн из Domain-Driven Design (DDD). Абстрагирует коллекцию доменных объектов. Работает на более высоком уровне абстракции — имитирует коллекцию объектов в памяти. Скрывает не только детали хранилища, но и то, что хранилище вообще существует. Репозиторий может объединять данные из нескольких источников. В Spring Data JPA интерфейс `UserRepository extends JpaRepository<User, Long>` — классический пример.

Ключевые отличия: DAO ближе к хранилищу (думает в терминах таблиц и SQL). Repository ближе к домену (думает в терминах объектов и бизнес-операций). Repository может использовать внутри несколько DAO. В Spring-приложениях часто используют Repository (через Spring Data), но концептуально граница размыта — оба паттерна решают схожую задачу разными акцентами.

JEE, Docker, Git, Spring, REST — Подробные Ответы
JEE (Java Enterprise Edition)

1. Сервер приложений vs Веб-сервер

Чтобы разобраться в этом вопросе, нужно понять, какие задачи решает каждый из этих компонентов в инфраструктуре веб-приложений.

Веб-сервер — это программа, которая обрабатывает HTTP-запросы и возвращает статический контент: HTML-файлы, CSS, JavaScript, изображения, видео. Веб-сервер понимает протокол HTTP/HTTPS и умеет отдавать файлы с диска напрямую клиенту (браузеру). Также веб-серверы умеют проксировать запросы на другие серверы и балансировать нагрузку. Классические примеры веб-серверов: Apache HTTP Server (Apache httpd) — один из самых популярных в мире, существует с 1995 года; Nginx — высокопроизводительный веб-сервер и обратный прокси, отлично справляется с большим количеством одновременных соединений; Microsoft IIS (Internet Information Services) — веб-сервер для Windows.

Веб-сервер сам по себе не умеет выполнять Java-код или другую бизнес-логику. Для этого ему нужна помощь.

Сервер приложений — это более тяжеловесная программная среда, которая предоставляет полную инфраструктуру для выполнения корпоративных приложений. Сервер приложений включает в себя функциональность веб-сервера и добавляет поддержку выполнения кода (Java EE / Jakarta EE), управление жизненным циклом компонентов (сервлетов, EJB), пул соединений к БД, управление транзакциями (JTA), внедрение зависимостей (CDI), безопасность (JAAS), JMS (обмен сообщениями), JNDI (именование ресурсов) и многое другое. Примеры серверов приложений: Apache Tomcat — строго говоря, это контейнер сервлетов (servlet container), а не полноценный сервер приложений, но часто называется им в разговоре; поддерживает сервлеты и JSP, но не полный Jakarta EE; WildFly (бывший JBoss) — полноценный Jakarta EE сервер от Red Hat; GlassFish — эталонная реализация Jakarta EE от Oracle; IBM WebSphere — коммерческий сервер приложений; Oracle WebLogic — коммерческий сервер от Oracle; Payara — форк GlassFish с активной поддержкой.

Ключевое отличие: веб-сервер отдаёт статику по HTTP, сервер приложений выполняет динамическую бизнес-логику. На практике в продакшн-системах часто используют связку: Nginx (как обратный прокси и для отдачи статики) → Tomcat/WildFly (для выполнения Java-кода).

2. Контейнер сервлетов. Что такое сервлет

Сервлет — это Java-класс, работающий на стороне сервера и обрабатывающий запросы от клиентов (как правило, HTTP-запросы из браузера) и генерирующий динамические ответы. Сервлет наследует от `javax.servlet.http.HttpServlet` (или `jakarta.servlet.http.HttpServlet` в новых версиях) и переопределяет методы для обработки разных HTTP-методов. Сервлет получает объект `HttpServletRequest` с информацией о запросе и `HttpServletResponse` для формирования ответа.

Контейнер сервлетов (Servlet Container, Servlet Engine) — это часть сервера приложений (или отдельная программа, как Tomcat), которая управляет жизненным циклом сервлетов, маршрутизирует HTTP-запросы к нужным сервлетам и предоставляет им необходимую инфраструктуру. Контейнер берёт на себя: создание экземпляров сервлетов и управление ими (один экземпляр сервлета обслуживает множество запросов в разных потоках), маппинг URL на конкретные сервлеты, управление сессиями, поддержку фильтров, управление безопасностью. Apache Tomcat — это именно контейнер сервлетов, который также поддерживает JSP.

Сервлет — это компонент, контейнер — среда его выполнения. Контейнер — это IoC-контейнер в терминах Jakarta EE: он создаёт объекты (сервлеты) и управляет ими, а не программист.

3. Методы сервлета. Жизненный цикл

Жизненный цикл сервлета определяется тремя ключевыми методами, вызываемыми контейнером.

`init(ServletConfig config)` — вызывается контейнером один раз при создании экземпляра сервлета (при первом обращении к сервлету или при старте сервера, в зависимости от конфигурации `load-on-startup`). Используется для инициализации ресурсов: открытие соединений с БД, чтение конфигурации. После успешного завершения `init()` сервлет готов обрабатывать запросы. Если `init()` выбрасывает `ServletException` или `UnavailableException` — сервлет считается неинициализированным.

`service(ServletRequest req, ServletResponse res)` — вызывается контейнером для каждого запроса в отдельном потоке. В `HttpServlet` метод `service()` анализирует HTTP-метод запроса и вызывает соответствующий метод: `doGet()`, `doPost()`, `doPut()`, `doDelete()`, `doHead()`, `doOptions()`, `doTrace()`. Важно понимать, что один экземпляр сервлета обслуживает множество одновременных запросов в разных потоках — поэтому сервлеты должны быть потокобезопасными (не хранить изменяемое состояние в полях экземпляра).

`destroy()` — вызывается контейнером один раз перед уничтожением экземпляра сервлета (при перезапуске приложения, при выключении сервера). Используется для освобождения ресурсов: закрытие соединений, сохранение состояния. После вызова `destroy()` сервлет больше не обрабатывает запросы.

Полный жизненный цикл: Загрузка класса сервлета → Создание экземпляра (`new`) → Вызов `init()` → Многократный вызов `service()` (параллельно в разных потоках) → Вызов `destroy()` → Объект доступен GC.

Основные методы для переопределения в `HttpServlet`: `doGet()` — обработка GET-запросов, `doPost()` — POST-запросов, `doPut()` — PUT, `doDelete()` — DELETE.

4. JSP (JavaServer Pages). Жизненный цикл

JSP (JavaServer Pages) — это технология, позволяющая создавать динамические веб-страницы путём встраивания Java-кода прямо в HTML-шаблон. JSP-файл (.jsp) содержит смесь HTML, специальных JSP-тегов и Java-кода. На первый взгляд это удобнее, чем генерировать HTML из Java-кода в сервлете (`out.println("<html>...")`) — это неудобно).

JSP-файл использует специальный синтаксис: скриптлеты `<% код %>` для вставки Java-кода, выражения `<%= значение %>` для вывода значения, директивы `<%@ ... %>` для настройки страницы (импорт классов, включение файлов), декларации `<%! ... %>` для объявления полей и методов, EL-выражения `${переменная}` (Expression Language) и JSTL-теги для шаблонизации без Java в JSP.

Жизненный цикл JSP: по своей сути JSP — это синтаксический сахар над сервлетами. Контейнер при первом обращении к JSP-файлу выполняет трансляцию: JSP преобразуется в Java-класс, наследующий `HttpServlet` (точнее `HttpJspBase`). Этот класс компилируется в байт-код. Далее жизненный цикл совпадает с жизненным циклом сервлета: `jspInit()` (аналог `init()`), `_jspService()` (аналог `service()`, вызывается для каждого запроса), `jspDestroy()` (аналог `destroy()`). При последующих запросах трансляция и компиляция не выполняются — используется уже скомпилированный класс (если JSP-файл не изменился).

На практике в современных приложениях JSP вытеснен шаблонизаторами — Thymeleaf, FreeMarker, Mustache — или полностью разделёнными клиентскими приложениями (SPA на React, Angular, Vue).

5. Сессия. Жизненный цикл

Сессия (HTTP Session) — это механизм хранения состояния между несколькими HTTP-запросами одного пользователя. HTTP сам по себе является stateless (без состояния) протоколом — каждый запрос независим, и сервер не знает, что предыдущий запрос был от того же пользователя. Сессия решает эту проблему, предоставляя "хранилище" данных на стороне сервера,

связанное с конкретным пользователем через уникальный идентификатор.

В Java сессия представлена объектом `HttpSession`. Создается через `request.getSession()` (создаёт если нет) или `request.getSession(false)` (возвращает `null` если нет). Данные хранятся в сессии через `setAttribute("key", value)` и читаются через `getAttribute("key")`.

Идентификатор сессии (Session ID) — случайная строка, которая по умолчанию передаётся клиенту через Cookie с именем `JSESSIONID`. При каждом следующем запросе браузер отправляет этот cookie обратно, и сервер по Session ID находит соответствующий объект `HttpSession` с данными пользователя. Альтернатива cookie — URL rewriting, когда Session ID добавляется к URL: `example.com/page?JSESSIONID=abc123`.

Жизненный цикл сессии: создание при первом вызове `getSession()` или `getSession(true)` → использование (`setAttribute/getAttribute`) → истечение времени неактивности (`session.setMaxInactiveInterval(seconds)`, по умолчанию обычно 30 минут) или явное завершение через `session.invalidate()` → удаление из памяти сервера. Сессии хранятся в памяти сервера (по умолчанию), что создаёт проблему при горизонтальном масштабировании — решается репликацией сессий или хранением в внешнем хранилище (Redis, Memcached) — Sticky Sessions или Session Replication.

6. Request. Из чего состоит. Жизненный цикл

HTTP Request — это сообщение, отправляемое клиентом (браузером) серверу. В мире Java-сервлетов запрос представлен объектом `HttpServletRequest`.

HTTP-запрос состоит из следующих частей:

Стартовая строка (Request Line): метод (GET, POST, PUT, DELETE и другие), URL запрашиваемого ресурса, версия HTTP (HTTP/1.1, HTTP/2). Например: `GET /users/123?format=json HTTP/1.1`.

Заголовки (Headers): пары ключ-значение с метайнформацией. Примеры: Host: example.com (обязательный в HTTP/1.1), Content-Type: application/json (тип тела запроса), Authorization: Bearer token123, Cookie: JSESSIONID=abc, Accept: text/html,application/json, User-Agent: Mozilla/5.0..., Content-Length: 42.

Пустая строка — разделитель между заголовками и телом.

Тело запроса (Body/Payload): данные, передаваемые в запросе. Присутствует в POST, PUT, PATCH. Для GET обычно отсутствует. Формат определяется заголовком Content-Type.

Объект HttpServletRequest предоставляет доступ ко всем этим данным: `getMethod()`, `getRequestURI()`, `getHeader("name")`, `getParameter("name")`, `getInputStream()` для тела, `getSession()`, `getCookies()`, `getAttribute()`, `getRemoteAddr()` (IP клиента).

Жизненный цикл объекта Request: создаётся контейнером при получении HTTP-запроса → передаётся сервлету (и фильтрам) → уничтожается контейнером после завершения обработки (после возврата из метода `service()`). Область видимости объекта Request — один запрос/ответ. Нельзя хранить ссылку на Request и использовать после завершения запроса.

7. Cookies. Как достать. Если удалить cookies, можно ли достать сессию. Жизненный цикл

Cookie — это небольшой фрагмент данных (пара ключ-значение), который сервер отправляет браузеру, а браузер хранит и автоматически отправляет обратно серверу при каждом последующем запросе к тому же домену. Cookie используются для хранения Session ID, пользовательских предпочтений, данных для аутентификации, аналитики.

Как сервер устанавливает Cookie: в ответе через заголовок Set-Cookie: `name=value; Path=/; HttpOnly; Secure; SameSite=Strict; Expires=....` В сервлете: `Cookie c = new Cookie("name", "value"); response.addCookie(c);`.

Как достать Cookies в сервлете: `Cookie[] cookies = request.getCookies();` — возвращает массив всех cookie, пришедших с запросом. Затем итерируем и ищем нужный по имени.

Если удалить Cookie с Session ID (JSESSIONID): нет, достать сессию в обычном сценарии будет невозможно. Если браузер удалил cookie JSESSIONID, то при следующем запросе сервер не получит Session ID, не сможет найти соответствующий объект `HttpSession` в своём хранилище, и вызов `request.getSession(false)` вернёт `null`. `request.getSession(true)` создаст новую сессию с новым ID. Данные старой сессии останутся на сервере до истечения тайм-аута. Если используется URL rewriting (Session ID в URL), то сессию всё же можно восстановить, если URL с JSESSIONID сохранён. На практике удаление cookie сессии равносильно "выходу из системы" с точки зрения сервера.

Жизненный цикл Cookie: создание сервером и отправка клиенту → хранение в браузере → автоматическая отправка с каждым запросом к соответствующему домену → истечение срока (через `setMaxAge(seconds): 0` — удалить немедленно, отрицательное — сессионный cookie, удаляется при закрытии браузера, положительное — хранится указанное число секунд) или явное удаление пользователем/браузером.

Атрибуты безопасности Cookie: `HttpOnly` — cookie недоступен из JavaScript, защита от XSS; `Secure` — cookie отправляется только по HTTPS; `SameSite` — защита от CSRF-атак (Strict, Lax, None).

8. Отличия POST и GET. Метод по умолчанию

GET — запрос на получение данных. Параметры передаются в URL в строке запроса (query string): `GET /search?q=java&page=1`. Данные видны в URL, в истории браузера, в логах сервера — поэтому не подходит для передачи конфиденциальных данных. GET-запросы могут кешироваться браузером и прокси-серверами. GET является идемпотентным и безопасным —

многократное выполнение одного запроса не изменяет состояние сервера и всегда возвращает одинаковый результат. Объем данных ограничен длиной URL (обычно около 2000-8000 символов в зависимости от браузера и сервера).

POST — запрос на отправку данных для обработки (создание ресурса, передача формы). Данные передаются в теле запроса, а не в URL. Данные не видны в URL, не кешируются. POST не является идемпотентным — каждый вызов может создавать новый ресурс или производить побочный эффект. Объем передаваемых данных практически не ограничен (ограничен лишь настройками сервера). Используется для форм, загрузки файлов, передачи JSON-тела.

Метод по умолчанию: если в HTML-форме не указан атрибут `method`, по умолчанию используется GET. Аналогично, при переходе по ссылке в браузере всегда используется GET. Если браузер обращается к ресурсу напрямую (ввод URL в адресную строку) — тоже GET.

9. Отличия `doPost()` и `doPut()`. Идемпотентные методы

`doPost()` / HTTP POST — предназначен для создания нового ресурса или отправки данных для обработки. Каждый вызов POST может создать новый ресурс — поэтому POST не является идемпотентным. Повторная отправка одной и той же POST-формы создаст дубликат. Сервер сам определяет URI нового ресурса.

`doPut()` / HTTP PUT — предназначен для полного замещения (обновления) существующего ресурса по конкретному URI. Клиент заранее знает URI ресурса. PUT является идемпотентным: многократное выполнение одинакового PUT-запроса приводит к одному и тому же результату — ресурс каждый раз замещается теми же данными. Если ресурса нет — некоторые реализации создают его. PUT заменяет ресурс целиком (в отличие от PATCH, который обновляет частично).

Идемпотентные HTTP-методы — те, многократное выполнение которых с одинаковыми параметрами приводит к одному и тому же результату: GET — идемпотентный и безопасный (не изменяет состояние), HEAD — аналогично GET но без тела ответа, PUT — идемпотентный, DELETE — идемпотентный (удаление уже удалённого ресурса даёт тот же результат — ресурса нет), OPTIONS — идемпотентный. Не идемпотентные: POST (каждый раз может создавать новый ресурс), PATCH (частичное обновление может быть не идемпотентным).

10. Scopes (области видимости) в JSP

В JSP существует четыре области видимости для хранения атрибутов, отличающиеся жизненным циклом и доступностью.

Page scope (область страницы) — атрибуты доступны только в пределах текущей JSP-страницы (и включаемых через `<jsp:include>` страниц в рамках той же страницы). Жизненный цикл совпадает с обработкой одного запроса одной JSP-страницей. Доступ через неявный объект `pageContext`: `pageContext.setAttribute("key", value)`. Наиболее ограниченная область.

Request scope (область запроса) — атрибуты доступны в течение обработки одного HTTP-запроса и во всех JSP/сервлетах, которым запрос был передан через `forward`. Хранится в объекте `HttpServletRequest`. Доступ: `request.setAttribute("key", value)` / `request.getAttribute("key")`. Уничтожается после отправки ответа клиенту.

Session scope (область сессии) — атрибуты доступны в течение всей HTTP-сессии пользователя (нескольких запросов). Хранится в объекте `HttpSession`. Доступ: `session.setAttribute("key", value)` / `session.getAttribute("key")`. Уничтожается при инвалидации сессии или истечении тайм-аута. Используется для пользовательских данных (корзина покупок, данные авторизации).

Application scope (область приложения) — атрибуты доступны всем

пользователям и всем сервлетам/JSP в пределах одного веб-приложения. Хранится в объекте ServletContext. Доступ: `application.setAttribute("key", value)` или `getServletContext().setAttribute(...)`. Живёт всё время работы приложения. Используется для разделяемых ресурсов: конфигурация, кеш, счётчики.

В EL-выражениях (`${name}`) поиск атрибута идёт по порядку: `page` → `request` → `session` → `application` — используется первый найденный.

11. Отличия `forward` и `redirect`

`forward` (серверный перенаправление, `RequestDispatcher.forward()`): запрос перенаправляется на другой ресурс (сервлет, JSP) внутри сервера, без участия клиента. Браузер не знает о перенаправлении — URL в адресной строке браузера не изменяется. Объекты `request` и `response` передаются без изменений — атрибуты запроса сохраняются. Только один цикл запрос-ответ. Выполняется быстрее, чем `redirect`. Работает только в пределах одного веб-приложения (нельзя `forward` на внешний URL). Используется, когда несколько компонентов обрабатывают один запрос: сервлет подготавливает данные, JSP их отображает:

Java

```
RequestDispatcher rd = request.getRequestDispatcher("/result.jsp");
```

```
rd.forward(request, response);
```

`redirect` (клиентское перенаправление, `HttpServletResponse.sendRedirect()`): сервер отправляет клиенту ответ со статусом 302 Found (или 301, 303, 307, 308) и заголовком `Location` с новым URL. Браузер получает этот ответ и автоматически делает новый запрос по указанному URL. URL в адресной строке браузера изменяется. Два цикла запрос-ответ (первый — с 302-ответом, второй — на новый URL). Атрибуты запроса (`setAttribute`) теряются между двумя запросами (нужно использовать сессию или `flash`-атрибуты). Может перенаправлять на любой URL, в том числе внешний. Используется для паттерна PRG (Post-Redirect-Get) — после обработки

POST-формы выполняется `redirect`, чтобы повторная загрузка страницы не вызывала повторную отправку формы:

Java

```
response.sendRedirect("/success-page");
```

Правило выбора: если нужно передать управление внутри приложения, сохранив данные запроса — `forward`. Если нужно изменить URL или перенаправить после POST — `redirect`.

12. Отличия `getAttribute()` от `getParameter()`

`getParameter(String name)` — получает значение параметра, переданного клиентом в HTTP-запросе. Для GET-запроса это параметры из строки запроса (query string): `?name=Alice&age=30`. Для POST-запроса с телом `application/x-www-form-urlencoded` — параметры из тела формы. Всегда возвращает `String` (или `null` если параметра нет). Параметры устанавливаются клиентом (браузером) и не могут быть установлены серверным кодом через этот механизм. `getParameterValues(name)` возвращает массив строк для параметра с несколькими значениями (например, несколько чекбоксов с одним именем).

`getAttribute(String name)` — получает атрибут, установленный серверным кодом через `setAttribute(String name, Object value)`. Атрибуты — это объекты произвольного типа (не только `String`), передаваемые между компонентами на стороне сервера. Используется для передачи данных от сервлета к JSP (через `forward`), или для хранения данных в сессии и контексте приложения. Клиент не может установить или изменить атрибуты напрямую.

Итого: `getParameter()` — данные от клиента (всегда `String`), `getAttribute()` — данные от сервера (любой тип `Object`).

13. Из чего состоит URL

URL (Uniform Resource Locator) — это адрес ресурса в интернете. Полная структура URL:

`scheme://userinfo@host:port/path?query#fragment`

Scheme (схема/протокол): определяет протокол для доступа к ресурсу. http, https, ftp, mailto, jdbc, file и другие. Обязательная часть.

Authority включает: userinfo (необязательно) — имя пользователя и пароль: user:password@; host — доменное имя (example.com) или IP-адрес (192.168.1.1); port (необязательно) — номер порта. Если не указан, используется порт по умолчанию (80 для HTTP, 443 для HTTPS, 3306 для MySQL).

Path (путь): иерархический путь к ресурсу. Начинается с /. Например: /api/v1/users/123.

Query String (строка запроса): начинается с ?. Содержит пары key=value, разделённые &. Используется для передачи параметров: ?search=java&page=2&sort=name. Специальные символы кодируются в URL-encoding (%20 — пробел).

Fragment (фрагмент/якорь): начинается с #. Указывает на конкретный раздел на странице. Не отправляется серверу — используется только браузером для прокрутки: #section2.

Пример	полного	URL:
<code>https://user:pass@api.example.com:8080/v1/users?role=admin&active=true#results</code>		

14. Порядок выполнения сервлет-фильтров

Фильтры (Filters) — это компоненты, перехватывающие запросы и ответы до/после их обработки сервлетом. Фильтры реализуют интерфейс

`javax.servlet.Filter` с методами `init()`, `doFilter()`, `destroy()`. В методе `doFilter()` фильтр может модифицировать запрос и ответ, а затем передать управление следующему элементу цепочки через `chain.doFilter(request, response)`.

Порядок выполнения фильтров образует цепочку (паттерн `Chain of Responsibility`). Фильтры выполняются в том порядке, в котором они зарегистрированы в `web.xml` или через аннотации `@WebFilter`. Важно: порядок аннотаций `@WebFilter` не гарантирован спецификацией — для гарантированного порядка нужно использовать `web.xml` или программную регистрацию.

Схема выполнения: Запрос от клиента → Filter1 (код до `chain.doFilter`) → Filter2 (код до `chain.doFilter`) → Filter3 (код до `chain.doFilter`) → Servlet (обработка) → Filter3 (код после `chain.doFilter`) → Filter2 (код после `chain.doFilter`) → Filter1 (код после `chain.doFilter`) → Ответ клиенту.

То есть фильтры выполняются как стек: при прохождении запроса — в прямом порядке, при формировании ответа — в обратном. Это позволяет фильтрам обрабатывать как запрос (до вызова `chain.doFilter`), так и ответ (после вызова). Типичные применения: аутентификация и авторизация, логирование, сжатие ответа, кодировка символов, CORS-заголовки, кеширование.

DOCKER

1. Суть Docker и какую проблему он решает

Проблема: классическая ситуация в разработке — "на моей машине работает, а на сервере нет". Причины: разные версии операционных систем, разные версии JDK, Python, Node.js, разные конфигурации, разные переменные окружения, разные версии библиотек. Настройка окружения занимает много времени и сопровождается ошибками, воспроизведение ошибок с продакшена сложно.

Docker — это платформа для разработки, доставки и запуска приложений в

контейнерах. Контейнер — это изолированная лёгковесная среда выполнения, которая содержит всё необходимое для запуска приложения: код, runtime, библиотеки, конфигурацию, переменные окружения. Контейнеры изолированы от хост-системы и друг от друга (через механизмы Linux: namespaces, cgroups, union file system).

Ключевые принципы: один и тот же образ (image) запускается одинаково на любой машине, где установлен Docker (Linux, Windows, macOS). "Собери один раз — запускай везде". Контейнер быстро запускается (секунды) в отличие от виртуальных машин (минуты). Контейнеры разделяют ядро хост-ОС (в отличие от VM, каждая из которых имеет своё ядро) — это делает их значительно легче.

Отличие от виртуальных машин: VM эмулирует полное железо и запускает гостевую ОС целиком. Docker использует ядро хост-ОС через namespaces и cgroups, изолируя только пространство имён процессов, файловую систему и сеть. Docker-контейнер занимает мегабайты, VM — гигабайты.

2. Структура Dockerfile

Dockerfile — это текстовый файл с инструкциями для сборки Docker-образа. Каждая инструкция создаёт новый слой образа.

Dockerfile

FROM — базовый образ, от которого строится наш образ. Обязательная первая инструкция.

```
FROM eclipse-temurin:21-jre-alpine
```

LABEL — метаданные образа (необязательно)

```
LABEL maintainer="dev@example.com" version="1.0"
```

ENV — переменные окружения внутри контейнера

```
ENV APP_HOME=/app APP_PORT=8080
```

WORKDIR — рабочая директория внутри контейнера. Создаётся если не существует.

WORKDIR \${APP_HOME}

COPY — копирует файлы из хост-системы в образ

Формат: COPY <источник_на_хосте> <назначение_в_образе>

COPY target/myapp.jar app.jar

ADD — похож на COPY, но дополнительно умеет распаковывать архивы и скачивать URL

Рекомендуется предпочитать COPY для простого копирования

RUN — выполняет команду при сборке образа (создаёт новый слой)

RUN apk add --no-cache curl && mkdir -p /app/logs

EXPOSE — документирует порт, который приложение слушает внутри контейнера

Не публикует порт на хост — только декларация намерения

EXPOSE \${APP_PORT}

VOLUME — создаёт точку монтирования для внешних данных

VOLUME /app/logs

USER — под каким пользователем запускать процесс (безопасность)

USER nobody

ENTRYPOINT — основная команда контейнера, не переопределяется при docker run

ENTRYPOINT ["java", "-jar"]

CMD — аргументы к ENTRYPOINT или команда по умолчанию (переопределяется при docker run)

CMD ["app.jar"]

Принципы хорошего Dockerfile: использовать минимальный базовый образ (alpine варианты); объединять RUN-команды через && для уменьшения слоёв; располагать редко изменяемые инструкции раньше (используется кеш слоёв); не запускать от root; использовать .dockerignore для исключения ненужных файлов при COPY.

3. Docker Compose: что это и зачем нужен

Docker Compose — это инструмент для определения и запуска многоконтейнерных Docker-приложений. Описание всех сервисов, их конфигурации, сетей и томов хранится в одном YAML-файле (docker-compose.yml). Одной командой docker-compose up поднимается всё приложение целиком.

Зачем нужен: реальные приложения редко состоят из одного контейнера. Типичная связка: Java-приложение + PostgreSQL + Redis + Nginx. Без Compose пришлось бы вручную запускать каждый контейнер с длинными командами и параметрами, настраивать сети между ними. Compose решает это декларативно.

YAML

```
version: '3.9'
```

```
services:
```

```
  app:
```

```
    build: .                # собрать из Dockerfile в текущей директории
```

```
    ports:
```

```
      - "8080:8080"        # HOST_PORT:CONTAINER_PORT
```

```
    environment:
```

```
      - DB_URL=jdbc:postgresql://db:5432/mydb
```

```
      - DB_PASSWORD=secret
```

```
    depends_on:
```

```
      db:
```

```

    condition: service_healthy # ждать, пока db не пройдёт healthcheck
networks:
  - backend

db:
  image: postgres:16-alpine
  volumes:
    - postgres_data:/var/lib/postgresql/data # именованный том для
персистентности
  environment:
    POSTGRES_DB: mydb
    POSTGRES_PASSWORD: secret
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - backend

volumes:
  postgres_data:

networks:
  backend:
    driver: bridge

```

Основные команды: `docker-compose up -d` (запустить в фоне), `docker-compose down` (остановить и удалить контейнеры), `docker-compose logs -f app` (логи сервиса), `docker-compose exec app bash` (войти в контейнер).

4. Слои (Layers) в Docker-образах

Образ (Image) в Docker — это набор слоёв (layers), каждый из которых представляет изменения файловой системы относительно предыдущего. Слои

являются неизменяемыми (immutable). Каждая инструкция в Dockerfile (FROM, RUN, COPY, ADD) создаёт новый слой. Слои используют union file system (OverlayFS в современных Linux) — они "накладываются" друг на друга, образуя единую файловую систему контейнера.

Какую проблему решают слои:

Кеширование сборки: Docker кеширует слои. Если слой не изменился (инструкция та же и входные данные те же) — Docker использует кешированный слой, не пересоздаёт его. Это резко ускоряет повторные сборки. Поэтому важно располагать редко изменяемые инструкции (установка зависимостей) раньше часто изменяемых (копирование кода).

Экономия места: слои разделяются между образами. Если несколько образов используют один и тот же базовый образ (FROM eclipse-temurin:21) — этот слой хранится на диске один раз. При скачивании образа загружаются только недостающие слои.

Ускорение передачи: при docker push/docker pull передаются только слои, которых нет на целевой машине.

Хорошая практика: скопировать pom.xml и скачать зависимости (mvn dependency:go-offline) в отдельном слое до копирования исходного кода. Тогда при изменении только исходников пересоздаётся только последний слой, а слой с зависимостями берётся из кеша.

5. Типы сетей в Docker Compose

Bridge (по умолчанию): виртуальная сеть, создаваемая Docker. Контейнеры в одной bridge-сети могут общаться друг с другом по имени сервиса (DNS-разрешение внутри сети: ping db найдёт контейнер с именем db). Изолированы от хоста (кроме опубликованных портов). driver: bridge — стандартный выбор для большинства приложений.

Host: контейнер разделяет сетевой стек хост-системы напрямую. Нет сетевой изоляции. Порты контейнера напрямую доступны на хосте без маппинга. Используется для максимальной сетевой производительности. Не работает нативно на Docker Desktop для Mac/Windows.

None: контейнер не имеет доступа к сети вообще. Полная изоляция. Используется для batch-задач, не требующих сети.

Overlay: для распределённых многохостовых окружений (Docker Swarm). Позволяет контейнерам на разных физических серверах общаться как в одной сети.

Macvlan: контейнер получает собственный MAC-адрес и выглядит для физической сети как отдельное устройство. Используется для интеграции с существующей сетевой инфраструктурой.

В Compose можно определить несколько сетей и подключать разные сервисы к разным сетям, обеспечивая изоляцию: например, frontend-сеть (nginx + app) и backend-сеть (app + db), при этом nginx не имеет прямого доступа к db.

Кейс: PostgreSQL в Docker не доступен на порту 5432

Диагностика пошагово:

Шаг 1 — проверить, что контейнер запущен: `docker ps` — убедиться что контейнер postgres присутствует в списке и статус Up. Если нет — `docker ps -a` (все, включая остановленные), `docker logs <container_name>` — посмотреть причину остановки.

Шаг 2 — проверить маппинг портов: `docker ps` показывает колонку PORTS. Должно быть что-то вроде `0.0.0.0:5432->5432/tcp`. Если там только `5432/tcp` без маппинга на хост — порт не опубликован. В `docker-compose.yml` должно быть:

YAML

ports:

- "5432:5432" # HOST:CONTAINER

Шаг 3 — проверить, что PostgreSQL запустился внутри контейнера: `docker exec -it <container> pg_isready` или `docker exec -it <container> psql -U postgres`. Если PostgreSQL не стартовал (ошибка инициализации БД, неверные переменные окружения), он не будет принимать соединения.

Шаг 4 — проверить, нет ли конфликта портов на хосте: `ss -tlnp | grep 5432` (Linux) или `netstat -an | grep 5432` — вдруг на хосте уже запущен PostgreSQL и занял порт 5432. В таком случае изменить маппинг: "5433:5432".

Шаг 5 — проверить firewall/сетевые настройки: `iptables -L` или `ufw status`. Файрволл может блокировать порт.

Шаг 6 — проверить переменные окружения PostgreSQL: контейнер не запустится без `POSTGRES_PASSWORD`. Посмотреть логи: `docker logs <container>`.

GIT

1. Что такое Git и отличия от других систем контроля версий

Git — это распределённая система контроля версий (DVCS — Distributed Version Control System), созданная Линусом Торвальдсом в 2005 году для разработки ядра Linux. Git отслеживает историю изменений файлов, позволяет нескольким разработчикам работать над одним проектом, создавать ветки для параллельной разработки и объединять изменения.

Отличия от централизованных систем (SVN, CVS, Perforce):

В централизованных системах (SVN) существует один центральный репозиторий. Разработчики делают `checkout` отдельных файлов с сервера и `commit` обратно. Работа без подключения к серверу ограничена. История

хранится только на сервере. Потеря сервера означает потерю истории.

В Git каждый разработчик имеет полную копию всего репозитория со всей историей на своём компьютере. Большинство операций (`commit`, `log`, `diff`, `branch`, `merge`) выполняются локально, без сети — быстро и всегда доступно. Работа с сетью (`push`, `pull`, `fetch`) нужна только для синхронизации с другими. Нет единой точки отказа.

Другие ключевые особенности Git: хранение данных как снимков (`snapshots`) — Git хранит состояние всех файлов при каждом коммите, а не дельты изменений; целостность через SHA-1/SHA-256 — каждый объект идентифицируется хэшем; дешёвые ветки (ветка — просто указатель на коммит, создание занимает миллисекунды).

2. Локальный и удалённый репозиторий

Локальный репозиторий — полная копия репозитория на машине разработчика. Находится в скрытой директории `.git` в корне проекта. Содержит всю историю коммитов, все ветки, теги, конфигурацию. Все операции с локальным репозиторием выполняются без сети.

Удалённый репозиторий (`remote`) — репозиторий на сервере (GitHub, GitLab, Bitbucket, собственный сервер). Служит как центральное место для обмена кодом между разработчиками. Команда `git remote -v` показывает настроенные удалённые репозитории. По соглашению основной удалённый репозиторий называется `origin`. Можно иметь несколько удалённых репозиториев.

`git clone URL` — создаёт локальную копию удалённого репозитория. `git push origin main` — отправляет изменения из локального в удалённый. `git fetch origin` — загружает изменения из удалённого, но не сливает в текущую ветку. `git pull` — `fetch` + `merge` (или `rebase`).

3. HEAD и его связь с текущей веткой

HEAD — это специальный указатель (`pointer/reference`) в Git, который

указывает на текущую позицию в истории репозитория. Обычно HEAD указывает на текущую ветку (branch), а ветка в свою очередь указывает на последний коммит этой ветки.

В файловой системе HEAD хранится в файле `.git/HEAD` и содержит что-то вроде `ref: refs/heads/main` — это символическая ссылка на ветку `main`. Когда вы делаете новый коммит, текущая ветка перемещается вперёд на новый коммит, и HEAD следует за ней.

Detached HEAD (отсоединённый HEAD) — состояние, когда HEAD указывает непосредственно на коммит, а не на ветку. Возникает при `git checkout <commit-hash>` или `git checkout v1.0 (tag)`. В этом состоянии новые коммиты создаются, но ни одна ветка на них не указывает — они могут быть потеряны (garbage collected). Для сохранения изменений нужно создать новую ветку: `git checkout -b new-branch`.

4. Устройство коммитов, веток, merge

Коммит (commit) — это снимок состояния репозитория в определённый момент. Каждый коммит содержит: хэш SHA-1 (40-значная hex-строка, уникальный идентификатор: `a1b2c3d4...`), tree (ссылка на объект-дерево, представляющий состояние файловой системы), parent(s) (ссылки на родительские коммиты — у первого коммита нет родителя, у merge-коммита — два), автор (имя, email, дата), committer (кто создал коммит, может отличаться от автора при rebase), сообщение коммита.

Объекты Git хранятся в `.git/objects/` как сжатые файлы. Git хранит четыре типа объектов: blob (содержимое файла), tree (структура директории), commit (снимок с метаданными), tag (аннотированный тег).

Ветка (branch) — это просто текстовый файл в `.git/refs/heads/`, содержащий хэш коммита, на который указывает ветка. Создание ветки — это запись 40 символов в файл. Переключение ветки — обновление HEAD и файлов рабочего каталога. Ветки — это "named pointers" на коммиты.

Merge — объединение изменений из двух веток. Fast-forward merge — если ветка feature является прямым потомком main (main не получал новых коммитов) — main просто "перематывается" вперёд до feature. История линейна, merge-коммит не создаётся. Three-way merge — если обе ветки имели изменения — Git находит общего предка (base), сравнивает три состояния и создаёт новый merge-коммит с двумя родителями. История нелинейна, видно слияние.

5. Отличия fetch и pull

`git fetch [remote]` — загружает изменения из удалённого репозитория в локальный, но не изменяет рабочие файлы и текущую ветку. Обновляются только remote-tracking ветки (`origin/main`, `origin/feature`). После fetch можно посмотреть что изменилось: `git log origin/main..main`, `git diff origin/main main`. Fetch "безопасен" — ничего не сломает. После fetch можно вручную решить, как интегрировать изменения (`merge` или `rebase`).

`git pull` — это `git fetch` + `git merge` (или `git fetch` + `git rebase` при `--rebase` флаге или конфигурации `pull.rebase=true`). Загружает изменения и сразу интегрирует их в текущую ветку. Если есть конфликты — их нужно разрешить. `git pull --rebase` — более чистый вариант, не создаёт лишних merge-коммитов.

Рекомендация: `fetch` + ревью изменений + `merge/rebase` — более контролируемый подход. `pull` — удобен когда доверяете изменениям и хотите быстро синхронизироваться.

6. Rebase: что это и когда использовать

`git rebase` — это операция "перебазирования" коммитов. Rebase берёт коммиты из одной ветки и переприменяет их поверх другой ветки. При этом для каждого перемещаемого коммита создаётся новый коммит (с новым хэшем!) — история переписывается.

Пример: у вас есть ветка `feature`, ответвившаяся от `main` три коммита назад. За это время в `main` появились новые коммиты. `git rebase main` переместит ваши `feature`-коммиты так, будто они были сделаны поверх последнего коммита `main`. Результат — линейная история без `merge`-коммитов.

Когда использовать `rebase`:

Перед мержем `feature`-ветки в `main` — чтобы получить чистую линейную историю.

`git pull --rebase` — для обновления своей ветки без лишних `merge`-коммитов.

Интерактивный `rebase` `git rebase -i HEAD~N` — для редактирования `N` последних коммитов: изменение порядка, `squash` (объединение нескольких коммитов в один), редактирование сообщений, удаление коммитов.

Когда НЕ использовать: золотое правило `rebase` — не ребейзить публичные (общие) ветки. Если ветку `main` или `develop` уже скачали другие разработчики, `rebase` переписывает историю — у других возникнут проблемы при синхронизации. `Rebase` безопасен для локальных или личных `feature`-веток.

7. Откат изменений: `revert` и `reset`

`git revert <commit>` — создаёт новый коммит, который отменяет изменения указанного коммита. История не переписывается — старый коммит остаётся, добавляется новый "обратный" коммит. Безопасно для публичных веток — не нарушает историю других разработчиков. Используется, когда нужно отменить уже опубликованные изменения.

`git reset` — перемещает указатель ветки `HEAD` на другой коммит. Переписывает историю — коммиты "теряются" (но объекты в `Git` хранятся ещё некоторое время и доступны через `git reflog`). Три режима:

`git reset --soft HEAD~N` — `HEAD` перемещается назад на `N` коммитов. Изменения из этих коммитов остаются в индексе (`staging area`) — готовы к новому коммиту. Полезно для переделки последних коммитов.

`git reset --mixed HEAD~N` (по умолчанию) — HEAD перемещается назад, изменения остаются в рабочем каталоге, но удаляются из индекса. Нужно снова `git add`.

`git reset --hard HEAD~N` — HEAD перемещается назад, изменения безвозвратно удаляются из индекса и рабочего каталога. Осторожно — несохранённые изменения теряются.

`git restore <file>` (Git 2.23+) — отменить изменения в рабочем каталоге для конкретного файла. `git restore --staged <file>` — убрать файл из индекса. Эти команды заменяют части функциональности `git checkout`.

8. Решение конфликтов при слиянии

Конфликт слияния возникает, когда два разработчика изменили один и тот же участок одного файла по-разному, и Git не может автоматически определить, какую версию оставить.

При возникновении конфликта Git помечает конфликтующие файлы маркерами:

text

```
<<<<<<< HEAD
```

```
// ваши изменения (текущая ветка)
```

```
public void process() { doNewWay(); }
```

```
=====
```

```
// входящие изменения (мержимая ветка)
```

```
public void process() { doOtherWay(); }
```

```
>>>>>>> feature-branch
```

Процесс разрешения конфликта:

Шаг 1 — увидеть конфликтующие файлы: `git status` — они помечены как `both`

modified.

Шаг 2 — открыть файл и разрешить конфликт: выбрать нужный вариант (или объединить оба), удалить маркеры <<<<<<<, =====, >>>>>>>. Современные IDE (IntelliJ IDEA, VS Code) показывают конфликты визуально с кнопками "Accept Current", "Accept Incoming", "Accept Both".

Шаг 3 — после разрешения: `git add <resolved-file>`.

Шаг 4 — завершить слияние: `git commit` (для merge) или `git rebase --continue` (для rebase).

Для отмены merge при сложном конфликте: `git merge --abort`. Для rebase: `git rebase --abort`. Также полезны инструменты `git mergetool` (запускает внешний merge-инструмент) и `git log --merge` (показывает коммиты, вызвавшие конфликт).

Предотвращение конфликтов: чаще синхронизироваться с основной веткой, делать маленькие фокусированные коммиты, коммуницировать с командой о параллельных изменениях.

SPRING

1. Bean Lifecycle: жизненный цикл бина

Bean в Spring — это объект, управляемый Spring IoC-контейнером. Жизненный цикл бина — это последовательность этапов от создания до уничтожения.

Полная последовательность: 1) Загрузка определений бинов (BeanDefinition) из конфигурации (XML, аннотации, Java Config) → 2) BeanFactoryPostProcessor — постобработка метаданных бинов до их создания (например, PropertySourcesPlaceholderConfigurer подставляет `${property}`) → 3) Создание экземпляра (new), включая внедрение конструкторных зависимостей → 4) Внедрение зависимостей через сеттеры и поля

(@Autowired) → 5) Обработка Aware-интерфейсов (BeanNameAware.setBeanName(), BeanFactoryAware.setBeanFactory(), ApplicationContextAware.setApplicationContext()) → 6) BeanPostProcessor.postProcessBeforeInitialization() — для всех бинов перед инициализацией → 7) @PostConstruct-метод (аннотированный JSR-250) → 8) InitializingBean.afterPropertiesSet() → 9) Метод init-method из конфигурации → 10) BeanPostProcessor.postProcessAfterInitialization() — здесь Spring создаёт AOP-прокси → 11) Бин готов к использованию → ... → 12) @PreDestroy-метод → 13) DisposableBean.destroy() → 14) Метод destroy-method из конфигурации.

На этапе 10 Spring может заменить оригинальный бин на прокси-объект (для @Transactional, @Async, @Cacheable), поэтому из контекста Spring всегда возвращается прокси, а не оригинальный объект.

2. Способы объявления бинов: плюсы и минусы

XML-конфигурация (applicationContext.xml): <bean id="..." class="..." />. Плюсы: чёткое разделение конфигурации и кода, удобно для сторонних классов. Минусы: многословность, нет проверки типов на этапе компиляции, сложно поддерживать при большом количестве бинов. Сегодня практически не используется.

Аннотация @Component и производные (@Service, @Repository, @Controller) + сканирование компонентов (@ComponentScan): Spring автоматически обнаруживает и регистрирует классы с этими аннотациями. Плюсы: минимальный бойлерплейт, автоматическое обнаружение, интуитивно. Минусы: нельзя использовать для сторонних классов (нет доступа к исходному коду), иногда неясно, какие бины создаются.

Java Configuration (@Configuration + @Bean): класс конфигурации с методами, помеченными @Bean, которые создают и возвращают объекты. Плюсы: полный контроль над созданием бина (параметры конструктора, сложная логика инициализации), проверка типов компилятором, легко

работать со сторонними классами, легко понять что создаётся. Минусы: больше кода, чем с аннотациями.

Программная регистрация (`GenericApplicationContext.registerBean()`): регистрация бинов программно через API. Плюсы: максимальная гибкость, условное создание. Минусы: редко нужно, усложняет понимание.

3. Bean Scopes: виды и жизненный цикл

`singleton` (по умолчанию) — один экземпляр бина на весь `ApplicationContext`. Создаётся при старте контейнера (`eager`), уничтожается при закрытии. Используется для `stateless` сервисов, DAO, репозиториев.

`prototype` — новый экземпляр создаётся при каждом запросе бина (каждый `getBean()` или каждое внедрение). Spring создаёт бин, внедряет зависимости, вызывает `@PostConstruct`, но не управляет его уничтожением — `@PreDestroy` не вызывается. Клиент отвечает за освобождение ресурсов. Используется для `stateful` объектов.

`request` — один экземпляр на HTTP-запрос. Создаётся при начале запроса, уничтожается при его завершении. Только для веб-приложений. Используется для объектов, специфичных для запроса.

`session` — один экземпляр на HTTP-сессию. Живёт пока жива сессия. Только для веб-приложений. Используется для корзины покупок, пользовательских настроек.

`application` — один экземпляр на `ServletContext` (на всё веб-приложение). Практически эквивалентен `singleton` для веб-приложений.

`websocket` — один экземпляр на `WebSocket`-сессию.

Проблема внедрения бина с меньшим `scope` в бин с большим `scope` (например, `prototype` в `singleton`): `singleton` создаётся один раз, и зависимость

тоже будет одна. Решение: `@Lookup`-метод, `ObjectProvider<T>`, `ApplicationContext.getBean()` или `scoped proxy` (`@Scope(proxyMode = ScopedProxyMode.TARGET_CLASS)`).

4. Инструменты создания прокси. **Dynamic Proxy vs CGLib**

Spring активно использует прокси-объекты для реализации AOP (аспектно-ориентированного программирования), транзакций, безопасности, кеширования.

JDK Dynamic Proxy (`java.lang.reflect.Proxy`): работает только для классов, реализующих интерфейс. Создаётся прокси-объект, реализующий те же интерфейсы. Все вызовы перехватываются `InvocationHandler`. Требуется интерфейс — если бин не реализует интерфейс, JDK Proxy не применим. Встроен в JDK, без внешних зависимостей.

CGLib (Code Generation Library): работает для любых классов, включая не имеющие интерфейсов. Создаётся подкласс прогируемого класса через байт-код генерацию. Переопределяет методы, вставляя перехватчик. Ограничения: не может проксировать `final`-классы и `final`-методы (нельзя переопределить). Spring Boot по умолчанию использует CGLib начиная с Spring 4, даже если есть интерфейс.

Spring по умолчанию выбирает: если бин реализует интерфейс — раньше использовался JDK Proxy, сейчас Spring Boot настроен на CGLib по умолчанию (`spring.aop.proxy-target-class=true`). Можно принудительно использовать JDK Proxy через `@EnableAspectJAutoProxy(proxyTargetClass = false)`.

Важное следствие: если использовать CGLib-прокси, нельзя делать `@Transactional` методы `private` или `final` — Spring не сможет их перехватить. Вызов `@Transactional`-метода из того же класса тоже не будет перехвачен (self-invocation problem) — потому что вызов идёт не через прокси, а напрямую через `this`.

5. Spring MVC vs Spring Boot

Spring MVC — это веб-фреймворк поверх Servlet API, часть Spring Framework. Предоставляет паттерн MVC для построения веб-приложений: DispatcherServlet — фронт-контроллер, принимающий все запросы; @Controller / @RestController — обработчики запросов; @RequestMapping — маппинг URL на методы; ViewResolver — выбор шаблона для рендеринга; ModelAndView — модель данных и имя шаблона. Spring MVC требует ручной настройки: конфигурация DispatcherServlet, подключение зависимостей, настройка сканирования компонентов, конфигурация ViewResolver и другого.

Spring Boot — это фреймворк поверх Spring Framework (включая Spring MVC), который реализует принцип "convention over configuration" (соглашение важнее конфигурации). Spring Boot предоставляет: автоконфигурацию (@SpringBootApplication включает @EnableAutoConfiguration) — Spring Boot анализирует classpath и автоматически настраивает компоненты; встроенный сервер (Tomcat, Jetty, Undertow) — не нужен внешний сервер, приложение запускается как java -jar app.jar; starters — наборы зависимостей для типичных сценариев (spring-boot-starter-web, spring-boot-starter-data-jpa); application.properties/application.yml для централизованной конфигурации; Actuator для мониторинга.

Spring Boot не заменяет Spring MVC — он использует его внутри, просто устраняет необходимость в boilerplate-конфигурации.

6. @Component vs @Service vs @Repository

Все три аннотации являются специализациями @Component и технически делают одно и то же — помечают класс как Spring-бин для автообнаружения. Разница семантическая и функциональная.

@Component — общий стереотип для любого Spring-управляемого

компонента. Используется когда другие специализации не подходят (утилитные классы, аспекты и другие).

`@Service` — семантически обозначает бизнес-логику, сервисный слой. Не добавляет дополнительного поведения по сравнению с `@Component`. Помогает структурировать код и ясно выражать намерения.

`@Repository` — семантически обозначает слой доступа к данным (DAO). Добавляет реальное функциональное отличие: активирует трансляцию исключений — Spring автоматически перехватывает специфические исключения хранилища (например, `PersistenceException` от Hibernate, `SQLException` от JDBC) и оборачивает их в иерархию `Spring DataAccessException`. Это позволяет бизнес-логике не зависеть от конкретной технологии хранения.

`@Controller` — для Spring MVC, обозначает обработчик веб-запросов.
`@RestController` = `@Controller` + `@ResponseBody`.

7. Нативный запрос в Spring Data JPA

Spring Data JPA предоставляет несколько способов выполнения нативных SQL-запросов.

`@Query` с `nativeQuery = true` на методе репозитория:

Java

```
@Query(value = "SELECT * FROM users WHERE age > :age AND active = true", nativeQuery = true)
```

```
List<User> findActiveUsersOlderThan(@Param("age") int age);
```

`@Query` с нативным запросом для пагинации требует отдельного запроса для count:

Java

```
@Query(value = "SELECT * FROM users WHERE age > :age",  
      countQuery = "SELECT count(*) FROM users WHERE age > :age",  
      nativeQuery = true)  
Page<User> findUsersOlderThan(@Param("age") int age, Pageable pageable);  
EntityManager.createNativeQuery() — через JPA EntityManager напрямую:
```

Java

```
@PersistenceContext EntityManager em;
```

```
List<Object[]> results = em.createNativeQuery("SELECT id, name FROM users  
WHERE age > ?1")  
    .setParameter(1, 30)  
    .getResultList();
```

JdbcTemplate из Spring JDBC — для полного контроля над SQL без JPA:

Java

```
List<User> users = jdbcTemplate.query("SELECT * FROM users WHERE age >  
?",  
    (rs, rowNum) -> new User(rs.getLong("id"), rs.getString("name")),  
    30);
```

8. @Transactional: как работает и вложенные транзакции

@Transactional работает через AOP-прокси. Когда Spring создаёт бин с @Transactional-методами, он оборачивает его в прокси. При вызове транзакционного метода прокси перехватывает вызов, начинает транзакцию (или присоединяется к существующей), выполняет метод, и в зависимости от результата (нормальное завершение или исключение) коммитит или откатывает транзакцию.

Ключевые параметры: propagation — поведение при наличии/отсутствии активной транзакции, isolation — уровень изоляции, readOnly — оптимизация

для read-only операций, rollbackFor — при каких исключениях откатывать (по умолчанию unchecked exceptions), timeout — таймаут транзакции.

Что происходит при вызове @Transactional-метода из другого @Transactional-метода:

Поведение определяется параметром propagation. По умолчанию — REQUIRED: если транзакция уже существует — метод выполняется в рамках существующей транзакции. Если нет — создаётся новая.

Пример: ServiceA.methodA() (REQUIRED) вызывает ServiceB.methodB() (REQUIRED). Обе работают в одной транзакции — если methodB() откатит транзакцию (или выбросит unchecked exception), это откатит всю транзакцию, включая изменения methodA().

Другие варианты propagation: REQUIRES_NEW — всегда создаёт новую транзакцию (независимую от внешней); NESTED — вложенная транзакция с savepoint; SUPPORTS — использует существующую или выполняется без транзакции; NOT_SUPPORTED — всегда без транзакции; MANDATORY — обязательно должна быть активная транзакция; NEVER — нельзя вызывать в транзакции.

Важная проблема self-invocation: если метод класса вызывает @Transactional-метод того же класса через this, прокси не перехватывает вызов, транзакция не начинается. Решение: внедрить сам себя через @Autowired или использовать AopContext.currentProxy().

9. Популярные Spring Boot Starters

Spring Boot Starters — это Maven/Gradle зависимости, которые автоматически подтягивают все необходимые библиотеки для определённой функциональности и настраивают AutoConfiguration.

spring-boot-starter-web — Spring MVC + встроенный Tomcat + Jackson (JSON)

для REST API и веб-приложений.

spring-boot-starter-data-jpa — Spring Data JPA + Hibernate + HikariCP (пул соединений).

spring-boot-starter-security — Spring Security для аутентификации и авторизации.

spring-boot-starter-test — JUnit 5 + Mockito + AssertJ + Spring Test для тестирования.

spring-boot-starter-data-redis — Spring Data Redis + Lettuce/Jedis для работы с Redis.

spring-boot-starter-actuator — endpoint'ы для мониторинга (/health, /metrics, /info).

spring-boot-starter-validation — Bean Validation (Hibernate Validator) для @Valid, @NotNull.

spring-boot-starter-aop — поддержка AOP через AspectJ.

spring-boot-starter-thymeleaf — шаблонизатор Thymeleaf для серверного рендеринга.

spring-boot-starter-amqp — Spring AMQP + RabbitMQ.

spring-boot-starter-kafka — Spring Kafka для работы с Apache Kafka.

10. DI и IoC

IoC (Inversion of Control — Инверсия управления) — это принцип проектирования, при котором контроль над созданием объектов и управлением зависимостями передаётся фреймворку (контейнеру), а не

прикладному коду. Вместо того чтобы объект сам создавал нужные ему зависимости (`new SomeService()`), контейнер создаёт объекты и предоставляет их тому, кто нуждается. "Голливудский принцип" — "Не звоните нам, мы позвоним вам".

DI (Dependency Injection — Внедрение зависимостей) — это конкретная реализация принципа IoC. DI — это паттерн, при котором зависимости объекта передаются ему извне (через конструктор, сеттер или поле), а не создаются внутри.

Виды внедрения: Constructor Injection — через конструктор (рекомендуется: зависимости явны, объект всегда полностью инициализирован, можно сделать поля `final`). Setter Injection — через сеттер (для опциональных зависимостей). Field Injection — через `@Autowired` на поле (не рекомендуется: нарушает инкапсуляцию, сложнее тестировать вручную без Spring-контейнера).

В Spring IoC-контейнер — это `ApplicationContext`. Он читает конфигурацию, создаёт бины, разрешает зависимости и внедряет их. Преимущества DI: слабая связность (`loose coupling`), лёгкость тестирования (можно подставлять моки), гибкость (легко заменить реализацию).

REST

1. Принцип ресурсной ориентации. Примеры ресурсов

Ресурсная ориентация (Resource-Oriented Architecture) — ключевой принцип REST (Representational State Transfer), предложенного Роем Филдингом в 2000 году. Всё в системе является ресурсом — именованной сущностью, к которой можно обратиться. API строится вокруг ресурсов, а не вокруг действий.

Ресурс — это любая сущность предметной области: пользователь, заказ, продукт, статья, комментарий. Ресурс идентифицируется уникальным URI и может иметь несколько представлений (JSON, XML, HTML).

Примеры ресурсов и их URI:

Коллекция пользователей: GET /users

Конкретный пользователь: GET /users/123

Заказы конкретного пользователя: GET /users/123/orders

Конкретный заказ: GET /users/123/orders/456

Продукты: GET /products, GET /products?category=electronics&page=1

Что не является ресурсно-ориентированным подходом — RPC-стиль: GET /getUser?id=123, POST /createOrder, POST /deleteUser. В REST нет глаголов в URI — действие определяется HTTP-методом, а не путём.

2. Уникальные URI для каждого ресурса

Каждый ресурс должен иметь уникальный идентификатор — URI (Uniform Resource Identifier). Это фундаментальный принцип REST, обеспечивающий адресуемость (addressability) ресурсов.

Почему это важно: если два разных ресурса имеют одинаковый URI — возникает неоднозначность: какой ресурс клиент хотел получить? Уникальность URI позволяет кешировать ответы — кеш знает, что /users/123 всегда возвращает данные конкретного пользователя. Уникальные URI делают API самодокументируемым и предсказуемым. Клиент может сохранить URI ресурса (закладка) и обратиться к нему позже — это называется bookmarkability.

Влияние на взаимодействие клиента и сервера: клиент оперирует URI как "именами" ресурсов. Зная URI /orders/789, клиент знает как получить, обновить или удалить этот заказ — без дополнительного знания о внутренней реализации сервера. Сервер не хранит состояние клиента (statelessness) — URI в запросе содержит всю информацию, необходимую для идентификации ресурса.

Принципы построения URI: использовать существительные во

множественном числе (/users, /orders), иерархическая структура для вложенных ресурсов (/users/123/orders), slug или id для идентификации (/articles/my-first-article или /articles/456), строчные буквы и дефисы (не camelCase и не underscore для URL).

3. HTTP-методы и CRUD-операции

REST использует стандартные HTTP-методы для выражения операций над ресурсами, что делает API интуитивным.

GET → Read (Чтение): получение представления ресурса. Безопасный (не изменяет состояние) и идиempотентный. GET /users — список пользователей. GET /users/123 — конкретный пользователь. Ответ: 200 OK + тело с данными, 404 если не найден.

POST → Create (Создание): создание нового ресурса. Не идиempотентный (повторный вызов создаёт ещё один ресурс). POST /users с телом {"name": "Alice", "email": "alice@example.com"}. Ответ: 201 Created + заголовок Location: /users/124 с URI нового ресурса.

PUT → Update/Replace (Полное обновление): полное замещение ресурса. Идиempотентный. PUT /users/123 с полным телом ресурса. Ответ: 200 OK или 204 No Content.

PATCH → Update/Modify (Частичное обновление): частичное изменение ресурса (только изменяемые поля). PATCH /users/123 с {"email": "newemail@example.com"}. Ответ: 200 OK или 204 No Content.

DELETE → Delete (Удаление): удаление ресурса. Идиempотентный. DELETE /users/123. Ответ: 204 No Content или 200 OK.

4. Разница между PUT и PATCH

PUT — полное замещение ресурса. Клиент должен передать полное представление ресурса. Если какое-то поле не передано — оно будет

сброшено в значение по умолчанию или удалено. PUT идемпотентен: многократное выполнение одинакового запроса приводит к одному результату. Применяется когда клиент знает полное состояние ресурса и хочет его полностью заменить. Например, обновление профиля пользователя целиком.

PATCH — частичное обновление. Клиент передаёт только те поля, которые хочет изменить. Остальные поля остаются нетронутыми. PATCH может быть или не быть идемпотентным — зависит от реализации (например, {"views": "+1"} — не идемпотентное PATCH-обновление). Применяется когда нужно изменить одно или несколько полей без передачи всего ресурса. Экономит трафик. Например, изменение только email пользователя.

Выбор: используйте PUT, когда клиент управляет полным состоянием ресурса (форма редактирования, где все поля заполнены). Используйте PATCH для точечных обновлений (включить/выключить флаг, изменить одно поле). В реальных API PATCH используется гораздо чаще, чем PUT, из-за удобства.

5. Statelessness (Безсостояниость) в REST

Statelessness — каждый HTTP-запрос к REST API должен содержать всю информацию, необходимую серверу для его обработки. Сервер не хранит никакого состояния клиента между запросами. Каждый запрос — самодостаточная единица.

Что это означает на практике: нет серверных сессий (HttpSession) для хранения состояния авторизации — вместо этого каждый запрос содержит токен (Bearer JWT в заголовке Authorization). Параметры пагинации, фильтрации — в URI (?page=2&size=20). Контекст запроса — в самом запросе (не "помни кто я из предыдущего запроса").

Преимущества statelessness:

Масштабируемость — любой сервер в кластере может обработать любой запрос, не нуждаясь в данных о предыдущих запросах этого клиента. Нет проблемы sticky sessions.

Надёжность — если сервер упадёт, клиент просто повторит запрос к другому серверу — не нужно восстанавливать состояние сессии.

Простота — сервер не управляет состоянием множества клиентов, нет утечек памяти из-за забытых сессий.

Кешируемость — ответы легче кешировать, потому что запрос полностью самоописывается.

Компромисс: каждый запрос может быть немного "тяжелее" (нужно передавать токен аутентификации), но это компенсируется преимуществами в масштабировании.

6. HTTP-статус коды

Сервер сообщает о результате обработки через HTTP-статус код в ответе.

2xx — Успех:

200 OK — стандартный успешный ответ (для GET, PUT, PATCH, DELETE с телом).

201 Created — ресурс успешно создан (для POST). Заголовок Location указывает URI нового ресурса.

204 No Content — успех, но тело ответа пустое (для DELETE, PUT/PATCH без тела ответа).

202 Accepted — запрос принят для асинхронной обработки.

3xx — Перенаправление:

301 Moved Permanently — ресурс переехал навсегда. 302 Found — временный редирект.

304 Not Modified — для кеширования: ресурс не изменился, клиент может

использовать кешированную версию.

4xx — Ошибка клиента:

400 Bad Request — некорректный запрос (невалидный JSON, отсутствующие обязательные поля).

401 Unauthorized — требуется аутентификация (не передан или невалидный токен).

403 Forbidden — аутентифицирован, но нет прав доступа.

404 Not Found — ресурс не найден.

405 Method Not Allowed — метод не поддерживается для этого URI.

409 Conflict — конфликт (например, создание дублирующей записи).

422 Unprocessable Entity — запрос синтаксически корректен, но семантически невалиден (ошибки валидации).

429 Too Many Requests — rate limiting.

5xx — Ошибка сервера:

500 Internal Server Error — непредвиденная ошибка на сервере.

502 Bad Gateway — проблема с вышестоящим сервером (прокси).

503 Service Unavailable — сервер временно недоступен (перегрузка, обслуживание).

7. Разделение клиентской и серверной логики. Масштабируемость

Принцип Client-Server — один из шести ограничений (constraints) REST-архитектуры. Клиент и сервер разделены чётким интерфейсом (API). Клиент отвечает за пользовательский интерфейс и пользовательское состояние. Сервер отвечает за хранение данных и бизнес-логику.

Почему это важно для масштабируемости:

Независимое масштабирование — серверная часть может масштабироваться горизонтально (добавление серверов) независимо от клиентов. Клиентов может быть миллионы (браузеры, мобильные приложения) — сервер не знает о них ничего, кроме того, что содержится в запросе.

Независимая разработка — frontend-команда разрабатывает UI независимо от backend-команды, договорившись об API-контракте. Можно менять реализацию на сервере (переписать на другой язык, сменить СУБД), не затрагивая клиентов — пока API-контракт соблюдается. Несколько разных клиентов (веб, iOS, Android) используют один и тот же API.

Портабельность клиента — клиент не привязан к конкретному серверу. Клиент работает с любым сервером, реализующим тот же API (полезно для тестирования, mock-серверов).

Упрощение — каждая сторона может быть проще, фокусируясь на своей ответственности. REST API становится "контрактом" между системами — это основа микросервисной архитектуры и интеграции.

Java Core & More — Полные Ответы

JAVA CORE — ООП

Принципы ООП

Объектно-ориентированное программирование строится на четырёх фундаментальных принципах, каждый из которых решает конкретную архитектурную задачу.

Инкапсуляция — объединение данных и методов, работающих с этими данными, внутри одного класса, с одновременным сокрытием внутренней реализации от внешнего мира. Поля класса делаются `private`, доступ к ним предоставляется через `public` геттеры и сеттеры. Это позволяет изменять внутреннюю реализацию класса без изменения кода, который его использует, и обеспечивает контроль над состоянием объекта (валидация в сеттерах).

Наследование — механизм, позволяющий создавать новые классы на основе существующих, наследуя их поля и методы. Подкласс является

специализацией суперкласса (отношение "является" — is-a). В Java поддерживается только одиночное наследование классов через `extends`, но множественное наследование интерфейсов через `implements`. Наследование позволяет переиспользовать код и строить иерархии типов.

Полиморфизм — способность объекта принимать много форм. Один и тот же код может работать с объектами разных типов. Подробнее — в разделе "Полиморфизм, виды" ниже.

Абстракция — выделение существенных характеристик объекта и сокрытие несущественных деталей. Позволяет работать на высоком уровне концепций. В Java реализуется через абстрактные классы и интерфейсы. Пример: интерфейс `List` абстрагирует понятие упорядоченной коллекции, скрывая детали того, реализована она на массиве или связном списке.

Abstract class vs Interface

Это один из самых частых вопросов, и важно понимать не только синтаксические различия, но и концептуальные.

Абстрактный класс — это класс, объявленный с ключевым словом `abstract`. Он может содержать как абстрактные методы (без реализации), так и конкретные методы (с реализацией). Может иметь поля любых типов и модификаторов доступа. Может иметь конструкторы. Может хранить состояние. Класс может наследовать только один абстрактный класс.

Интерфейс — это контракт, определяющий набор методов. До Java 8 содержал только абстрактные методы и константы. С Java 8 — также `default` и `static` методы. С Java 9 — также `private` методы. Поля интерфейса всегда `public static final`. Нет конструкторов. Класс может реализовывать любое количество интерфейсов.

Концептуальное различие: абстрактный класс выражает отношение

"является" (is-a) и используется когда есть общая реализация для группы связанных классов. Интерфейс выражает контракт (can-do, has-ability) и используется для определения поведения, которое могут реализовывать несвязанные классы.

Пример: `Animal` — хороший кандидат для абстрактного класса (у животных есть общие поля: `name`, `age`; общая реализация: `eat()`). `Flyable` — хороший кандидат для интерфейса (летать могут и птицы, и самолёты, и вертолёты — не связанные классы).

Может ли быть конструктор у абстрактного класса и зачем

Да, абстрактный класс может и часто должен иметь конструктор. Нельзя создать экземпляр абстрактного класса напрямую (`new AbstractClass()` вызовет ошибку компиляции), но конструктор абстрактного класса вызывается при создании объекта подкласса.

Зачем это нужно. Во-первых, для инициализации общих полей, объявленных в абстрактном классе. Если у всех животных есть поле `name`, логично инициализировать его в конструкторе `Animal`, а не дублировать в каждом подклассе. Во-вторых, для принудительной передачи необходимых параметров — если конструктор абстрактного класса требует параметры, каждый подкласс обязан их передать через `super(...)`. Это гарантирует корректную инициализацию.

Java

```
abstract class Animal {  
    private final String name;  
  
    protected Animal(String name) { // конструктор есть!  
        this.name = Objects.requireNonNull(name);  
    }  
}
```

```

    public String getName() { return name; }
    abstract void makeSound();
}

class Dog extends Animal {
    Dog(String name) {
        super(name); // обязательный вызов конструктора суперкласса
    }
    void makeSound() { System.out.println("Woof"); }
}

```

Функциональный интерфейс, виды

Функциональный интерфейс — интерфейс, содержащий ровно один абстрактный метод (SAM — Single Abstract Method). При этом он может содержать любое количество default и static методов — они не нарушают правило. Аннотация `@FunctionalInterface` необязательна, но рекомендуется — она заставляет компилятор проверять соответствие правилу SAM.

Функциональные интерфейсы являются целевыми типами для лямбда-выражений и ссылок на методы.

Основные виды из пакета `java.util.function`:

`Function<T, R>` — принимает `T`, возвращает `R`. `apply(T t)`. Трансформация данных.

`BiFunction<T, U, R>` — принимает два аргумента `T` и `U`, возвращает `R`.

`Supplier<T>` — не принимает аргументов, возвращает `T`. `get()`. Фабрика, ленивая инициализация.

`Consumer<T>` — принимает `T`, ничего не возвращает. `accept(T t)`. Побочные эффекты.

`BiConsumer<T, U>` — принимает два аргумента.

`Predicate<T>` — принимает `T`, возвращает `boolean`. `test(T t)`. Фильтрация, условия.

`BiPredicate<T, U>` — два аргумента.

`UnaryOperator<T>` — частный случай `Function<T, T>` (вход и выход одного типа).

`BinaryOperator<T>` — частный случай `BiFunction<T, T, T>`.

Также есть примитивные специализации для избежания автобоксинга: `IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntSupplier`, `IntConsumer`, `IntPredicate` и аналоги для `long` и `double`.

Собственные функциональные интерфейсы: `Runnable` (`run()`), `Callable<V>` (`call()`), `Comparator<T>` (`compare()`).

Могут ли быть реализованные методы в интерфейсе и абстрактном классе

Абстрактный класс — да, может содержать любое количество конкретных (реализованных) методов. Это одна из его основных фиш — предоставлять базовую реализацию. Подклассы могут переопределять конкретные методы или использовать наследованную реализацию.

Интерфейс — до Java 8 только абстрактные методы. Начиная с Java 8 и далее:

default методы (Java 8+) — методы с реализацией по умолчанию, помеченные ключевым словом `default`. Реализующие классы могут переопределить или использовать дефолтную реализацию. Введены для добавления новых методов в существующие интерфейсы без нарушения обратной совместимости. Пример: `forEach()` в `Iterable`, `stream()` в `Collection`.

static методы (Java 8+) — статические методы с реализацией. Не наследуются классами. Вызываются через имя интерфейса: `Comparator.comparing(...)`.

private методы (Java 9+) — приватные вспомогательные методы для повторного использования кода внутри default методов. Не видны снаружи интерфейса и не наследуются.

Java

```
interface MyInterface {  
    void abstractMethod();           // абстрактный  
    default void defaultMethod() { ... } // с реализацией  
    static void staticMethod() { ... } // статический  
    private void privateHelper() { ... } // приватный (Java 9+)  
}
```

Какие модификаторы у полей/методов интерфейса

Поля интерфейса всегда неявно являются `public static final`. Это константы. Даже если написать просто `int MAX = 100;`, компилятор добавит `public static final`. Изменить значение такого поля невозможно.

Методы интерфейса:

Обычные (абстрактные) методы — неявно `public abstract`. Нельзя сделать `private`, `protected` или `package-private` (до Java 9).

default методы — всегда `public` (можно не писать). Нельзя `protected` или `private`.

static методы — всегда `public` (или `private` с Java 9).

private методы (Java 9+) — явно `private`. Не могут быть `abstract`.

private static методы (Java 9+) — явно `private static`.

Таким образом, до Java 9 всё в интерфейсе было `public`. Java 9 добавила возможность приватных методов для внутренней организации кода.

Что такое **static**

Ключевое слово **static** означает, что поле, метод, блок или вложенный класс принадлежат классу, а не конкретному экземпляру. Статические члены существуют в единственном экземпляре и создаются при загрузке класса в JVM.

Статическое поле — одна переменная, общая для всех экземпляров класса. Изменение через один объект видно через все остальные. Хранится в куче, ассоциированной с объектом **Class**.

Статический метод — вызывается через имя класса без создания экземпляра. Не имеет доступа к **this** и не может напрямую обращаться к полям экземпляра и нестатическим методам. Может вызывать только другие статические методы и обращаться к статическим полям.

Статический блок инициализации — выполняется один раз при загрузке класса, до создания любых экземпляров. Используется для инициализации статических полей со сложной логикой.

Статический вложенный класс — не требует экземпляра внешнего класса для создания. Не имеет неявной ссылки на внешний класс. Предпочтительнее нестатического внутреннего класса, если не нужен доступ к полям экземпляра внешнего класса.

Важный нюанс: статический метод не может быть переопределён (только сокрыт — **method hiding**), что нарушит полиморфизм. Аннотация **@Override** неприменима к статическим методам.

Что такое **final** для методов, классов и переменных

final — ключевое слово, означающее "нельзя изменить/расширить/переопределить".

final переменная — после инициализации значение нельзя изменить

(переменная становится константой). Для примитивов — нельзя изменить само значение. Для ссылочных типов — нельзя изменить ссылку (сделать её указывающей на другой объект), но состояние объекта менять можно. Поле `final` экземпляра должно быть инициализировано при объявлении или в конструкторе. Локальная `final` переменная — при объявлении или при первом присвоении.

`final` метод — не может быть переопределён в подклассах. Используется для закрепления реализации (например, критически важной логики безопасности, которую нельзя менять). JVM может встраивать (`inline`) вызовы `final`-методов для оптимизации.

`final` класс — от него нельзя наследоваться. Все его методы неявно становятся `final`. Примеры: `String`, `Integer`, `Double`. Используется для защиты от нарушения инвариантов через наследование и для обеспечения неизменяемости.

Если сделать `HashMap` `final` переменной, то что будет

Если объявить `final HashMap<String, Integer> map = new HashMap<>();`, то это означает, что переменная `map` не может быть переназначена на другой объект. Попытка написать `map = new HashMap<>();` в другом месте вызовет ошибку компиляции.

Однако сам объект `HashMap` остаётся изменяемым. Можно спокойно вызывать `map.put("key", 1)`, `map.remove("key")`, `map.clear()` — всё это изменяет внутреннее состояние объекта, на который ссылается `map`, но не саму ссылку `map`. `final` запрещает изменение ссылки, а не состояния объекта.

Чтобы сделать `Map` действительно неизменяемым, нужно использовать `Collections.unmodifiableMap(map)` или `Map.of(...)` (Java 9+), которые вернут обёртку, выбрасывающую `UnsupportedOperationException` при попытке модификации.

Полиморфизм, виды

Полиморфизм — способность обращаться с объектами разных типов через единый интерфейс. Буквально — "много форм".

Полиморфизм подтипов (включения, runtime polymorphism) — наиболее классический вид в ООП. Переменная типа суперкласса (или интерфейса) может хранить объект подкласса. Вызов метода определяется во время выполнения на основе реального типа объекта (dynamic dispatch, позднее связывание). Пример: `Animal a = new Dog(); a.makeSound();` — вызовется `makeSound()` класса `Dog`, несмотря на то что тип переменной `Animal`.

Java

```
List<Animal> animals = List.of(new Dog(), new Cat(), new Bird());  
animals.forEach(Animal::makeSound); // каждый звучит по-своему
```

Параметрический полиморфизм (Generics) — код написан обобщённо, для параметра типа `T`. Один и тот же класс `ArrayList<T>` работает и с `ArrayList<String>`, и с `ArrayList<Integer>`. Обеспечивает типобезопасность при сохранении обобщённости.

Перегрузка (Ad-hoc полиморфизм, compile-time polymorphism) — несколько методов с одним именем, но разными параметрами. Выбор метода происходит во время компиляции на основе статических типов аргументов. `println(String s)`, `println(int i)`, `println(boolean b)` — разные методы, но одно имя.

Принудительный полиморфизм (Coercion) — неявное или явное преобразование типов. Например, автобоксинг (`int → Integer`), расширяющее преобразование (`int → long`).

JAVA CORE — КОЛЛЕКЦИИ

Иерархия коллекций

Система коллекций Java — две независимые иерархии: `Collection` и `Map`.

Иерархия Collection: Iterable $\rightarrow$ Collection $\rightarrow$ [List, Set, Queue/Deque].

List: ArrayList, LinkedList, Vector (устарел), Stack (устарел), CopyOnWriteArrayList.

Set: HashSet ($\rightarrow$ LinkedHashSet), TreeSet, EnumSet, CopyOnWriteArraySet.

Queue/Deque: LinkedList, ArrayDeque, PriorityQueue; блокирующие: ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue.

Иерархия Map: HashMap ($\rightarrow$ LinkedHashMap), TreeMap, Hashtable (устарел), EnumMap, WeakHashMap, ConcurrentHashMap, ConcurrentSkipListMap.

Почему Map не является коллекцией

Map<K,V> не наследует от Collection<E> по концептуальным причинам. Collection работает с отдельными элементами. Map работает с парами ключ-значение — это фундаментально другая структура. Нет единого типа элемента — есть два типа: ключ и значение. Операции тоже другие: Collection имеет add(element), Map имеет put(key, value). Итерация в Collection — по элементам, в Map — по ключам (keySet()), значениям (values()) или парам (entrySet()). Включить Map в иерархию Collection было бы нарушением принципа подстановки Лисков — Map не может выполнить контракт Collection.

При этом Map является частью Java Collections Framework — он взаимодействует с коллекциями: keySet() возвращает Set, values() возвращает Collection, entrySet() возвращает Set<Entry>.

HashMap — устройство, как работает put()

Внутренняя структура: массив корзин (buckets) Node[] table. Каждая корзина — начало цепочки Node объектов. Каждый Node содержит hash, key, value, next. С Java 8: если длина цепочки ≥ 8 и размер таблицы ≥ 64 — цепочка преобразуется в красно-чёрное дерево (TreeNode), давая $O(\log n)$ вместо $O(n)$.

в худшем случае.

Как работает `put(key, value)`:

Если `key == null` — особый случай, помещается в `bucket[0]`.

Вычисляется `hash = key.hashCode()`, затем применяется спрединг-функция: $h \wedge (h \ggg 16)$ — это перемешивает старшие и младшие биты для лучшего распределения.

Индекс корзины: `index = hash & (capacity - 1)` (работает как `hash % capacity` но быстрее, потому что `capacity` всегда степень двойки).

Если корзина пустая — создаётся новый `Node`, помещается туда.

Если непустая — проходим по цепочке: ищем ключ с тем же `hash` И `equals()`.

Если нашли — заменяем значение. Если не нашли — добавляем в конец цепочки (с Java 8, раньше — в начало).

После вставки проверяем `size > capacity * loadFactor` (default: 0.75). Если да — вызывается `resize()`: таблица удваивается, элементы перехэшируются.

Контракт между `equals` и `hashCode`

Контракт гласит: если `a.equals(b) == true`, то `a.hashCode() == b.hashCode()`.

Обратное не обязательно — разные объекты могут иметь одинаковый `hashCode` (коллизия).

Следствие для `HashMap`: алгоритм поиска сначала находит корзину по `hashCode`, затем перебирает цепочку по `equals`. Если нарушить контракт (переопределить `equals`, не переопределив `hashCode`), два "равных" объекта могут попасть в разные корзины, и `HashMap` не сможет найти один по другому.

Если два объекта равны по `equals` — обязательно ли равен `hashCode`? Да, это прямое требование контракта. Обратное неверно: равный `hashCode` не означает равные объекты (это лишь коллизия). Всегда при переопределении одного нужно переопределять другой.

Почему HashMap не реализует Collection

По тем же причинам, что Map не реализует Collection (см. выше). HashMap реализует Map — это другая иерархия. Нет единого типа элемента, другая семантика операций. Кроме того, `HashMap.containsKey()` и `HashMap.containsValue()` имеют разную сложность ($O(1)$ vs $O(n)$) — это тоже нарушало бы контракт `Collection.contains()`.

LinkedHashMap

`LinkedHashMap` расширяет `HashMap`, добавляя двусвязный список через все записи. Это позволяет сохранять порядок итерации. Есть два режима: `insertion order` (по умолчанию) — итерация в порядке добавления; `access order` (конструктор с `accessOrder = true`) — итерация от наименее к наиболее недавно используемому (LRU). При `accessOrder = true` каждый `get()` и `put()` перемещают запись в конец.

Это делает `LinkedHashMap` идеальной основой для LRU-кеша: достаточно наследоваться и переопределить `removeEldestEntry()`:

Java

```
new LinkedHashMap<K, V>(capacity, 0.75f, true) {  
    protected boolean removeEldestEntry(Map.Entry eldest) {  
        return size() > MAX_SIZE;  
    }  
};
```

Производительность: $O(1)$ для `get/put`, немного медленнее `HashMap` из-за поддержки двусвязного списка.

HashMap vs TreeMap, null в качестве ключа/значения

`HashMap` — хэш-таблица. $O(1)$ для `get/put/remove` в среднем. Не гарантирует порядок итерации. Допускает один `null`-ключ (хранится в `bucket[0]`) и любое количество `null`-значений.

TreeMap — красно-чёрное дерево. $O(\log n)$ для всех операций. Ключи хранятся в отсортированном порядке (естественный или через Comparator). Не допускает null-ключей (нельзя сравнить null при сортировке — NullPointerException). null-значения допускаются. Дополнительные методы: firstKey(), lastKey(), headMap(), tailMap(), subMap(), floorKey(), ceilingKey().

Выбор: HashMap — когда нужна скорость и порядок не важен. TreeMap — когда нужен отсортированный порядок или операции с диапазонами ключей.

Hashtable vs HashMap

Hashtable — устаревший класс из Java 1.0. Все методы синхронизированы (synchronized). Не допускает ни null-ключей, ни null-значений. Наследует от Dictionary. Медленнее из-за синхронизации.

HashMap — введён в Java 2, современный. Не синхронизирован. Один null-ключ, любые null-значения. Быстрее.

Для потокобезопасности в современном коде — ConcurrentHashMap, а не Hashtable. Hashtable сохраняется только для обратной совместимости.

ArrayList vs LinkedList

ArrayList — динамический массив. Доступ по индексу $O(1)$. Добавление в конец $O(1)$ амортизировано. Вставка/удаление в середину $O(n)$ (сдвиг элементов). Отличная локальность кеша (элементы в памяти последовательно). Потребляет меньше памяти.

LinkedList — двусвязный список. Также реализует Deque. Доступ по индексу $O(n)$ (обход от начала/конца). Добавление/удаление в начало и конец $O(1)$. Вставка по итератору $O(1)$. Плохая локальность кеша (узлы разбросаны по памяти). Потребляет больше памяти (каждый узел: значение + 2 ссылки).

На практике ArrayList почти всегда быстрее даже для вставок в середину, потому что System.arraycopy() использует нативный код и выигрывает за счёт

локальности кеша. LinkedList оправдан как Deque (addFirst/addLast/removeFirst/removeLast).

Компараторы (Comparable vs Comparator)

Comparable<T> (пакет java.lang) — интерфейс для определения естественного порядка объектов. Метод int compareTo(T o). Реализуется самим классом. Используется в Collections.sort(), Arrays.sort(), TreeSet, TreeMap без дополнительных аргументов. Один класс — один порядок.

Comparator<T> (пакет java.util) — внешний компаратор для определения произвольного порядка. Метод int compare(T o1, T o2). Определяется отдельно от сравниваемого класса. Можно создавать несколько компараторов для одного класса. Функциональный интерфейс — можно писать лямбдой. Java 8 добавила мощные методы: Comparator.comparing(), thenComparing(), reversed(), naturalOrder(), nullsFirst().

Java

```
// сортировка по возрасту, затем по имени, null первыми
Comparator<Person> c = Comparator.comparingInt(Person::getAge)
    .thenComparing(Person::getName)
    .reversed();
```

Synchronized коллекции

Collections.synchronizedXxx() — обёрточные методы: synchronizedList(), synchronizedMap(), synchronizedSet(). Каждый метод коллекции синхронизируется через synchronized(mutex), где mutex — сам объект-обёртка. Полностью блокирует объект на каждую операцию. При итерации нужна внешняя синхронизация:

Java

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
synchronized (list) {
```

```
Iterator<String> it = list.iterator();  
while (it.hasNext()) { ... }  
}
```

Простое, но медленное решение — весь объект блокируется целиком.

Concurrent коллекции

Коллекции из `java.util.concurrent`, оптимизированные для многопоточного использования:

`ConcurrentHashMap` — сегментированная блокировка (до Java 8), затем блокировка на уровне отдельного bucket (Java 8+). Чтение вообще не блокируется. Высокая пропускная способность. Не допускает null.

`ConcurrentSkipListMap` / `ConcurrentSkipListSet` — неблокирующие, потокобезопасные, отсортированные. На основе skip-list (вероятностная структура данных). $O(\log n)$.

`LinkedBlockingQueue` / `ArrayBlockingQueue` — блокирующие очереди. `put()` блокирует если полна, `take()` блокирует если пуста. Для паттерна "производитель-потребитель".

`ConcurrentLinkedQueue` — неблокирующая очередь на CAS.

CopyOnWriteArrayList

`CopyOnWriteArrayList` — потокобезопасная реализация `List`. При каждой модификации (`add`, `set`, `remove`) создаётся полная копия внутреннего массива. Все читатели работают со старой копией без блокировок — чтение абсолютно потокобезопасно и не требует синхронизации. Итераторы работают на снимке массива на момент создания итератора — не видят последующие изменения, не выбрасывают `ConcurrentModificationException`.

Применение: когда чтений намного больше, чем записей (списки слушателей событий, конфигурационные данные). Запись дорогостоящая — $O(n)$. Не

подходит для частых модификаций.

Дженерики — что это и зачем

Generics (обобщения) — механизм параметрического полиморфизма, позволяющий писать классы, интерфейсы и методы с параметрами типа: `class Box<T>`, `interface Comparator<T>`, `<T> T identity(T t)`.

Зачем: до Generics (до Java 5) коллекции хранили `Object`, и при извлечении нужен был явный каст. Это было источником `ClassCastException` в рантайме. Generics переносят проверку типов на этап компиляции — ошибки типов обнаруживаются раньше и без явных кастов:

Java

// До Java 5:

```
List list = new ArrayList();
```

```
list.add("hello");
```

```
String s = (String) list.get(0); // ClassCastException если не String
```

// С Java 5:

```
List<String> list = new ArrayList<>();
```

```
list.add("hello");
```

```
String s = list.get(0); // без каста, безопасно
```

```
// list.add(42); // ошибка компиляции!
```

Почему стираются типы во время компиляции (Type Erasure)

Причина: обратная совместимость. Когда в Java 5 добавили Generics, нужно было сохранить совместимость с байт-кодом Java 4. Байт-код, написанный без Generics, должен был продолжать работать. JVM была разработана без знания о Generics.

Как работает Type Erasure: компилятор использует информацию о типах для проверки корректности на этапе компиляции, а затем удаляет параметры типов из байт-кода. Параметр типа заменяется его ограничением (`<T>` →

Object, <T extends Comparable> → Comparable). Вставляются необходимые касты автоматически. В рантайме ArrayList<String> и ArrayList<Integer> — это один и тот же класс ArrayList.

Следствия стирания: нельзя new T() (нет информации о типе в рантайме), нельзя new T[10], нельзя instanceof List<String> (можно только instanceof List<?>), нельзя перегружать методы только по параметру Generics. Однако через рефлексию частично можно получить информацию о Generics (через ParameterizedType).

JAVA CORE — СТРОКИ

String иммутабельный — почему

String является неизменяемым по нескольким причинам.

Безопасность пула строк. String Pool (интернирование строк) работает только потому, что строки неизменяемы. Несколько переменных могут безопасно ссылаться на один объект в пуле — если бы строки были изменяемы, изменение через одну ссылку затронуло бы все другие.

Потокобезопасность. Иммутабельные объекты потокобезопасны по своей природе — не нужна синхронизация при чтении из нескольких потоков.

Безопасность как ключей. HashMap использует hashCode ключа при поиске. Если строка-ключ была бы изменена после помещения в HashMap, hashCode изменился бы, и элемент стал бы недоступен. Иммутабельность гарантирует стабильность hashCode.

Кеширование hashCode. Строки кешируют свой hashCode в поле private int hash. Это безопасно только потому, что строка неизменна.

Безопасность. Строки используются для хранения паролей, имён файлов, URL. Иммутабельность защищает от случайного изменения.

Технически: String реализован через `private final byte[] value` (с Java 9) или `private final char[] value`. `final` массив — ссылку нельзя изменить. Значения в массиве технически можно изменить через рефлексия, но это нарушение контракта.

Проблема при конкатенации строк

Поскольку String иммутабельный, каждая конкатенация через `+` в цикле создаёт новый объект строки:

Java

```
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i; // создаётся новый String при каждой итерации!
}
```

Это $O(n^2)$ по времени и по памяти — каждый шаг копирует всю предыдущую строку плюс новый символ. При 10000 итерациях создаётся 10000 промежуточных строк, большинство из которых сразу становятся мусором.

Компилятор оптимизирует `+` вне циклов в одно выражение (через `StringBuilder`), но в цикле — нет. Решение — явно использовать `StringBuilder`:

Java

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i); // модифицирует буфер, нет создания лишних объектов
}
```

```
String result = sb.toString();
```

StringBuilder vs StringBuffer

StringBuilder — изменяемая последовательность символов. Не синхронизирован, не потокобезопасен. Быстрее StringBuffer в однопоточном коде из-за отсутствия накладных расходов на синхронизацию. Введён в Java

5. Использовать в однопоточном коде.

`StringBuffer` — изменяемая последовательность символов. Синхронизирован (все методы `synchronized`). Потокбезопасен. Медленнее из-за синхронизации. Введён в Java 1.0. Устаревший — на практике лучше использовать внешнюю синхронизацию с `StringBuilder` или другие подходы.

Оба имеют одинаковый API: `append()`, `insert()`, `delete()`, `reverse()`, `substring()`. Внутренне хранят символы в массиве `char[]` (до Java 9) или `byte[]` с кодером. Оба `final` классы. `toString()` создаёт новый `String`.

JAVA CORE — МНОГОПОТОЧНОСТЬ

Как создать поток (Thread, Runnable, Callable)

Способ 1 — `extends Thread`: переопределить `run()`, создать экземпляр, вызвать `start()`. Минус: класс не может наследовать другой класс (Java не поддерживает множественное наследование).

Способ 2 — `implements Runnable`: реализовать `run()`, передать в конструктор `Thread`, вызвать `start()`. Предпочтительный способ — отделяет задачу от потока. С лямбдой: `new Thread(() -> doWork()).start()`.

Способ 3 — `implements Callable<V>`: реализовать `call()`, который возвращает результат и может бросать `checked exceptions`. Передаётся в `ExecutorService.submit()`, возвращает `Future<V>`.

`ExecutorService` (рекомендуется в продакшн-коде): `Executors.newFixedThreadPool(4).submit(runnable)` — управляемый пул потоков без ручного создания.

Runnable vs Callable

`Runnable.run()` — не возвращает значение (`void`), не может бросить `checked exception`. Подходит для задач без результата и без проверяемых исключений.

`Callable<V>.call()` — возвращает значение типа `V`, может бросить `Exception`.
Используется с `ExecutorService.submit()`, возвращающим `Future<V>`.
Позволяет получить результат асинхронной задачи и обработать исключения.

volatile — что это и как работает под капотом

`volatile` — ключевое слово, гарантирующее видимость изменений переменной для всех потоков. Без `volatile` каждый процессор может кешировать значение переменной в L1/L2/L3 кеше или регистре, и изменение в одном потоке может быть невидимо другому потоку длительное время.

Под капотом: `volatile` вставляет `memory barrier` (барьер памяти) при записи и чтении. Запись `volatile` поля: flush всех изменений из локального кеша в основную память + запрет переупорядочивания (`reordering`) инструкций. Чтение `volatile` поля: invalidate локального кеша, чтение напрямую из основной памяти.

Таким образом, `volatile` гарантирует `visibility` и `ordering` (частичный `happens-before`), но не атомарность составных операций. `i++` на `volatile int` всё ещё не потокобезопасно — это три операции (`read`, `increment`, `write`), которые могут перемежаться.

Применение: флаг остановки потока (`volatile boolean running`), Double-Checked Locking в Singleton (поле `instance` должно быть `volatile`), публикация объекта (когда один поток создаёт объект, другой читает ссылку на него).

Атомики

Классы из `java.util.concurrent.atomic` обеспечивают атомарные операции без блокировок, используя аппаратные инструкции CAS (`Compare-And-Swap`): `compareAndSwap(expected, newValue)` — атомарно: если текущее значение `== expected`, установить `newValue`, вернуть `true`.

`AtomicInteger`, `AtomicLong` — атомарные счётчики: `incrementAndGet()`,

decrementAndGet(), getAndAdd(), compareAndSet().

AtomicBoolean — атомарный boolean.

AtomicReference<V> — атомарная ссылка на объект. compareAndSet(expected, update).

AtomicIntegerArray, AtomicLongArray, AtomicReferenceArray — атомарные массивы.

LongAdder, DoubleAdder (Java 8+) — высокопроизводительные счётчики для высокой конкуренции. Внутри используют несколько ячеек вместо одной, уменьшая конкуренцию. sum() возвращает итог. Быстрее AtomicLong при высокой конкуренции, но менее точны при одновременном чтении и записи.

Synchronized

synchronized — механизм взаимного исключения через монитор объекта. Гарантирует, что в данный момент только один поток выполняет синхронизированный код для данного монитора. Гарантирует и атомарность, и видимость (полный memory barrier).

Синхронизированный метод экземпляра: монитор — this.
Синхронизированный статический метод: монитор — ClassName.class.
Синхронизированный блок: монитор — указанный объект.

synchronized — реентерантный (реентерабельный): поток, уже удерживающий монитор, может повторно войти в синхронизированный блок на том же мониторе (это позволяет вызывать синхронизированные методы из других синхронизированных методов того же класса).

Можно ли добавить volatile к полю INSTANCE в Singleton

Да, и нужно при использовании паттерна Double-Checked Locking:

Java

```
public class Singleton {  
    private static volatile Singleton instance; // volatile обязателен!  
  
    public static Singleton getInstance() {  
        if (instance == null) {           // первая проверка — без синхронизации  
            synchronized (Singleton.class) {  
                if (instance == null) { // вторая проверка — под блокировкой  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

Без `volatile` возможна ошибка: JVM может переупорядочить инструкции при `new Singleton()` (выделить память → обновить ссылку → вызвать конструктор). Другой поток при первой проверке увидит ненулевую ссылку и использует частично сконструированный объект. `volatile` запрещает такое переупорядочивание.

Heap vs Stack

Stack (Стек) — область памяти для каждого потока (у каждого потока свой стек). Хранит фреймы вызовов: локальные переменные, параметры методов, адреса возврата. Для примитивных локальных переменных — хранит само значение. Для ссылочных типов — хранит ссылку (но объект — в куче). Быстро выделяется/освобождается (просто двигается указатель). Ограниченный размер (обычно 512KB-8MB, настраивается `-Xss`). При переполнении — `StackOverflowError`.

Heap (Куча) — общая область памяти для всех потоков. Хранит все объекты (созданные через `new`) и массивы. Управляется GC. Большой (настраивается

через `-Xmx`). Доступ медленнее из-за GC, синхронизации аллокации. При переполнении — `OutOfMemoryError`.

Ключевое: объекты всегда в `heap`, ссылки на них — в `stack` (если локальные) или в других объектах (если поля).

Что такое монитор

Монитор — примитив синхронизации, встроенный в каждый Java-объект. Это концепция, реализованная через заголовок объекта (`object header` в памяти). Монитор обеспечивает взаимное исключение (`synchronized`), ожидание и уведомление (`wait/notify/notifyAll`).

Состояния монитора: свободен → захвачен потоком (другие блокируются) → поток вызвал `wait()` (монитор освобождается, поток в `wait set`) → поток пробуждается `notify()` (пытается снова захватить монитор). Монитор реентерантен — поток может захватить его несколько раз (счётчик), освобождается только когда счётчик достигает 0.

В JVM реализован через `markword` в заголовке объекта. Оптимизации: `biased locking` (первый поток фиксируется без операций синхронизации), `thin lock` (CAS для незаспорных случаев), `fat lock` (полноценный монитор ОС — только при высокой конкуренции).

Deadlock

Deadlock — ситуация когда два или более потоков бесконечно ждут друг друга: поток А держит ресурс 1 и ждёт ресурс 2, поток В держит ресурс 2 и ждёт ресурс 1. Ни один не может продолжить.

Условия возникновения (условия Коффмана): взаимное исключение (ресурс только у одного), удержание и ожидание (держит один, ждёт другой), нет принудительного отъёма, циклическое ожидание.

Предотвращение: всегда захватывать блокировки в одном порядке;

использовать `tryLock(timeout)` — если не получилось, отступить и попробовать снова; уменьшать число одновременно удерживаемых блокировок; использовать высокоуровневые примитивы (`java.util.concurrent`).

Диагностика: `thread dump (jstack)`, `VisualVM`, `JConsole` — покажут потоки в состоянии `BLOCKED` и кто удерживает нужную блокировку.

parallel stream — что это, как работает под капотом

`parallelStream()` или `stream().parallel()` — создаёт параллельный поток, операции которого выполняются параллельно на нескольких потоках.

Под капотом использует `ForkJoinPool` (конкретно `ForkJoinPool.commonPool()` — общий пул для всего приложения). `ForkJoinPool` реализует паттерн `fork-join`: задача рекурсивно делится (`fork`) на подзадачи, подзадачи выполняются параллельно, результаты объединяются (`join`). Также использует `work-stealing`: если поток свободен, он "крадёт" задачу из очереди другого занятого потока.

Размер `commonPool`: `Runtime.getRuntime().availableProcessors() - 1`. Можно изменить системным свойством `java.util.concurrent.ForkJoinPool.common.parallelism` или использовать кастомный пул.

Параллельный стрим эффективен при: большом количестве элементов, вычислительно дорогих операциях, легко делимых источниках данных (`ArrayList`, массивы). Неэффективен при: малом количестве элементов (`overhead` на разделение > выгода), операциях ввода-вывода, упорядоченных операциях (`forEachOrdered`), небезопасных в многопоточности лямбдах.

Future

`Future<V>` — интерфейс, представляющий результат асинхронной вычислительной операции. Возвращается `ExecutorService.submit(Callable)`.

Методы: `get()` — блокирующее ожидание результата; `get(timeout, unit)` — ожидание с таймаутом; `isDone()` — завершено ли; `cancel(mayInterruptIfRunning)` — отменить задачу; `isCancelled()` — отменена ли.

`CompletableFuture<V>` (Java 8+) — значительно более мощный инструмент. Неблокирующий API с цепочками: `thenApply()`, `thenAccept()`, `thenRun()`, `thenCompose()`, `thenCombine()`. Можно комбинировать несколько: `allOf()`, `anyOf()`. Ручное завершение: `complete(value)`, `completeExceptionally(exception)`. Позволяет строить реактивные цепочки без блокирования потоков.

Виды сборщиков мусора, какой по умолчанию

`Serial GC (-XX:+UseSerialGC)` — однопоточный. Stop-the-world для всех операций. Для однопроцессорных систем или маленьких приложений.

`Parallel GC (-XX:+UseParallelGC)` — многопоточный. Stop-the-world, но параллельно. Высокая пропускная способность. В Java 8 — по умолчанию.

`G1 GC (-XX:+UseG1GC)` — Garbage First. Делит heap на регионы. Старается уложиться в заданное время паузы (`-XX:MaxGCPauseMillis`). Хорошо для больших heap. По умолчанию начиная с Java 9.

`ZGC (-XX:+UseZGC)` — сверхнизкие паузы (< 1ms независимо от размера heap). Большинство операций конкурентные (без stop-the-world). Для требовательных к задержкам приложений. Стабилен с Java 15.

`Shenandoah GC` — аналог ZGC от Red Hat. Конкурентная компактация. Доступен в некоторых JDK.

По умолчанию: Java 8 — `Parallel GC`. Java 9+ — `G1 GC`.

JAVA CORE — STREAM API

Что такое Stream API и для чего

Stream API — функциональный API для обработки последовательностей данных в декларативном стиле. Введён в Java 8. Stream — это конвейер операций над данными, а не структура хранения.

Ключевые свойства: не хранит данные (данные — в источнике: коллекция, массив, генератор); не изменяет источник; ленивые вычисления; одноразовый (после терминальной операции — потреблён); поддержка параллелизма через `parallelStream()`.

Промежуточные и терминальные операции

Промежуточные (intermediate) — возвращают новый Stream. Ленивые — не выполняются пока не вызвана терминальная. Основные: `filter(Predicate)`, `map(Function)`, `flatMap(Function)`, `distinct()`, `sorted()`, `sorted(Comparator)`, `limit(n)`, `skip(n)`, `peek(Consumer)`, `mapToInt/Long/Double()`, `takeWhile(Predicate)` (Java 9), `dropWhile(Predicate)` (Java 9).

Терминальные (terminal) — запускают выполнение конвейера, потребляют Stream. Основные: `forEach(Consumer)`, `collect(Collector)`, `count()`, `reduce()`, `findFirst()`, `findAny()`, `anyMatch()`, `allMatch()`, `noneMatch()`, `min()`, `max()`, `toArray()`, `sum()/average()/summaryStatistics()` (для примитивных потоков).

Зачем нужна ленивая обработка

Lazy evaluation позволяет JVM оптимизировать выполнение конвейера. Элементы обрабатываются по одному через всю цепочку операций, а не поэтапно (сначала все фильтруются, потом все маппируются). Это даёт short-circuit evaluation: `filter().findFirst()` остановится как только найдёт первый подходящий элемент. `limit(n)` с бесконечным источником работает корректно — генерируются только первые n элементов. Экономия памяти — не нужно хранить промежуточные коллекции. Меньше вычислений — не обрабатываются ненужные элементы.

Методы Stream, примеры

Java

```
List<String> names = List.of("Alice", "Bob", "Charlie", "Anna", "David");
```

```
// filter + map + collect
```

```
List<String> result = names.stream()  
    .filter(s -> s.startsWith("A"))  
    .map(String::toUpperCase)  
    .collect(Collectors.toList()); // [ALICE, ANNA]
```

```
// reduce
```

```
int sum = IntStream.rangeClosed(1, 100).reduce(0, Integer::sum); // 5050
```

```
// flatMap
```

```
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));  
List<Integer> flat = nested.stream()  
    .flatMap(Collection::stream)  
    .collect(Collectors.toList()); // [1,2,3,4]
```

```
// groupingBy
```

```
Map<Integer, List<String>> byLength = names.stream()  
    .collect(Collectors.groupingBy(String::length));
```

```
// sorted + distinct + limit
```

```
List<Integer> nums = Stream.of(3,1,4,1,5,9,2,6)  
    .distinct().sorted().limit(4)  
    .collect(Collectors.toList()); // [1,2,3,4]
```

orElse vs orElseGet

orElse(T value) — принимает уже вычисленное значение.
Выражение-аргумент вычисляется всегда, даже если Optional не пустой.

Java

```
Optional<String> opt = Optional.of("hello");
```

```
String result = opt.orElse(expensiveOperation()); // expensiveOperation()  
ВСЕГДА вызывается!
```

`orElseGet(Supplier<T> supplier)` — принимает функцию (`Supplier`), которая вызывается только если `Optional` пустой:

Java

```
String result = opt.orElseGet(() -> expensiveOperation()); // вызывается ТОЛЬКО  
если opt пустой
```

Вывод: если значение по умолчанию дорогостоящее (запрос к БД, создание объекта) — всегда используйте `orElseGet()`. Если это простая константа — разницы практически нет, но `orElseGet()` семантически точнее.

ПАТТЕРНЫ

Singleton — рассказать, как создаётся

Singleton гарантирует единственный экземпляр класса. Реализации:

Eager initialization — создаётся при загрузке класса:

Java

```
public class Singleton {  
    private static final Singleton INSTANCE = new Singleton();  
    private Singleton() {}  
    public static Singleton getInstance() { return INSTANCE; }  
}
```

Lazy initialization через Holder (наилучший вариант без `volatile`):

Java

```
public class Singleton {
```

```

private Singleton() {}
private static class Holder {
    static final Singleton INSTANCE = new Singleton();
}
public static Singleton getInstance() { return Holder.INSTANCE; }
}

```

Holder загружается только при первом вызове getInstance(). Потокбезопасно гарантией JVM о корректной инициализации класса.

Enum (рекомендован Effective Java):

Java

```

public enum Singleton { INSTANCE; public void doWork() {} }

```

Потокобезопасный Singleton

Все три варианта выше являются потокобезопасными. Дополнительно — Double-Checked Locking с volatile:

Java

```

public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

}

volatile здесь обязателен для предотвращения проблемы частично сконструированного объекта (reordering при создании объекта).

Chain of Responsibility

Паттерн передаёт запрос по цепочке обработчиков. Каждый обработчик решает обработать или передать дальше. Компоненты: Handler (интерфейс с методом `handle(request)` и ссылкой на следующий), ConcreteHandler (конкретный обработчик).

В Java: Servlet Filters (`FilterChain.doFilter()`), логирование (Logger → parent Logger), обработчики исключений, middleware в веб-фреймворках. Позволяет динамически формировать цепочку и добавлять обработчики без изменения клиентского кода.

Factory — почему нельзя создать объект через конструктор

Factory Method / Abstract Factory скрывают логику создания объектов. Почему нельзя всегда создавать через конструктор:

Конструктор не может вернуть подтип (`new Shape()` всегда вернёт Shape, но фабричный метод может вернуть Circle или Rectangle). Конструктор не может вернуть кешированный объект (Factory может вернуть существующий из кеша — как в `Integer.valueOf()`). Конструктор имеет фиксированное имя класса — трудно выбирать реализацию в рантайме. Конструктор затрудняет тестирование — нельзя подменить реализацию. Factory Method позволяет подклассам переопределить, какой объект создавать.

Prototype

Создание нового объекта путём копирования существующего. Полезно когда создание "с нуля" дорогостоящее, а копирование — дешевле. В Java через `Object.clone()` + `Cloneable`.

Мелкое копирование (shallow): `super.clone()` — копирует поля побитово. Для

примитивов достаточно, для ссылочных типов — обе копии ссылаются на одни объекты. Глубокое копирование (deep): вручную клонировать каждое ссылочное поле. Альтернативы: конструктор копирования (`new MyClass(original)`), сериализация/десериализация.

Proxy — что это и 2 вида

Proxy — объект-заместитель, контролирующий доступ к другому объекту. Реализует тот же интерфейс, что и реальный объект, и делегирует ему вызовы, добавляя дополнительное поведение (кеширование, логирование, контроль доступа, lazy loading).

JDK Dynamic Proxy (`java.lang.reflect.Proxy`): работает только с объектами, реализующими интерфейс. Создаётся через `Proxy.newProxyInstance(classLoader, interfaces, invocationHandler)`. Все вызовы перехватываются `InvocationHandler.invoke()`. Встроен в JDK. Используется Spring AOP когда бин реализует интерфейс.

CGLib Proxy: работает с любыми классами (не обязательно имеющими интерфейс). Создаёт подкласс через генерацию байт-кода (с помощью библиотеки CGLib / ByteBuddy). Переопределяет методы, вставляя перехватчик. Ограничение: не может проксировать `final`-классы и `final`-методы. Используется Spring AOP по умолчанию (Spring Boot с `proxyTargetClass=true`).

Какие паттерны знаешь

Порождающие: Singleton, Factory Method, Abstract Factory, Builder, Prototype, Object Pool.

Структурные: Adapter, Decorator, Proxy, Facade, Composite, Bridge, Flyweight.

Поведенческие: Strategy, Observer, Command, Chain of Responsibility, Template Method, Iterator, State, Mediator, Visitor, Memento, Interpreter.

SPRING

Разница между Spring и Spring Boot

Spring Framework — огромная экосистема модулей (Spring Core, Spring MVC, Spring Data, Spring Security, Spring AOP). Требуется ручной конфигурации: настройка `DispatcherServlet`, указание зависимостей, написание `@Configuration`-классов.

Spring Boot — поверх Spring, реализует "convention over configuration". `AutoConfiguration` автоматически настраивает компоненты по наличию JAR в `classpath`. Встроенный сервер (Tomcat). `Starters` — управляемые наборы зависимостей. `application.properties` для всей конфигурации. `Actuator` для мониторинга. Не заменяет Spring — использует его внутри, убирая boilerplate.

Spring принципы

Spring построен на: IoC (Inversion of Control) — контейнер управляет жизненным циклом объектов; DI (Dependency Injection) — зависимости внедряются контейнером; AOP (Aspect-Oriented Programming) — разделение сквозных интересов (транзакции, логирование, безопасность) через аспекты; POJO-based — компоненты — обычные Java-объекты без наследования от фреймворковых классов; Convention over Configuration (особенно в Spring Boot).

Что такое Bean

Bean — объект, управляемый Spring IoC контейнером (`ApplicationContext`). Контейнер создаёт bean, конфигурирует, внедряет зависимости, управляет жизненным циклом. Объявляется через `@Component` (+ производные), `@Bean` в `@Configuration`-классе, или XML.

Жизненный цикл Bean

Подробно: загрузка `BeanDefinition` → `BeanFactoryPostProcessor` (обработка метаданных) → `instantiation` (создание объекта) → `dependency injection` (поля, сеттеры) → Aware-интерфейсы (`setBeanName`, `setApplicationContext`) →

BeanPostProcessor.postProcessBeforeInitialization() → @PostConstruct →
InitializingBean.afterPropertiesSet() → init-method →
BeanPostProcessor.postProcessAfterInitialization() (здесь создаются
AOP-прокси) → Bean готов → ... → @PreDestroy → DisposableBean.destroy()
→ destroy-method.

Dependency Injection и Inversion of Control

IoC — принцип: контроль над созданием объектов и их зависимостями передаётся фреймворку (контейнеру). Не ваш код создаёт и связывает объекты — контейнер делает это за вас.

DI — конкретная реализация IoC. Зависимости передаются объекту извне: через конструктор (рекомендуется — явные зависимости, поля можно сделать final, легко тестировать), через сеттер (для опциональных зависимостей), через поле @Autowired (не рекомендуется — нарушает инкапсуляцию, сложнее тестировать).

Как Spring внедряет объекты

Spring использует рефлексию. При создании бина ApplicationContext анализирует класс через рефлексию: находит конструктор (если @Autowired или единственный), сеттеры с @Autowired, поля с @Autowired. Для каждой зависимости ищет подходящий бин в контексте по типу (byType), при неоднозначности — по имени (byName) или через @Qualifier. Для field injection вызывает Field.setAccessible(true) и Field.set(bean, dependency).

Bean Post Processor

BeanPostProcessor — специальный интерфейс с двумя методами: postProcessBeforeInitialization(bean, beanName) — вызывается для каждого бина до @PostConstruct; postProcessAfterInitialization(bean, beanName) — вызывается для каждого бина после инициализации. Оба метода возвращают объект — можно вернуть оригинальный или подменённый (обёрнутый в прокси). Именно здесь Spring создаёт AOP-прокси (в postProcessAfterInitialization). Пример использования: автоматическая

обработка кастомных аннотаций, логирование, создание прокси.

SOLID

S — Single Responsibility: каждый класс — одна причина для изменения. Не смешивать бизнес-логику с логированием, сохранением и отображением.

O — Open/Closed: открыт для расширения (новый класс), закрыт для модификации (не меняем существующий код). Через интерфейсы, наследование, паттерны Strategy/Decorator.

L — Liskov Substitution: объект подтипа должен заменять объект суперкласса, не нарушая корректности программы. Нельзя в подклассе нарушать контракт суперкласса.

I — Interface Segregation: много специализированных интерфейсов лучше, чем один "толстый". Классы не должны реализовывать методы, которые им не нужны.

D — Dependency Inversion: модули высокого уровня не должны зависеть от модулей низкого уровня — оба должны зависеть от абстракций. Абстракции не зависят от деталей. Основа DI.

БД

Реляционные vs нереляционные БД

Реляционные (SQL): данные в таблицах со строгой схемой. ACID-транзакции. Мощный SQL. Примеры: PostgreSQL, MySQL, Oracle, SQL Server. Хороши для структурированных данных с чёткими связями, транзакционных систем.

Нереляционные (NoSQL): гибкая или отсутствующая схема. Разные модели хранения: документы (MongoDB — JSON-документы), ключ-значение (Redis), колоночные (Cassandra, HBase), графовые (Neo4j). Горизонтальное масштабирование. BASE вместо ACID. Хороши для больших объёмов слабоструктурированных данных, высоконагруженных систем.

ACID principles

Atomicity — транзакция либо полностью выполнена, либо полностью откатана. Нет промежуточных состояний.

Consistency — транзакция переводит БД из одного корректного состояния в другое. Все constraints соблюдены.

Isolation — параллельные транзакции не влияют на результаты друг друга (уровень влияния определяется уровнем изоляции).

Durability — зафиксированные изменения сохраняются постоянно даже при сбое системы (через WAL — write-ahead log).

Transaction Isolation Levels

Проблемы: Dirty Read (чтение незафиксированных данных), Non-Repeatable Read (повторное чтение той же строки даёт другой результат), Phantom Read (повторный запрос возвращает другой набор строк).

READ UNCOMMITTED — ничего не защищает. Все три проблемы возможны.

READ COMMITTED — нет dirty read. Non-repeatable и phantom возможны. Default в PostgreSQL, Oracle.

REPEATABLE READ — нет dirty и non-repeatable read. Phantom возможен. Default в MySQL InnoDB.

SERIALIZABLE — все проблемы исключены. Транзакции выполняются как последовательные. Самый медленный.

Виды JOIN

INNER JOIN — только строки с совпадением в обеих таблицах.

LEFT JOIN — все строки левой + совпадения правой (null если нет совпадения).

RIGHT JOIN — все строки правой + совпадения левой.

FULL OUTER JOIN — все строки обеих таблиц.

CROSS JOIN — декартово произведение ($N \times M$ строк).

SELF JOIN — таблица соединяется сама с собой (иерархии).

WHERE vs HAVING

WHERE — фильтрует строки до группировки. Не может использовать агрегатные функции.

HAVING — фильтрует группы после GROUP BY. Может использовать COUNT, SUM, AVG и другие.

Порядок выполнения: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Индексы

Индекс — структура данных для ускорения поиска. Без индекса — full table scan $O(n)$. С B-tree индексом — $O(\log n)$. Создаются автоматически для PK и UNIQUE. Ускоряют SELECT с WHERE, JOIN, ORDER BY. Замедляют INSERT/UPDATE/DELETE (нужно обновлять индексы). Занимают место на диске.

Clustered vs Non-Clustered индексы

Clustered (кластеризованный) — определяет физический порядок хранения строк в таблице. Строки хранятся на диске в порядке ключа. Один на таблицу (строки могут быть только в одном порядке). В InnoDB — PRIMARY KEY является clustered index. Очень быстр для range queries по PK.

Non-Clustered (некластеризованный) — отдельная структура, содержащая ключ индекса + указатель на строку (row locator: RID или PK). Несколько на таблицу. Дополнительный lookup для получения полных данных строки (index scan → table lookup), кроме covering index.

Covering index — некластеризованный индекс, содержащий все необходимые запросы столбцы. Тогда table lookup не нужен.

Hash vs B-tree индексы

B-tree — сбалансированное дерево. $O(\log n)$. Поддерживает точное совпадение, range queries ($>$, $<$, BETWEEN), сортировку, prefix matching (LIKE 'abc%'). Универсальный. Default в PostgreSQL и MySQL.

Hash — хэш-таблица. $O(1)$ для точного совпадения. Не поддерживает range queries, сортировку. Быстрее B-tree для точного поиска. В PostgreSQL есть Hash-индексы. В MySQL InnoDB — только Memory-таблицы.

Оптимистичная vs пессимистичная блокировка

Пессимистичная блокировка — предполагает конфликты, блокирует ресурс на всё время работы с ним. SELECT ... FOR UPDATE — блокирует строки. Другие транзакции ждут. Надёжно, но снижает конкурентность. Хороша при высокой вероятности конфликтов.

Оптимистичная блокировка — предполагает редкость конфликтов. Не блокирует при чтении. При обновлении проверяет, не изменились ли данные с момента чтения (через version/timestamp поле). Если изменились — ошибка, клиент повторяет. Высокая конкурентность, но нужна обработка конфликтов.

Если надо сделать много изменений: зависит от конкуренции. При высокой вероятности конфликтов (одни данные меняются многими) — пессимистичная. При низкой вероятности конфликтов — оптимистичная (не тратим ресурсы на блокировки). Для batch-обновлений одним потоком — пессимистичная (гарантия консистентности).

EXPLAIN vs EXPLAIN ANALYZE

EXPLAIN SELECT ... — показывает план выполнения без выполнения запроса. Показывает estimated cost, estimated rows, алгоритмы (Seq Scan, Index

Scan, Hash Join и другие).

EXPLAIN ANALYZE SELECT ... — выполняет запрос и показывает реальное время выполнения каждого шага, реальное количество строк. Позволяет сравнить estimated vs actual и найти причины медленной работы.

Смотреть на: Seq Scan на больших таблицах (нет индекса или неиспользуемый), расхождение estimated rows и actual rows (устаревшая статистика — ANALYZE), высокий cost в конкретных узлах.

Нормализация

Процесс организации данных для устранения избыточности.

1NF — атомарные значения, уникальные строки, нет повторяющихся групп.

2NF — 1NF + нет частичных зависимостей (не-ключевые атрибуты зависят от всего ключа).

3NF — 2NF + нет транзитивных зависимостей (не-ключевой атрибут не зависит от другого не-ключевого).

BCNF — для каждой нетривиальной FD $X \rightarrow Y$, X — суперключ.

На практике нормализуют до 3NF. Денормализация — намеренное добавление избыточности для производительности чтения.

View (материализованные vs нематериализованные)

Обычный View — виртуальная таблица. Не хранит данные — каждое обращение выполняет запрос. Актуальные данные всегда. Может быть медленным для сложных запросов.

Materialized View — реально хранит результат запроса. Быстрое чтение. Нужно явное обновление (REFRESH MATERIALIZED VIEW) — данные могут быть устаревшими. Используется для тяжёлых аналитических запросов, агрегаций. PostgreSQL поддерживает. В MySQL нет нативной поддержки (эмулируется через таблицы).

Команда UNION

UNION — объединяет результаты двух SELECT-запросов. Удаляет дубликаты (аналог DISTINCT). UNION ALL — не удаляет дубликаты, быстрее. Оба запроса должны иметь одинаковое количество столбцов и совместимые типы. Порядок столбцов в результате определяется первым запросом.

GIT

Merge vs Rebase

Merge — создаёт merge-коммит с двумя родителями. История нелинейна, но отражает реальный ход разработки. Безопасно для публичных веток.

Rebase — переносит коммиты ветки поверх другой базы. История линейна. Хэши коммитов изменяются — история переписывается. Нельзя ребейзить публичные ветки (у других конфликт при pull). Используется для "подтягивания" фичевой ветки к актуальному main перед мержем.

Revert vs Reset

Revert — создаёт новый коммит, отменяющий изменения указанного. История не переписывается. Безопасно для публичных веток.

Reset — перемещает HEAD (и ветку) назад. Переписывает историю. Три режима: --soft (изменения в staging area), --mixed (изменения в working directory, default), --hard (изменения удаляются). Безопасно только для локальных, ещё не опубликованных коммитов.

Cherry-pick

git cherry-pick <commit-hash> — применяет изменения конкретного коммита из одной ветки в текущую. Создаётся новый коммит с теми же изменениями (но другим хэшем). Полезно для перемещения отдельного фикса из develop в release-ветку. git cherry-pick A..B — применить диапазон коммитов.

Что такое коммит и ветка

Коммит — снимок состояния репозитория. Содержит: SHA-1 хэш, tree (файловая система), parent(s) (предыдущие коммиты), автор, дата, сообщение. Коммиты образуют граф (DAG — Directed Acyclic Graph).

Ветка — лёгкий указатель (текстовый файл с хэшем коммита) на конкретный коммит. При новом коммите ветка автоматически движется вперёд. Создание ветки — запись 40 символов.

Как перенести коммиты с одной ветки на другую

`git cherry-pick <hash>` — перенести конкретный коммит.

`git cherry-pick A..B` — диапазон коммитов (без A, до B включительно).

`git rebase feature onto main` — перенести все коммиты ветки feature поверх main.

`git merge feature` — принести все изменения (merge-коммит).

Fetch vs Pull

Fetch — загружает изменения из remote в локальный репозиторий, обновляет remote-tracking ветки (origin/main). Рабочая директория и текущая ветка не изменяются. Безопасно.

Pull = fetch + merge (или rebase при pull.rebase=true). Загружает и сразу интегрирует. Может вызвать конфликты.

HTTP/WEB

Методы HTTP, разница GET и POST

GET — получение ресурса. Параметры в URL. Кешируется. Идемпотентен и безопасен. Ограничение по длине URL. Данные видны в URL, истории, логах.

POST — отправка данных для обработки. Данные в теле. Не кешируется. Не идемпотентен. Нет ограничения по размеру данных. Безопаснее для конфиденциальных данных (не в URL).

PUT — полное обновление ресурса. Идемпотентен. PATCH — частичное обновление. DELETE — удаление. Идемпотентен. HEAD — как GET, но без тела ответа. OPTIONS — узнать поддерживаемые методы. CONNECT — туннель (для HTTPS). TRACE — диагностика.

JWT

JWT (JSON Web Token) — стандарт для безопасной передачи данных между сторонами. Состоит из трёх частей, разделённых точкой: Header.Payload.Signature.

Header — алгоритм и тип: {"alg": "HS256", "typ": "JWT"} → Base64Url.

Payload — claims (утверждения): {"sub": "userId", "exp": 1234567890, "roles": ["ADMIN"]} → Base64Url. Не зашифрован — только закодирован! Не хранить секреты.

Signature — HMACSHA256(base64Header + "." + base64Payload, secret). Позволяет проверить подлинность и целостность токена.

Применение: stateless аутентификация в REST API. Клиент получает JWT при логине, отправляет в заголовке Authorization: Bearer <token> при каждом запросе. Сервер проверяет подпись — не нужно хранить сессии. Недостатки: нельзя отозвать до истечения (если не вести blacklist), при компрометации secret — все токены скомпрометированы.

Cookies — что хранится

Cookie — небольшой фрагмент данных (строка), который сервер устанавливает в браузере, а браузер отправляет с каждым запросом к тому же

домену.

Что хранится: Session ID (JSESSIONID для Java) — для идентификации сессии пользователя. Authentication tokens. Пользовательские предпочтения (язык, тема). Аналитика (Google Analytics).

Атрибуты безопасности: HttpOnly — недоступен из JS, защита от XSS. Secure — только по HTTPS. SameSite (Strict/Lax/None) — защита от CSRF.

MVC

Model-View-Controller — архитектурный паттерн разделения ответственности.

Model — данные и бизнес-логика. Независим от UI.

View — отображение данных пользователю (HTML, JSON, XML). Зависит от Model, не содержит логики.

Controller — обрабатывает запросы пользователя, обновляет Model, выбирает View. В Spring MVC — @Controller с @RequestMapping. DispatcherServlet — front controller, распределяет запросы.

Как в Java подключиться к БД

Через JDBC: добавить JDBC-драйвер в classpath. Connection conn = DriverManager.getConnection(url, user, password). Создать Statement или PreparedStatement. Выполнить запрос. Обработать ResultSet. Закрыть ресурсы (через try-with-resources).

В Spring через DataSource (пул соединений HikariCP) + Spring Data JPA / JdbcTemplate:

Java

```
// application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=user
spring.datasource.password=pass
spring.jpa.hibernate.ddl-auto=update
```

Что за фреймворк Mockito

Mockito — фреймворк для мокирования (imitation) объектов в unit-тестах. Позволяет создавать mock-объекты — заглушки, имитирующие поведение зависимостей, и проверять взаимодействие с ними.

Основные возможности: `@Mock` / `Mockito.mock(Class)` — создать мок. `when(mock.method()).thenReturn(value)` — настроить поведение (stubbing). `verify(mock).method()` — проверить что метод был вызван. `@Spy` — реальный объект с возможностью выборочного мокирования. `@InjectMocks` — создать тестируемый объект и внедрить в него моки. `@Captor` — перехватить аргументы вызова.

Интеграция с JUnit 5: `@ExtendWith(MockitoExtension.class)` — активирует обработку аннотаций Mockito.

Java

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock UserRepository repository;
    @InjectMocks UserService service;

    @Test
    void shouldFindUser() {
        when(repository.findById(1L)).thenReturn(Optional.of(new User("Alice")));
        User user = service.findById(1L);
        assertEquals("Alice", user.getName());
    }
}
```

```
        verify(repository).findById(1L);
    }
}
```